Hochschule Worms
Fachbereich Informatik
- B.Sc. in Wirtschaftsinformatik-

GRACE-Konfigurationsprozess: Erhebung, Dokumentation und Optimierung mittels systematischer Wissenserfassung und LLM-gestützter Automatisierung

Bachelorarbeit im Unternehmen

seitcom GmbH

vorgelegt von

Aleksandar Topić (677502)

Erstbetreuer:	Prof. Dr. Marie-Luise Sessler
Zweitbetreuer:	Prof. Dr. Paul Langer
Betrieblicher Betreuer:	Dipl. Inf. Daniel Schwertner
Bearbeitungszeitraum:	14. Oktober 2025 - 5. Februar 2026
Abgabedatum:	5. Februar 2026
Sperrvermerk:	Ja

EIDESSTATTLICHE ERKLÄRUNG

Ich versichere hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt oder mit einem entsprechenden Quellennachweis versehen worden.

Die Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Worms, den 5. Februar 2026

Aleksandar Topić

SPERRVERMERK

Die vorliegende Arbeit beinhaltet interne, vertrauliche Informationen der Firma *seitcom GmbH*. Die Weitergabe des Inhalts dieser Arbeit bzw. eventuell beiliegender Zeichnungen und Daten im Gesamten oder in Teilen ist grundsätzlich untersagt.
Es dürfen keinerlei Kopien oder Abschriften angefertigt werden, auch nicht in digitaler Form. Ausnahmen bedürfen der schriftlichen Genehmigung der Firma *seitcom GmbH*.

INHALTSVERZEICHNIS

1 Abstract . . . 4
2 Einleitung . . . 4
2.1 Problemstellung . . . 5
2.2 Forschungsfrage und Zielsetzung . . . 6
2.3 Aufbau der Arbeit . . . 7
3 Grundlagen . . . 8
3.1 Advanced Planning and Scheduling (APS) . . . 9
3.2 Entity-Relationship-Modellierung . . . 10
3.3 Prozessmodellierung mit BPMN . . . 10
3.4 Wissensmanagement . . . 11
3.5 Large Language Models (LLMs) . . . 12
3.6 Datenhoheit und Compliance . . . 13
3.7 Skalenentwicklung und qualitative Inhaltsanalyse . . . 14
4 Ist-Analyse . . . 14
4.1 Teamstruktur . . . 14
4.2 Aktuelle Konfigurationsprozesse . . . 15
4.3 Identifizierte Probleme . . . 16
4.4 Demo-Werk als empirische Grundlage . . . 17
5 Methodik . . . 17
5.1 Aktionsforschung als methodischer Ansatz . . . 17
5.2 Forschungszyklus und Iterationen . . . 19
5.3 Datenauswertung . . . 23
5.4 Drei-Artefakte-Architektur als methodisches Framework . . . 25
5.5 Technische Infrastruktur und Datenhoheit . . . 26
5.6 Evaluationsdesign quantitativer Probleme . . . 26
5.7 Limitierungen der Methodik . . . 30
6 Implementierung . . . 31
6.1 Demo-Werk: Synthetische Konfiguration . . . 31
6.2 GRACE Onboarding Fragebogen . . . 34
6.3 Entity Mapping Workbook: Technische Spezifikation . . . 38
6.4 Standard Operating Procedures (SOPs) . . . 44
6.5 Master Creator: Architektur und Funktionsweise . . . 48
6.6 Validierung und Qualitätssicherung . . . 53
6.7 Iterative Entwicklung . . . 57
6.8 BPMN-Prozessmodelle . . . 63
7 Evaluation . . . 68

7.1 Problemlösung durch Artefakte - Übersicht 69
7.2 Fragmentierung des Konfigurationswissens 69
7.3 Wissensträgerabhängigkeit 70
7.4 Fehlende Standardisierung 71
7.5 Datenhoheit 71
7.6 Quantitative Problemevaluation 72
7.7 NAS-Crash-Rekonstruktion 74
7.8 Limitationen 74
7.9 Kritische Reflexion 76
7.10 Einordnung in den Stand der Forschung 76
8 Fazit 77
9 Ausblick 78
Literaturverzeichnis 79
Anhang 83

1 ABSTRACT

Das Partnerunternehmen ist auf dem Weg zum Break-Even, jedoch ist der Weg dorthin mit sehr viel Konfigurationsarbeit verbunden. GRACE die Feinplanungsoptimierungsoftware ist das Steckenpferd des Unternehmens und soll mit ihrem Vertrieb genau dies erreichen. GRACE kann hochkomplexe Sachverhalte realitätsnah abbilden, jedoch nicht einfach ohne Weiteres. Deshalb muss eine Vorgehensweise zur Optimierung des Konfigurationsprozesses gefunden werden. Der Master Creator ist ein Versuch das im Weg stehende Hindernis zu umgehen, das sogenannte Fragmentierte Wissen. Fragmentiertes Wissen ist Wissen, welches an keinem Ort in strukturierter Form vorliegt [Polanyi, 1966, Nonaka and Takeuchi, 1995].. Aus den bisherigen Konfigurationsarbeiten ließ sich ableiten, dass das Konfigurationswissen meistens auf aufwendige Weise vor der Konfiguration erhoben und strukturiert werden muss.
Mit dem Master Creator, einem KI-gestützten Tool soll die Strukturarbeit beschleunigt und automatisiert werden. Zur Entwicklung des Tools wird die Action-Research-Methode angewendet innerhalb der Zyklen dieser soll eine Drei-Artefakte-Architektur entwickelt werden [Lewin, 1946]. Die Drei-Artefakte-Architektur dient als Analysegrundlage zum Ableiten eines Konzepts für die Anforderungen des Master Creators. Jedoch sollen die Artefakte auch selbst der Opimierung der Produktion dienen.
Genauer gesagt wird untersucht wie fragmentiertes Wissen in LLM-ausführbare Strukturen transformiert werden kann. Folgende 3 Artefakte legen den Grundstein dazu: der GRACE Onboarding Fragebogen (nach DeVellis [2017]), das Entity Mapping Workbook (basierend auf Chen [1976]) und die Standard Operating Procedures. Aus dieser Wissensbasis entsteht der Master Creator, ein chatbasiertes LLM-Tool zur Generierung von GRACE-Konfigurationen mit Anbindung an ein lokales 7B-Modell zur Wahrung der Datensouveränität [Hummel et al., 2021, Gupta et al., 2025].
Ergebnisse:
Ein kontrolliertes Experiment mit einer Stichprobengröße von n=18, validiert die Wirksamkeit des LLM-Tools durch folgende Werte: 47% Zeitreduktion, 81% Fehlerreduktion, Vollständigkeitssteigerung von 60-70% auf 98,1%. Die zweistufige Analyse offenbart: Wissensstrukturierung verbessert die Vollständigkeit auf 86,1%. LLM-Automatisierung verstärkt die Wirkung auf 98,1%. Das lokal gehostede 7B-Modell erreicht trotz Hardware-Limitationen referenzielle Integrität über 6 Entitätstypen.
Beitrag:
Die Arbeit validiert LLM-gestützte Konfigurationsansätze für APS-Systeme unter realen Produktionsbedingungen. Sie demonstriert, dass Systematische Externalisierung von Wissen notwendig ist, wenn man eine erfolgreiche LLM-Automatisierung in komplexen Enterprise-Domänen produzieren will.

2 EINLEITUNG

GRACE (Green Renewable AI-powered Computing Environment) ist ein Advanced Planning and Scheduling (APS)-Softwaresystem der seitcom GmbH, das die Welt der Industrie komplett revolutionieren möchte. Das Partnerunternehmen möchte mit GRACE

nicht wie typische Repräsentanten von APS-Produkten einfach nur eine Ergänzung der Planungsfunktionen von Enterprise-Resource-Planning (ERP)-Systemen darstellen, sondern die Notwendigkeit der manuellen Planung komplett ablösen. Dieses Vorhaben soll unter Berücksichtigung aller in der Produktion relevanten Constraints CO_2-optimiert ablaufen. Ein hochkomplexer Prozess, der einen hohen Aufwand für den Konfigurationsprozess des Systems rechtfertigt. Deshalb besteht beim Partnerunternehmen ein starkes Bedürfnis diesen Prozess zu optimieren und zu verbessern. Jeder Prozent an Effizienz den die Konfiguration dazu gewinnt, bedeutet früheres Deployment und einen früheren Erfolg der Software. Diese Arbeit möchte einen Grundstein für die Optimierung des Konfigurationsprozesses legen. In den folgenden Abschnitten der Einleitung wird der Gegenstand dieser Arbeit noch genauer erläutert.

Zum Zeitpunkt dieser Arbeit befindet sich GRACE in der Pilotphase mit sieben Kundenwerken. Dies impliziert evolvierende Datenstrukturen, fragmentierte Dokumentation und intensive Betreuung durch ein kleines Konfigurationsteam von 5 Personen. Die Pilotphase ist durch eine charakteristische Eigenschaft geprägt: Formale Dokumentation existiert kaum, stattdessen fungiert der Quellcode selbst als primäre Dokumentationsquelle („Code as Documentation“). Diese Situation stellt besondere Anforderungen an die Wissenserfassung und -weitergabe.

2.1 PROBLEMSTELLUNG

Vor dem produktiven Einsatz von GRACE muss für jeden Kunden eine initiale Konfiguration erstellt werden. Diese umfasst sechs grundlegende Entitätstypen: Materials (Roh-, Hilfs- und Betriebsstoffe), Machines (Produktionsressourcen), Products (Endprodukte), Processes (Fertigungsschritte), BOMs (Stücklistenstrukturen) und Recipes (Verknüpfungen zwischen Material, BOM und Prozess). Die sechs Entitäten stellen die Kernkonfiguration. Weitere existierende Entitätstypen werden im Rahmen dieser Arbeit nicht behandelt, um den zeitlichen Rahmen einzuhalten. Die Abhängigkeiten zwischen den Grundentitäten erfordern eine spezifische Erstellungsreihenfolge und die Einhaltung referenzieller Integrität: Ein Recipe kann beispielsweise erst erstellt werden, wenn die referenzierten Materials, BOMs und Processes bereits existieren. Im Status quo erfordert diese initiale Konfiguration bis zu acht Wochen Durchlaufzeit. Die Herausforderung ist dabei zweifacher Natur: Einerseits besteht eine beachtliche *technische Komplexität* durch die JSON-Strukturen mit ihren Abhängigkeiten, Validierungsregeln und Datentypen. Andererseits liegt das fundamentale Problem in der *Fragmentierung des Konfigurationswissens.* Relevante Informationen sind über Azure DevOps Wikis, E-Mail-Korrespondenzen, Git-Historien, interne Dokumente und persönliche Erfahrung verteilt. Diese Situation entspricht dem von Nonaka and Takeuchi [1995] beschriebenen Problem der Wissensexternalisierung: Experten verfügen über implizites Wissen („tacit knowledge“), das schwer artikulierbar und transferierbar ist. Dies ist nicht nur intern bei der seitcom ein Problem, sondern auch sehr häufig bei den Kunden der Fall, das Produktionswissen ist dort auch häufig fragmentiert. Das Produktionswissen ist eine Teilmenge des Konfigurationswissens und die Problematik überspannt somit sowohl interne als auch externe Systeme, beide Ebenen erfordern De-

fragmentierung.

Polanyi [1966] prägte für dieses Phänomen die Formulierung „We can know more than we can tell“ - Menschen können mehr wissen, als sie explizit ausdrücken können. Im Kontext der GRACE-Konfiguration manifestiert sich dies konkret: Erfahrene Konfigurationsmitarbeiter wissen intuitiv, welche Validierungsregeln einzuhalten sind, welche Abhängigkeiten zwischen Entitäten bestehen und welche Zeiteinheiten das System erwartet. Dieses Wissen wurde jedoch nie systematisch dokumentiert, sondern existiert primär in den Köpfen weniger Personen.

Die Konsequenzen dieser dualen Herausforderung zeigten sich exemplarisch bei einer Kundenkonfiguration: Das relevante Wissen war auf drei Personen verteilt und lag in keiner strukturierten Form vor. Prozessbeschreibungen waren unvollständig, sodass mehrfach verschiedene Kollegen konsultiert werden mussten. Die Datengrundlage verteilte sich auf vier separate Dokumente, von denen eines nur zufällig verfügbar war, weil ein Kollege die Notizen eines anderen gesichert hatte. Zudem waren die Kundendaten nur für einen von vier Prozessen weitgehend vollständig. Diese Fragmentierung führte zu zwei großflächigen Überarbeitungen der Konfiguration und der Korrektur von 10-15 verschiedenen Fehlern bei einem Kunden. Aktuelle Forschung zeigt, dass Organisationen bei Mitarbeiterwechseln kritisches Wissen verlieren, wenn keine systematischen Externalisierungsmechanismen existieren, ein weiteres Problem, das gelöst werden muss [Levallet and Chan, 2019].

2.2 FORSCHUNGSFRAGE UND ZIELSETZUNG

Aus der beschriebenen Problemstellung leiten sich vier zentrale Forschungsfragen ab:

FF1: Wie kann fragmentiertes Konfigurationswissen systematisch externalisiert werden?

FF2: Welche Artefakte sind notwendig, um reproduzierbaren Wissenstransfer zu ermöglichen?

FF3: Inwieweit kann LLM-gestützte Automatisierung den Konfigurationsprozess optimieren?

FF4: Welche quantifizierbaren Verbesserungen sind hinsichtlich Zeit, Vollständigkeit und Fehlerrate erreichbar?

Diese Fragen adressieren die theoretische Dimension (FF1, FF2) ebenso wie die praktische Umsetzung (FF3) und die empirische Validierung (FF4). Gemeinsam bilden sie den Untersuchungsrahmen für die vorliegende Arbeit.

Die Arbeit verfolgt drei komplementäre Ziele:

Primärziel: Entwicklung einer Drei-Artefakte-Architektur zur systematischen Wissensexternalisierung. Diese besteht aus (1) einem strukturierten Erhebungsfragebogen

zur Erfassung kundenspezifischer Produktionsdaten, (2) einem Entity Mapping Workbook zur formalen Definition der JSON-Schemata aller Entitätstypen und (3) Standard Operating Procedures (SOPs) zur Prozessstandardisierung. Die Standardisierung der Prozesse wurde durch BPMN-Modelle erweitert, außerdem dienen die Prozessmodelle auch als Hilfsmittel zur Analyse und Evaluation. Die Architektur folgt dem SECI-Modell nach Nonaka and Takeuchi [1995]: Der Fragebogen externalisiert implizites Kundenwissen, das Entity Mapping Workbook kombiniert explizites Schemawissen mit den erhobenen Daten, und die SOPs ermöglichen die Internalisierung durch standardisierte Arbeitsanweisungen.

Sekundärziel: Die Entwicklung eines synthetischen Demo-Werks zur Validierung der Drei-Artefakte-Architektur unter realistischen Bedingungen. Das Demo-Werk simuliert eine Pigmentpaste-Produktion und diente zwei Zwecken: (1) Präsentation der GRACE-Fähigkeiten auf der EC Conference Digitalisation, zur Akquise neuer Kundenkontakte und (2) als empirische Grundlage für die Entwicklung des Entity Mapping Workbooks durch Reverse Engineering der JSON-Strukturen.

Tertiärziel: LLM-gestützte Automatisierung der Entitätsgenerierung durch den „Master Creator“ - ein Chat-basiertes Tool, das auf Basis strukturierter Eingaben valide GRACE-Konfigurationen erzeugt. Im Gegensatz zum fragmentierten Status quo ermöglicht der Master Creator die direkte Eingabe von Notizen und Anforderungen, woraus er Prozessskizzen, Entity-Entwürfe und Session-Zusammenfassungen generiert. Das Tool nutzt ein lokal deployetes LLM, um Datenhoheitsanforderungen (DSGVO, NDA) zu erfüllen [Hummel et al., 2021].

Die vorliegende Arbeit strebt eine signifikante Zeitreduktion, eine Vollständigkeit von mehr als 90% und eine deutliche Reduktion der Fehlerrate an. Diese Ziele werden durch kontrollierte Experimente empirisch ermittelt und validiert. Sie adressieren sowohl die Effizienz (Zeitreduktion) als auch die Qualität (Vollständigkeit, Fehlerrate) des Konfigurationsprozesses.

2.3 AUFBAU DER ARBEIT

Die Arbeit gliedert sich in neun Kapitel, die einen durchgängigen argumentativen Bogen von der Problemidentifikation über die Lösungsentwicklung bis zur empirischen Validierung spannen:

Kapitel 3 (Grundlagen) legt das theoretische Fundament: APS-Systeme und ihre Rollen in der Supply Chain und Optimierung von Produktionsplanung. Außerdem legt es Grundlagen für die Datenstrukturierung, Wissensmanagement nach dem SECI-Modell, BPMN-Prozessmodellierung und Large Language Models als technologische Basis der Automatisierung. Im Prinzip alle Grundlagen die man kennen sollte um diese Arbeit verstehen zu können.

Kapitel 4 (Ist-Analyse) präsentiert die systematische Analyse des bestehenden Konfi-

gurationsprozesses. Zwei Prozessvarianten werden identifiziert (manuelle Konfiguration über GRACE UI und Importer-basierte Konfiguration) und hinsichtlich ihrer Stärken und Schwächen evaluiert. Fünf zentrale Problemfelder werden dokumentiert. Das Demo-Werk wird als empirische Grundlage eingeführt.

Kapitel 5 (Methodik) beschreibt die methodische Vorgehensweise nach dem Action-Research-Ansatz [Lewin, 1946]. Diese Methodik eignet sich besonders, wenn der Forscher in den Problemkontext eingebettet ist und traditionelle Experteninterviews aufgrund von Ressourcenbeschränkungen nicht praktikabel sind. Der iterative Forschungszyklus wird dokumentiert, ebenso die Ko-Evolution der entwickelten Artefakte und das Evaluationsdesign.

Kapitel 6 (Implementierung) entwickelt die Drei-Artefakte-Architektur im Detail und dokumentiert ihre technische Umsetzung: Der Erhebungsfragebogen wird nach Prinzipien der Skalenentwicklung [DeVellis, 2017] erzeugt. Das Entity Mapping Workbook dokumentiert die JSON-Schemata aller Entitätstypen. Die SOPs standardisieren die Arbeitsabläufe. Der Master Creator wird als LLM-gestütztes Automatisierungstool entwickelt, inklusive Systemarchitektur, Ollama-Integration und Validierungsmechanismen. Der neue Konfigurationsprozess wird vorgestellt.

Kapitel 7 (Evaluation) evaluiert die Ergebnisse problemorientiert entlang der fünf identifizierten Problemfelder. Für jedes Problemfeld werden Lösungsartefakte, Validierungsmethoden und der Grad der Problemlösung dokumentiert. Ein kontrolliertes Experiment validiert die quantitativen Verbesserungen hinsichtlich Zeitreduktion, Fehlerrate und Vollständigkeit. Die Ergebnisse werden im Kontext vergleichbarer Ansätze im Stand der Forschung eingeordnet.

Kapitel 8 (Fazit) fasst die Erkenntnisse zusammen und beantwortet die Forschungsfragen. Die Schwierigkeiten bei der Bearbeitung werden reflektiert und die konkreten Ergebnisse sowie deren Validierung zusammengefasst.

Kapitel 9 (Ausblick) diskutiert Weiterentwicklungspotenziale und zukünftige Forschungsrichtungen. Technische Erweiterungen und organisatorische Potenziale werden aufgezeigt. Die Transferierbarkeit auf andere Enterprise-Systeme wird erörtert. Automatisierungsvisionen werden als vielversprechende Zukunftsperspektive präsentiert.

3 GRUNDLAGEN

Dieses Kapitel legt das theoretische Fundament für die nachfolgenden Analysen und Entwicklungen. Die Grundlagen sind entlang der Drei-Artefakte-Architektur strukturiert: Advanced Planning Systems und Entity-Relationship-Modellierung bilden die Basis für das Entity Mapping Workbook, Wissensmanagement-Theorien fundieren den Erhebungsfragebogen, und Large Language Models ermöglichen die Automatisierung durch den Master Creator. Die Prozessmodellierung mit BPMN stellt das methodische

Werkzeug für die Ist-Analyse und den Prozessvergleich bereit.

3.1 ADVANCED PLANNING AND SCHEDULING (APS)

Advanced Planning and Scheduling (APS) Systeme erweitern klassische ERP-Funktionalitäten um fortgeschrittene Planungsalgorithmen. Während ERP-Systeme primär Materialbedarfsplanung (MRP) auf Basis von Stücklisten und Durchlaufzeiten durchführen, berücksichtigen APS-Systeme zusätzlich Kapazitätsbeschränkungen, Ressourcenverfügbarkeiten und komplexe Reihenfolgeabhängigkeiten [Stadtler et al., 2015].

DEFINITION UND EINORDNUNG

Nach Stadtler et al. [2015] lassen sich APS-Systeme in die Supply Chain Planning Matrix einordnen. Die Matrix besteht aus Planungsaufgaben nach Zeithorizont und Funktionsbereich. Mit folgenden Zeithorizonten: strategisch, taktisch, operativ und Funktionsbereichen: Beschaffung, Produktion, Distribution. GRACE adressiert primär die operative Produktionsplanung mit Fokus auf Feinplanung und Ressourcenallokation. Die zentrale Herausforderung bei APS-Konfiguration liegt in der kombinatorischen Komplexität: Bereits bei moderaten Problemgrößen (20 Aufträge, 5 Maschinen) existieren faktoriell viele mögliche Zuordnungen. Optimale Lösungen ohne algorithmische Unterstützung sind praktisch unauffindbar [Pinedo, 2016].

GRACE-ENTITÄTSMODELL

GRACE basiert auf einem Entitätsmodell mit sechs grundlegenden Typen, die das Fundament der Konfiguration bilden:

Materials Roh-, Hilfs- und Betriebsstoffe sowie Zwischenprodukte, charakterisiert durch ID, Name und optionale Attribute wie Einheit oder Lagerort.

Machines Produktionsressourcen mit Kapazitäten, Verfügbarkeitszeiten und ressourcenspezifischen Eigenschaften.

Products Endprodukte, die auf ein Material verweisen und Produktspezifikationen definieren.

Processes Fertigungsabläufe bestehend aus Prozessschritten (Steps) mit Bearbeitungszeiten und Ressourcenanforderungen.

BOMs Bill of Materials - Stücklisten, die Eingangsmaterialien und deren Mengen für die Herstellung eines Produkts definieren.

Recipes Verknüpfungsentitäten, die Material, BOM und Process zu einer produktionsfähigen Einheit zusammenführen.

Die Konfiguration einer Produktion in GRACE erfordert die vollständige Definition aller sechs Entitätstypen mit ihren jeweiligen Attributen und Beziehungen. Ein Prozess, der aufgrund der Komplexität und Abhängigkeiten erheblichen Aufwand bedeutet.

3.2 ENTITY-RELATIONSHIP-MODELLIERUNG

Die Entity-Relationship-Modellierung (ERM) nach Chen [1976] bietet ein konzeptuelles Rahmenwerk zur Darstellung von Datenstrukturen und deren Beziehungen. Für die vorliegende Arbeit ist ERM besonders relevant, da die GRACE-Entitäten komplexe Abhängigkeiten aufweisen, die bei der LLM-gestützten Generierung berücksichtigt werden müssen.

GRUNDLAGEN UND NOTATION

Ein Entity-Relationship-Modell besteht aus drei Grundelementen:

- **Entitäten:** Objekte der realen Welt, die eigenständig identifizierbar sind (z.B. Material, Maschine)
- **Attribute:** Eigenschaften von Entitäten (z.B. Material.name, Machine.capacity)
- **Beziehungen:** Assoziationen zwischen Entitäten (z.B. „Product referenziert Material“)

In dieser Arbeit wird die UML-Notationsform von ERM-Diagrammen verwendet [Object Management Group, 2017].

KARDINALITÄTEN UND REFERENZIELLE INTEGRITÄT

Die Kardinalität gibt an, mit wie vielen Objekten einer Entität ein einzelnes Objekt (Instanz) einer anderen Entität verknüpft ist oder sein kann:

- **1:1** - Eine Instanz ist genau einer anderen zugeordnet
- **1:n** - Eine Instanz kann mehreren anderen zugeordnet sein
- **n:m** - Mehrere Instanzen können mehreren anderen zugeordnet sein

Referenzielle Integrität: Dieser Grundsatz stellt sicher, dass Fremdschlüsselverweise stets auf existierende Datensätze zeigen müssen. Im spezifischen *GRACE*-Kontext bedeutet dies:

- Ein **Product** kann nur auf ein **Material** verweisen, das bereits im System vorhanden ist.
- Eine **BOM** (Bill of Materials) kann ausschließlich **Materials** referenzieren, die zuvor erfolgreich angelegt wurden.
- Verallgemeinert: Jede Instanz aus der Konfiguration, die auf eine andere Instanz referenziert, darf erst darauf referenzieren, wenn die Referenz existiert

3.3 PROZESSMODELLIERUNG MIT BPMN

Zur Dokumentation und zum Vergleich der Konfigurationsprozessvarianten wird die Business Process Model and Notation (BPMN) nach ISO 19510 verwendet [International Organization for Standardization, 2013].

NOTATION UND SYMBOLIK

BPMN definiert standardisierte Symbole zur Prozessdarstellung:

- **Events:** Auslöser oder Ergebnisse (Kreise) - Start, Ende, Zwischenereignisse
- **Activities:** Durchzuführende Aufgaben (abgerundete Rechtecke) - Tasks, Subprozesse
- **Gateways:** Verzweigungen und Zusammenführungen (Rauten) - XOR, AND, OR
- **Sequence Flows:** Ablaufreihenfolge (Pfeile)
- **Pools und Lanes:** Akteure und Verantwortlichkeiten

ANWENDUNG IN DIESER ARBEIT

BPMN wird in dieser Arbeit zur Dokumentation und zum Vergleich unterschiedlicher Konfigurationsprozesse eingesetzt. Die standardisierte Notation ermöglicht eine objektive Analyse von Geschäftsprozessen.

3.4 WISSENSMANAGEMENT

SECI-MODELL NACH NONAKA & TAKEUCHI

Das SECI-Modell von Nonaka and Takeuchi [1995] beschreibt vier Modi der Wissenskonversion, die in einer Wissensspirale iterativ durchlaufen werden:

Socialization (Tacit → Tacit): Implizites Wissen wird durch gemeinsame Erfahrung übertragen - beispielsweise lernt ein neuer Consultant durch Beobachtung erfahrener Kollegen.

Externalization (Tacit → Explicit): Implizites Wissen wird in explizite Formen überführt. Dokumentation, Modelle, Artefakte entsprechen expliziten Formen von Wissen. Dies ist der für diese Arbeit zentrale Modus.

Combination (Explicit → Explicit): Explizites Wissen wird systematisiert und neu kombiniert, etwa die Integration verschiedener Dokumentationsquellen in ein einheitliches Workbook.

Internalization (Explicit → Tacit): Explizites Wissen wird durch Anwendung verinnerlicht → Neue Consultants lernen anhand der SOPs.

Die Drei-Artefakte-Architektur dieser Arbeit fokussiert sich primär auf *Externalisierung*: Fragmentiertes Expertenwissen wird in strukturierte Artefakte (Fragebogen, Entity Mapping Workbook, SOPs) überführt, die anschließend als Input für LLM-gestützte Automatisierung dienen. Die Darstellungen von Kardinalitäten werden hier in vereinfachter ASCII-Form dargestellt, um die Implementierung von OCR-Funktionalitäten zu umgehen und trotzdem komplexe Informationen wie Kardinalitäten leicht lesbar für das Modell zu machen.

TACIT VS. EXPLICIT KNOWLEDGE

[Polanyi, 1966, S. 4] prägte die Unterscheidung zwischen implizitem und explizitem Wissen mit der Feststellung: „We can know more than we can tell.“ Implizites Wissen ist kontextgebunden, schwer artikulierbar und oft unbewusst. Die Intuition eines erfahrenen Konfigurationsspezialisten, welche Feldwerte „richtig aussehen“ wäre ein Beispiel für derartiges implizites Wissen.

Im GRACE-Kontext manifestiert sich diese Problematik deutlich: Erfahrene Teammitglieder erkennen Inkonsistenzen in Konfigurationen intuitiv. Jedoch sind die zugrunde liegenden Regeln nirgendwo explizit formuliert. Die Herausforderung besteht darin, dieses implizite Wissen systematisch zu externalisieren.

Die organisatorische Relevanz dieser Problematik wird durch empirische Befunde unterstrichen: Untersuchungen zeigen, dass Organisationen bei Mitarbeiterwechseln kritisches Wissen verlieren, wenn keine systematischen Externalisierungsmechanismen existieren, nach Levallet and Chan [2019] betrifft dies ca 50 % der befragten Unternehmen. Kritisches implizites Wissen geht verloren ohne Externalisierung. Im Kontext eines kleinen Konfigurationsteams von 3-4 Personen, wie es bei GRACE der Fall ist, stellt dies ein erhebliches Risiko dar.

ACTION RESEARCH NACH LEWIN

Action Research, begründet durch Lewin [1946], ist ein zyklischer Forschungsansatz, der Veränderung und Erkenntnisgewinn verbindet. Der Prozess umfasst vier Phasen: *Plan* (Problemanalyse und Interventionsplanung), *Act* (Durchführung der Intervention), *Observe* (Beobachtung der Auswirkungen) und *Reflect* (Reflexion und Anpassung).

Action Research eignet sich besonders für Kontexte, in denen der Forscher eingebettet ist und Wissen iterativ durch Beobachtung und Intervention erfasst werden kann [Reason and Bradbury, 2001].

3.5 LARGE LANGUAGE MODELS (LLMS)

Large Language Models (LLMs) sind neuronale Netzwerke, die auf der Transformer-Architektur basieren und durch Training auf umfangreichen Textkorpora die Fähigkeit erwerben, natürliche Sprache zu verstehen und zu generieren [Vaswani et al., 2017]. Die Kernidee besteht im *Attention*-Mechanismus, der es dem Modell ermöglicht, kontextuelle Beziehungen zwischen Wörtern unabhängig von ihrer Position im Text zu erfassen. Anders als regelbasierte Systeme lernen LLMs implizite Muster aus Trainingsdaten und können diese auf neue, ungesehene Eingaben anwenden.

Für die vorliegende Arbeit ist die Fähigkeit von LLMs zur *strukturierten Datengenerierung* zentral: Durch geeignete Prompt-Gestaltung können LLMs valide JSON-Strukturen erzeugen. Die zu nutzenden Schemata müssen in klarer Form in die Instruktion(Prompt)

eingebaut werden. Diese Eigenschaft unterscheidet den hier verfolgten Ansatz von klassischen NLP-Anwendungen wie Textklassifikation oder Zusammenfassung.

LLMS IM OPERATIONS MANAGEMENT

Large Language Models haben in jüngster Zeit Anwendungen im Bereich Operations Management gefunden. Forschungsarbeiten zeigen, dass LLMs zur Analyse von Produktionsplänen eingesetzt werden können [Yu et al., 2025, Fan et al., 2024]. Studien belegen, dass leichtgewichtige Modelle (7B Parameter) ausreichende Leistung für domänenspezifische Aufgaben erbringen können, sofern sie durch strukturierte Prompts und externe Validierung ergänzt werden.

REVERSE PROMPTING

Das Prompting ist die zentrale Instruktionsform mit der man der man ein LLM zur Generierung eines spezifischen Ergebnisses anweisen kann. Das Reverse Prompting ist eine besondere Unterart des Prompten, welche die Wirkungskette logisch umkehrt. Anstatt das Modell mit Instruktionen zu füttern, wird ihm ein fertiges Ergebnis präsentiert. Dieses Ergebnis soll das LLM analysieren und daraus eine Prompt generieren, die das Ergebnis reproduziert [Li and Klabjan, 2025].

3.6 DATENHOHEIT UND COMPLIANCE

Datenhoheit (Data Sovereignty) bezeichnet die Kontrolle über Datenverarbeitung und -speicherung entsprechend rechtlicher und organisatorischer Anforderungen [Hummel et al., 2021]. Im Kontext der GRACE-Konfiguration sind mehrere Compliance-Aspekte relevant:

- **DSGVO-Konformität:** Kundendaten enthalten potenziell personenbezogene Informationen (Mitarbeiternamen, Schichtpläne). Die DSGVO [Europäisches Parlament und Rat, 2018] erfordert Kontrolle über Datenflüsse zu Drittanbietern.
- **NDA-Verpflichtungen:** Produktionsdaten unterliegen Geheimhaltungsvereinbarungen. Cloud-basierte APIs würden Datentransfer an externe Anbieter bedeuten.
- **Kundenvertrauen:** Fertigungsunternehmen sind sensibel bezüglich ihrer Produktionsdaten (Rezepturen, Kapazitäten, Lieferantenbeziehungen).

Die Wahl geeigneter IT-Architekturen ist entscheidend für die Gewährleistung von Datenhoheit, insbesondere wenn sensible Unternehmensdaten verarbeitet werden.

Quellen die vom Partnerunternehmen als zu vertraulich klassifiziert werden, sind innerhalb dieser Arbeit als unzugänglich deklariert.

3.7 SKALENENTWICKLUNG UND QUALITATIVE INHALTSANALYSE

Um verlässliche Messinstrumente zu entwickeln, orientiert sich der Prozess an bewährten Regeln der Skalenentwicklung. Nach DeVellis [2017] sind dabei vor allem drei Qualitätsmerkmale entscheidend:

- **Redundanzeliminierung**: Es werden keine Merkmale erhoben, die inhaltlich fast das Gleiche abfragen (Redundanz).
- **Essenzialität**: Die Erhebung konzentriert sich strikt auf Informationen, die für die Beantwortung der Forschungsfrage wirklich notwendig sind.
- **Praktikabilität**: Der Umfang der Datenerhebung wird so gewählt, dass er zeitlich und organisatorisch im geplanten Rahmen bleibt.

Die qualitative Inhaltsanalyse nach Mayring [2015] dient dazu, große Mengen an Informationen systematisch auszuwerten und zusammenzufassen. Zentral ist hierbei ein fester Regelkatalog (der Kodierleitfaden): Jedes zu untersuchende Merkmal wird genau definiert, durch ein konkretes Beispiel (Ankerbeispiel) belegt und durch klare Regeln von anderen Kategorien abgegrenzt. Dieses Vorgehen stellt sicher, dass ungeordnete Informationen in klar definierte und auswertbare Kategorien überführt werden.

4 IST-ANALYSE

Dieses Kapitel analysiert den Status quo der GRACE-Konfiguration vor Einführung der optimierten Prozesse und Werkzeuge. Die Darstellung fokussiert sich auf die organisatorischen Rahmenbedingungen, die bestehenden Konfigurationsvarianten und die identifizierten Problemfelder.

4.1 TEAMSTRUKTUR

Das GRACE-Konfigurationsteam umfasst zum Zeitpunkt der Arbeit 5 Personen mit unterschiedlichen Hintergründen: Softwareentwickler mit APS-Kenntnissen, Operations-Research-Consultants und der Autor als dualer Student. Diese kleine Teamgröße hat zwei Implikationen:

- **Engpass-Risiko:** Jedes Teammitglied trägt kritisches Wissen. Ausfälle oder Fluktuation gefährden die Handlungsfähigkeit.
- **Wissensfragmentierung:** Unterschiedliche Spezialisierungen führen zwar zu komplementärem Know-How, jedoch liegt das Wissen ohne eine zentrale und strukturierte Dokumentation nur fragmentiert vor.

Zum Zeitpunkt der Arbeit befinden sich sieben Kundenwerke in der Pilotphase, was intensive Betreuung erfordert. Formale Wissensmanagement-Aktivitäten sind durch die hohe Auslastung erschwert.

4.2 AKTUELLE KONFIGURATIONSPROZESSE

Eine typische GRACE-Konfiguration umfasst ca. 20-30 Materialien, 10-20 Maschinen, 3-5 Produkte mit zugehörigen Prozessen, BOMs und Rezepten.[1] Zum Zeitpunkt der Ist-Analyse existieren zwei etablierte Konfigurationsvarianten, die je nach Kundenanforderungen und verfügbarer Datenlage eingesetzt werden. Die restlichen Informationen der Prozesse wurden aus den Präsentationsmaterialien zu GRACE und den Konfigurationsarbeiten während der Thesis abgeleitet.

MANUELLE KONFIGURATION ÜBER GRACE UI

Die manuelle Konfiguration erfolgt vollständig über die GRACE-Benutzeroberfläche. Der Prozess umfasst:

1. Kickoff-Meeting zur Anforderungsklärung
2. Iterative Datenerhebung (Telefonate, E-Mails, Vor-Ort-Besuche)
3. Manuelle Eingabe aller Entitäten in GRACE-Oberfläche
4. Korrekturschleifen bei Validierungsfehlern
5. Abnahme und Go-Live

Die Durchlaufzeit beträgt durchschnittlich 8 Wochen (2 Monate) inklusive Kommunikationszyklen mit dem Kunden.

IMPORTER-BASIERTE KONFIGURATION

Bei Kunden mit strukturierten Vorsystemen (ERP, MES) kann ein kundenspezifischer Importer entwickelt werden, der Konfigurationsdaten direkt aus den Quellsystemen extrahiert und in GRACE-konforme JSON-Strukturen transformiert.
Dieser Ansatz reduziert die manuelle Eingabe erheblich, erfordert jedoch:

- Initiale Entwicklung des Importers (Aufwand: 1-2 Wochen)
- Zugang zu strukturierten Quelldaten (oft nicht gegeben)
- Mapping-Logik zwischen Kundensystem und GRACE-Schema
- Manuelle Nachbearbeitung, da Importer nicht alle Feldtypen abdecken

Die Importer-basierte Variante ist ressourcenintensiv in der Vorbereitung und wirtschaftlich erst bei vollständiger Durchsicht der Daten und Produktionsumgebungen sinnvoll. In der Praxis kommt sie bei ca. 80% der Konfigurationsprojekte zum Einsatz, jedoch erst wenn alle Variablen der Produktion bekannt sind. Die Vollständige Durchsicht der Daten erfordert meistens einen vorgelagerten Ablauf der Manuellen Konfiguration über die GRACE UI. Die Erkenntnisse aus dem manuellen Durchlauf werden

[1] Quantitative Metriken abgeleitet aus internen Kundenprojekt-Daten (NDA-geschützt, nicht öffentlich hinterlegbar).

analysiert, um sie dann für den Importer zu verallgemeinern. Nach der Verallgemeinerung können alle restlichen Datenreihen die für die Initialkonfigurationen nicht genutzt worden sind automatisch importiert und hinzukommende Daten immer wieder verarbeitet werden.

4.3 IDENTIFIZIERTE PROBLEME

Die Analyse identifizierte fünf zentrale Problemfelder, die Zuin et al. [2025] als typisch für Konfigurationsmanagement beschreiben.

FRAGMENTIERUNG DES KONFIGURATIONSWISSENS

Relevantes Wissen ist über mehrere Quellen verteilt: Azure DevOps Wikis, E-Mail-Korrespondenz, Git-Historie, interne Dokumente und persönliche Erfahrung. Es existiert keine Single Source of Truth. Die Manifestation im Demo-Projekt zeigt: 48% doppelte Materialien (13 von 27), drei simultane ID-Konventionen (numerisch, UUID, kebab-case), Version-Inkonsistenzen (v1-v6). Es ist zu bemerken, dass manchmal auch bestimmte IDs von Kunden gewünscht sind und es deshalb auch zu unterschiedlichen ID-Konventionen kommen kann. Die Fragmentierung des Wissens ist auch bei den Kunden selbst ein großes Problem.

Diese Fragmentierung entspricht ebenso dem von Nonaka and Takeuchi [1995] beschriebenen Problem der nicht-externalisierten Tacit Knowledge in Organisationen, welche die Grundlage zur Arbeit von Zuin et al. [2025] darstellt.

WISSENSTRÄGERABHÄNGIGKEIT

Kritisches Konfigurationswissen ist bei wenigen Experten gebunden. Der Verlust eines Teammitglieds würde erhebliche Wissenslücken verursachen - ein Risiko, das Polanyi [1966] mit „we can know more than we can tell“ charakterisiert.

FEHLENDE STANDARDISIERUNG

Es existiert kein einheitliches Vorgehen bei der Konfiguration. Die Qualität variiert je nach Bearbeiter und Vorgehen. Validierungsmechanismen vor dem Import fehlen weitgehend.

ANFORDERUNGEN AN DATENHOHEIT

Kundenkonfigurationen enthalten sensible Produktionsdaten (Rezepturen, Kapazitäten, Schichtpläne), die unter DSGVO-Anforderungen und NDA-Verpflichtungen stehen. Cloud-basierte Automatisierungsansätze würden Datentransfer an Drittanbieter bedeuten und sind aus Compliance-Gründen nicht akzeptabel. Diese Anforderung schränkt die Auswahl technischer Lösungsansätze erheblich ein.

QUANTITATIVE PROBLEMMANIFESTATIONEN

- **Hohe Fehlerrate:** 10-15 Fehler pro Konfiguration im Durchschnitt
- **Geringe Vollständigkeit:** 60-70% der Pflichtfelder korrekt ausgefüllt
- **Keine Reproduzierbarkeit:** Fehlende zentrale Wissensquelle führt zu mindestens 3 Überarbeitungszyklen bis zur korrekten Konfiguration
- **Lange Durchlaufzeit:** 8 Wochen Kalenderlaufzeit für initiale Konfiguration [2]

4.4 DEMO-WERK ALS EMPIRISCHE GRUNDLAGE

Der seitcom GmbH wurde gewährt auf der EC Conference Digitalisation am 6. November 2025 ein Showcasing von GRACE durchzuführen. Aufgrund des Branchenumfelds wäre ein Demo-Werk aus der Lack- und Farbenindustrie ein ideales Showcasing-Objekt. Parallel zur Analyse der bestehenden Konfigurationsprozesse wurde ein synthetisches Demo-Werk zur Präsentation der Fähigkeiten von GRACE entwickelt und auf der EC Conference Digitalisation präsentiert. Das Werk musste eine branchengetreue Produktion abbilden. Aus diesem Grund wird es hier als Teil der Ist-Analyse aufgeführt auch wenn die Demo erst kurz nach dem Beginn der Bearbeitungszeit verfügbar war.
Die Entwicklung erfüllte eine duale Funktion: Showcasing-Tool für Kundenakquise und empirische Grundlage für die systematische Erfassung der GRACE-Konfigurationsanforderungen. Die Analyse dieses Projekts bildete die Basis für das Erstellen anderer Artefakte. (siehe Kapitel 5).
Das Demo-Werk simuliert eine Pigmentpasten-Produktion mit drei Farbvarianten (Weiß, Rot, Blau) und umfasst 22 Materialien, 7 Maschinen, 3 Produkten mit zugehörigen BOMs, Prozessen und Rezepten [seitcom GmbH, 2026]. Eine detailliertere Aufschlüsselung findet sich in späteren Kapiteln.

5 METHODIK

Dieses Kapitel erläutert das methodische Vorgehen zur Entwicklung der Drei-Artefakte-Architektur und des Master Creators. Im Zentrum steht die Frage, *wie* und *warum* die gewählten Methoden zur Lösung der in Kapitel 4 identifizierten Probleme geeignet sind. Die detaillierten Ergebnisse werden in späteren Kapiteln ausgeführt werden.

5.1 AKTIONSFORSCHUNG ALS METHODISCHER ANSATZ

Die vorliegende Arbeit folgt dem Paradigma der Aktionsforschung (Action Research) nach Lewin [1946]. Dieser Ansatz verbindet praktische Problemlösung mit wissenschaftlicher Erkenntnisgewinnung und eignet sich besonders für Kontexte, in denen der Forscher aktiv in das Untersuchungsfeld eingebettet ist [Reason and Bradbury, 2001].

[2] Quantitative Metriken abgeleitet aus internen Kundenprojekt-Daten (NDA-geschützt, nicht öffentlich hinterlegbar).

BEGRÜNDUNG DER METHODENWAHL

Die Wahl von Action Research wird durch vier kontextspezifische Faktoren begründet:

1. **Instabilität des Zielsystems:** GRACE befindet sich in der Pilotphase mit kontinuierlich evolvierenden Datenstrukturen und JSON-Schemata. Klassische Erhebungsmethoden würden veraltete Zustände dokumentieren, während das System parallel weiterentwickelt wird. Action Research begreift die dynamische Veränderung des Forschungsgegenstandes nicht als Hindernis, sondern als konstitutiven Bestandteil der Methode. Die Entwicklung der vorgestellten Artefakte erstreckte sich über einen Zeitraum von drei Monaten. Multiple Iterationen, sogenannte *Action Research Cycles*, wurden durchlaufen, um sich an die dynamischen Veränderungen anzupassen. Das Phänomen der dynamischen Veränderung des Analyseobjekts wird in der Literatur als *Moving Target*-Effekt bezeichnet wird.

2. **Fragmentiertes Wissen als Kernproblem:** Das interne Konfigurationswissen existiert verteilt über Azure DevOps Backlogs, E-Mail-Korrespondenzen, Git-Historien und informelle Teamgesprächen. Das externe Konfigurationswissen liegt häufig in ähnlich fragmentierter Form beim Kunden. Es fehlt eine zentrale, strukturierte Wissensbasis. Das SECI-Modell nach Nonaka and Takeuchi [1995] adressiert dieses Problem durch systematische Externalisierung: Die Drei-Artefakte-Architektur überführt dieses fragmentierte Wissen in explizite, maschinenlesbare Formate.

3. **Impraktikabilität formaler SME-Interviews:** Das Konfigurationsteam umfasst 5 Personen mit vollem Projektgeschäft während der Pilotphase. Mehrstündige, strukturierte Experteninterviews würden operative Kapazitäten überlasten. Regelmäßige informelle Gespräche während der täglichen Arbeit ermöglichen hingegen kontinuierliche Wissenserfassung ohne zusätzlichen Zeitaufwand. Jackson [2010] validiert diesen Ansatz: In einer vergleichbaren Studie zur Wissensexternalisierung bei Ressourcenknappheit erwies sich partizipative Aktionsforschung als praktikable Alternative zu formalen Interviewmethoden.

4. **Eingebettete Forscherrolle:** Der Autor ist selbst Teil des Konfigurationsteams und hat direkten Zugang zu Prozessen, Werkzeugen und implizitem Wissen. Diese Position ermöglicht teilnehmende Beobachtung und iterative Verfeinerung der entwickelten Artefakte. Susman and Evered [1983] argumentieren, dass Action Research besonders dann geeignet ist, wenn eine Distanzierung des Forschers vom Untersuchungsfeld nicht möglich oder nicht sinnvoll ist, eine Bedingung, die im vorliegenden Kontext erfüllt ist.

5. **Quantifizierbare Problemstellungen:** Die in Kapitel 4 identifizierten Probleme manifestieren sich in messbaren Größen (Fehlerrate, Vollständigkeit, Reproduzierbarkeit, Zeit), was eine empirische Validierung der entwickelten Artefakte durch kontrolliertes Experiment ermöglicht. Action Research kombiniert praktische Problemlösung mit wissenschaftlicher Evaluation dieser Lösungen.

5.2 FORSCHUNGSZYKLUS UND ITERATIONEN

Der Action-Research-Zyklus (Plan → Act → Observe → Reflect) wurde in dieser Arbeit über mehrere Iterationen operationalisiert und manifestiert sich auf multiple Arten und Weisen. Auch bei der Gliederung dieser Arbeit zeichnet sich die Action Research Methode ab, dabei ist die Ist-Analyse und die Methodik der Plan-Zyklus, die Implementierung der Act-Zyklus und die Ergebnisse der Observe-Zyklus mit Einleitung des Reflect-Zyklusses, der im Fazit und Ausblick vervollständigt wird. Die gesamte Entwicklung erstreckte sich über einen Zeitraum von drei Monaten zuzüglich zwei Wochen krankheitsbedingter Unterbrechung. Innerhalb der Entwicklung lässt sich das Muster ebenso betrachten, sogar bei den granularsten Einheiten. Die folgende Darstellung gliedert die Entwicklung in die typischen fünf Phasen sequenziell sortiert, wobei jede Phase multiple Artefakte parallel hervorbrachte. Daraufhin wird die Vorgehensweise in den jeweiligen Phasen erklärt.

Abbildung 1: Action Research Zyklus nach Lewin (1946): Iterative Phasen von Planung, Handlung, Beobachtung und Reflexion - Quelle: [Topić 2026]

PHASE 0: DEMO-WERK-ENTWICKLUNG ALS ERKENNTNISFUNDAMENT

Parallel zur Problemanalyse bestand eine praktische Aufgabenstellung (**Plan**): Die Entwicklung eines Demo-Werks zur Präsentation der GRACE-Fähigkeiten auf der EC Conference Digitalisation. Dieses Praxisprojekt erfüllte methodisch eine duale Funktion:

Einerseits diente es als Showcasing-Tool für Kundenakquise, andererseits fungierte es als empirische Grundlage für die systematische Erfassung der GRACE-Konfigurationsanforderungen.

Methodische Besonderheit: Learning by Doing Die Demo-Werk-Entwicklung folgte keinem vorab definierten Prozessmodell, sondern erfolgte explorativ (**Act**) durch „Code as Documentation"-Ansatz: Reverse Engineering bestehender JSON-Exporte, Trial-and-Error bei der Konfiguration, und iterative Fehlerkorrektur. Diese methodische Besonderheit entspricht dem Action-Research-Prinzip nach Lewin [1946]: Problemlösung und Erkenntnisgewinnung sind iterativ verzahnt.

Domänenauswahl: Farb- und Lackproduktion Die Wahl der Produktionsdomäne (Pigmentpaste-Produktion) wurde durch vier Faktoren begründet:

1. **Zielgruppe der EC Conference:** Fachpublikum aus der Lack- und Farbenindustrie
2. **Verfügbare Fachliteratur:** Umfangreiche technische Dokumentation ermöglichte fundierte Modellierung ohne Kundendaten
3. **Domänenexperte im Team:** Ehemaliger Maschinenführer mit Praxiserfahrung validierte Plausibilität
4. **GRACE-Feature-Validierung:** Komplexe Prozessabhängigkeiten (Reinigungszyklen, mengenabhängige Zeitberechnungen) eigneten sich zur Validierung von Properties und Formelausdrücken

Wissensquellen und Systematik Die Entwicklung folgte einer systematischen Wissensintegration aus drei Quellen:

- **Fachliteratur:** Branchenstandards für Materialklassen, Maschinentypen, Prozessabläufe und Zeitberechnungsmethoden [Fonds der Chemischen Industrie, 2018, Müller and Poth, 2017, Goldschmidt, 2019, REFA Verband für Arbeitsgestaltung, Betriebsorganisation und Unternehmensentwicklung, 2012]
- **Domänenexpertise:** Praxiserfahrung des Maschinenführers validierte Plausibilität und lieferte Erfahrungswerte [seitcom GmbH, 2025a]
- **GRACE-Anforderungen:** Bewusste Integration komplexer Features (mengenabhängige Zeitfunktionen, Property-basierte Materialzuordnung, verschachtelte Referenzstrukturen) zur späteren Validierung der Konfigurationslogik

Methodische Reflektion Die während dieser Phase durchgeführte Reverse-Engineering-Analyse offenbarte zentrale Erkenntnisse (**Observe**): (1) die Abhängigkeitskette zwischen den sechs Entitätstypen, (2) ID-Konsistenz als kritischer Erfolgsfaktor und (3) die Komplexität verschachtelter Referenzstrukturen bei Recipes. Die **Reflexion** dieser Beobachtungen führte zur Erkenntnis, dass das fragmentierte Konfigurationswissen systematisch externalisiert werden muss (**Reflect**). Diese Erkenntnisse bildeten die Basis

für das Entity Mapping Workbook (Phase 1) und die 6-Punkte-Validierungscheckliste. Die konkreten technischen Details des Demo-Werks werden in Kapitel 6 dokumentiert.

PHASE 1: WISSENSEXTERNALISIERUNG DURCH ENTITY MAPPING UND VALIDIERUNG

Basierend auf den Erkenntnissen der Demo-Werk-Entwicklung (**Plan**) wurden in dieser Phase zwei zentrale Artefakte zur Wissensexternalisierung entwickelt (**Act**).

Das **Entity Mapping Workbook** soll dem Problem der **Fragmentierung des Konfigurationswissens** dienen, indem es das über verschiedene Quellen verteilte implizite Wissen in eine zentrale, formale Struktur externalisiert. Gleichzeitig soll es durch Definition einheitlicher JSON-Schemata und Validierungsregeln dem Problem der **fehlenden Standardisierung** entgegenwirken.
Die Entwicklung erfolgte nach der qualitativen Inhaltsanalyse [Mayring, 2015]: Multiple Datenquellen wurden systematisch analysiert und zu allgemeinen ER-Kardinalitäten abstrahiert, um Overfitting auf einzelne Datensätze zu vermeiden. Die zentrale Herausforderung bestand darin, multi-direktionale Beziehungen mit Versionierung korrekt abzubilden.

Parallel wurde eine **6-Punkte-Validierungscheckliste** entwickelt, die ebenso dem Problem der **fehlenden Standardisierung** dienen soll, indem sie einheitliche Qualitätskriterien für Konfigurationen definiert. Die Checkliste systematisiert empirisch beobachtete Fehlertypen. Die konkreten Validierungsregeln werden in Kapitel 6 dokumentiert.

Methodische Reflektion Die Checkliste dient als explizites Werkzeug zur Qualitätssicherung (**Observe**) und wird in Kapitel 7 zur Messung der Konfigurationsqualität verwendet. Die **Reflexion** ergab, dass die Kombination aus strukturierter Schema-Dokumentation und operationalisierten Validierungsregeln die konzeptuelle Basis für LLM-gestützte Automatisierung bildet (**Reflect**). Diese Erkenntnis motivierte den Übergang zur Tool-Entwicklung in Phase 2.

PHASE 2: INITIALES PROTOTYPING UND GESCHEITERTE MONOLITH-ANSÄTZE

Diese Phase umfasste das initiale Prototyping zur LLM-gestützten Entitätsgenerierung (**Plan**). Die Prompts wurden nach den Prinzipien des Reverse Prompting aufgebaut, jedoch mit einer methodischen Verfeinerung. Das Modell sollte nicht einfach einen Prompt erstellen, um die analysierte Datengrundlage zu reproduzieren, sondern es wurde um ein Vorgehen erweitert, welches durch die Entwicklung von Sora AI für Android inspiriert war [Hum and Marsan, 2025]. Nach der Analyse im Reverse Prompting-Vorgehen werden mehrere Schritte zwischengeschoben. Die Annahmen des LLMs aus der Analyse werden vom Instrukteur zusätzlich analysiert. Die Trugschlüsse werden korrigiert und zurück an das LLM übermittelt. Erst dann wird das Modell angewiesen einen Prompt zu generieren, die das Ergebnis reproduziert. Diese Vorgehensweise wird wiederholt, bis das LLM richtige Ergebnisse erzielt. So wurde die Logik des Master

Creators stetig verbessert. Eine feine Änderung, die die Qualität der Ergebnisse iterativ steigern kann, aber dazu mehr in Kapitel 6, um nicht zu viel vorzugreifen.

Monolithischer Prompt-Ansatz Der verfolgte Ansatz (**Act**) nutzte einen umfänglichen Prompt, der alle Abhängigkeiten zwischen den sechs Entitätstypen direkt abbilden sollte. Das Ziel war, in einem einzigen LLM-Aufruf eine vollständige GRACE-Konfiguration zu generieren.

Methodische Reflektion Dieser Ansatz funktionierte zu diesem Zeitpunkt nicht (**Observe**). Die **Reflexion** ergab, dass die Komplexität der verschachtelten Abhängigkeiten (insbesondere Recipe-Verknüpfungen mit Material-BOM-Process-Referenzen) die Kapazität eines einzelnen Prompt-Kontexts übersteigt (**Reflect**). Die hohe Fehlerrate bei Referenzauflösungen motivierte den strategischen Wechsel zu einem modularen Ansatz, ebenso inspiriert nach Hum and Marsan [2025].

PHASE 3: BLUEPRINT-PRINZIP UND MODULARE EINZELCREATOREN

Als Reaktion auf die Limitationen der monolithischen Ansätze (**Plan**) wurde in dieser Phase ein modularer Entwicklungsansatz verfolgt (**Act**).

Das **Blueprint-Prinzip** basiert auf der schrittweisen Entwicklung: Ein einfaches Modul für einen einzelnen Entitätstyp wurde entwickelt und validiert. Dieses Grundprinzip wurde anschließend systematisch auf alle weiteren Entitätstypen übertragen.

Modularer Ansatz und Rückintegration Der Ansatz verfolgte zunächst eine schrittweise Spezialisierung: Ein einzelner Entitätstyp wurde isoliert betrachtet, dessen Generierungslogik entwickelt und validiert. Dieses Prinzip wurde systematisch auf alle sechs Entitätstypen übertragen. Nach erfolgreicher Validierung der Einzelmodule wurden die modularen Erkenntnisse wieder in eine zusammenhängende Prompt-Struktur integriert.

Methodische Reflektion Der modulare Ansatz ermöglichte kontrollierte Generierung mit reduzierten Fehlerquellen und validierte die Machbarkeit LLM-gestützter Entitätsgenerierung (**Observe**). Die **Reflexion** offenbarte jedoch neue Limitationen isolierter Module: fehlende Referenzauflösung zwischen Entitäten und mangelnde Session-Historie verhinderten die Erstellung vollständiger entitätsübergreifernder Beziehungen in den Konfigurationen (**Reflect**). Diese Erkenntnis motivierte die Rückintegration in ein unified System mit Session-Management in Phase 4.

PHASE 4: MASTER CREATOR MIT SESSION-MANAGEMENT

Diese Phase integrierte die modularen Erkenntnisse aus Phase 3 (**Plan**) in ein einheitliches System mit intelligenter Referenzauflösung und Session-Persistenz (**Act**).

Integration und Session-Management Der **Master Creator** adressierte die Limitationen der Einzelcreatoren durch Integration aller sechs Entitätstypen in ein einheitliches Chat-Interface. Ein zentrales Session-Management ermöglichte die automatische Referenzauflösung zwischen abhängigen Entitäten (z.B. Recipe-Generierung basierend auf zuvor erstellten Materials, BOMs und Processes).

Automatisierung und Validierung Der Master Creator soll durch automatisierte Referenzauflösung und integrierte Validierung zur Reduktion der Fehlerrate, durch Zeitersparnis bei der Konfigurationserstellung und durch dokumentierte Chat-Konversationen zur Reproduzierbarkeit beitragen.

Iterative Verfeinerung Jede Iteration bildete einen Mikro-Zyklus: Fehlerbeobachtung durch praktische Anwendung (**Observe**), **Reflexion** der Fehlerursachen (**Reflect**), Prompt-Anpassung als neuer **Plan**, erneute Testung (**Act**). Die technische Evolution mit konkreten Fehlerquellen und Lösungsansätzen wird in Kapitel 6 dokumentiert.

Methodische Reflektion Die technische Umsetzung wird in Kapitel 6 detailliert beschrieben. Diese Phase markiert den Abschluss des Entwicklungs-Zyklus und den Übergang von fragmentiertem, manuellem Konfigurationswissen zu einem integrierten, LLM-gestützten Workflow.

UNGEPLANTE METHODISCHE BESONDERHEIT: DATENVERLUST

Während der Bearbeitungszeit ereignete sich ein Ausfall des Network Attached Storage (NAS) im Unternehmen, der zum Verlust von Demo-Konfigurationsdaten führte. Zusätzlich verhinderten Bugs in der GRACE-Exportfunktion den vollständigen Export bestimmter Entitäten (insbesondere Properties).
Dieses unerwartete Ereignis ermöglichte eine praxisnahe Validierung des Action-Research-Ansatzes. Die Rekonstruktion erfolgte iterativ unter Nutzung des Master Creators bis zur vollständigen Rekonstruktion [seitcom GmbH, 2026].

5.3 DATENAUSWERTUNG

Die erhobenen Daten wurden mittels verschiedener Auswertungsmethoden analysiert und in strukturierte Artefakte überführt. Die Auswertung erfolgte iterativ und orientierte sich an etablierten Methoden der qualitativen Forschung sowie der Skalenentwicklung.

QUALITATIVE INHALTSANALYSE FÜR ENTITY MAPPING

Die Entwicklung des Entity Mapping Workbooks erfolgte nach der qualitativen Inhaltsanalyse nach Mayring [2015]. Die Methode umfasst drei Schritte:

1. **Kodierung:** JSON-Exporte aus der Demo-Werk-Konfiguration [seitcom GmbH, 2026] und generische Beispiel-JSONs [seitcom GmbH, 2025c] wurden systematisch analysiert. Für jedes Entitätsfeld wurde eine Kodierregel erstellt (Definition, Ankerbeispiel, Abgrenzung).

2. **Kategorisierung:** Wiederkehrende Muster (z.B. ID-Strukturen, Versionsattribute, Referenztypen) wurden zu allgemeinen Kategorien abstrahiert.

3. **Verallgemeinerung:** Aus den spezifischen Datensätzen wurden generalisierbare JSON-Schemata abgeleitet, um Overfitting auf einzelne Konfigurationen zu vermeiden.

Diese systematische Abstraktion stellte sicher, dass das Entity Mapping Workbook nicht auf ein spezifisches Demo-Werk zugeschnitten ist, sondern allgemeingültige Konfigurationsmuster dokumentiert.

FRAGEBOGENENTWICKLUNG NACH DEVELLIS

Der GRACE Onboarding Fragebogen wurde nach den Prinzipien der Skalenentwicklung von DeVellis [2017] entwickelt. Er wurde in den vorherigen Phasen nicht explizit erwähnt. Die Entwicklung folgt jedoch der gleichen iterativen Vorgehensweise nach Action Research, nur in einem separaten Loop. Über die Gliederung der Thesis lässt er sich wieder in einem gesamt Kontext der Action Research Methode überführen. Der Fragebogen soll dem Problem der **Fragmentierung** entgegenwirken. Indem er fragmentiertes Kundenwissen in ein standardisiertes Format überführt. Dies soll das Problem der **fehlenden Standardisierung** bei der Wissenserhebung lösen. Das Vorgehen umfasste:

1. **Item-Generierung:** Sammlung potenzieller Fragen aus unzugänglichen Quellen (Azure DevOps Wikis, E-Mail-Korrespondenz, Git-Historie, interne Dokumente)[3] und zugänglichen Wissensquellen (Expertengespräche, persönliche Erfahrung aus Konfigurationsprojekten, APS-Interviewskript aus früherer akademischer Arbeit [seitcom GmbH, 2024a], Gesprächstranskript zur Entwicklung des Fragenkatalogs [seitcom GmbH, 2025b])

2. **Redundanzeliminierung:** Identifikation und Zusammenführung von Fragen mit überlappenden Informationsgehalten.

3. **Essenzialitätsprüfung:** Fokussierung auf Fragen, die notwendige (nicht optionale) Informationen erheben.

4. **Praktikabilitätsanpassung:** Anpassung des Umfangs an zeitliche und ressourcenbezogene Rahmenbedingungen (Workshop-Dauer, Kundenkapazitäten).

Weitere iterative Verfeinerungen erfolgten durch Anwendung in realen Kundenprojekten und anschließender Anpassung basierend auf Feedback.

REVERSE ENGINEERING FÜR JSON-STRUKTUREN

Bestehende JSON-Konfigurationen wurden mittels Reverse Engineering und Reverse Prompting analysiert, um implizite Strukturregeln zu identifizieren:

[3] Abgeleitet aus internen Kundenprojekt-Daten (NDA-geschützt, nicht öffentlich hinterlegbar).

- **Strukturanalyse:** Feldtypen, Verschachtelungstiefe, optionale vs. obligatorische Felder
- **Abhängigkeitsanalyse:** Referenzbeziehungen zwischen Entitäten, Versionierungslogik
- **Validierungsregeln:** Implizite Constraints

Diese Analyse bildete die Grundlage für die 6-Punkte-Validierungscheckliste, die empirisch beobachtete Fehlerquellen systematisiert und in den System-Prompt und andere LLM-gerechte Artefakte aufgenommen wurde. Außerdem diente sie auch als Metrik im Evaluierungsexperiment mit dem Master Creator.

PROZESSMODELLIERUNG MIT BPMN

Die Konfigurationsprozesse wurden mittels Business Process Model and Notation (BPMN) dokumentiert. Die Entwicklung der Prozessmodelle erfolgte ebenso iterativ und parallel zur Artefaktentwicklung durch:

- **Teilnehmende Beobachtung:** Direktes Shadowing von Konfigurationsschritten während der Demo-Werk-Entwicklung und realer Kundenprojekte
- **Zeitmessungen:** Erfassung der Durchlaufzeiten für einzelne Prozessschritte unter realen Arbeitsbedingungen
- **Dokumentenanalyse:** Analyse bestehender Prozessskizzen und Arbeitsanweisungen aus Präsentationsmaterialien [seitcom GmbH, 2024e,f,d,b,c]
- **Retrospektive Modellierung:** Nach Abschluss der Tool-Entwicklung wurden die Prozessvarianten formal modelliert

Die Modellierung folgt der BPMN 2.0-Notation und bildet verschiedene Prozessvarianten ab, um die methodische Evolution des Konfigurationsansatzes zu dokumentieren. Die konkreten Prozessmodelle und deren Auswertung werden in Kapitel 7 präsentiert.

5.4 DREI-ARTEFAKTE-ARCHITEKTUR ALS METHODISCHES FRAMEWORK

Die **Drei-Artefakte-Architektur** bildet das konzeptuelle Fundament der Externalisierung und operationalisiert das SECI-Modell nach Nonaka and Takeuchi [1995]. Die Architektur transformiert implizites Domänenwissen in LLM-konsumierbare Strukturen und soll den in Kapitel 4 identifizierten Problemfeldern wie folgt dienen:

1. **GRACE Onboarding Fragebogen:** Systematische Wissenserhebung beim Kunden, externalisiert implizites Produktionswissen in strukturierte Antworten. Soll der Vollständigkeit dienen, indem ein standardisiertes Maß für benötigte Informationen definiert wird.

2. **Entity Mapping Workbook:** Formale JSON-Schemata mit Kardinalitäten und Validierungsregeln, externalisiert implizites Konfigurationswissen des Teams. Soll durch systematische Dokumentation der Abhängigkeiten zur Reduktion von Referenzfehlern und zur Standardisierung beitragen.

3. **Standard Operating Procedures (SOPs):** Prozessdokumentation für Workshop (SOP 01), Tool-Nutzung (SOP 02) und Import (SOP 03), externalisiert implizites Prozesswissen. Soll der Reproduzierbarkeit dienen, indem Wissensträgerabhängigkeit durch dokumentierte Prozesse reduziert wird.

Diese drei Artefakte entwickelten sich parallel zur technischen Tool-Entwicklung. Die gegenseitige Beeinflussung und iterative Verfeinerung dieser Artefakte wird in Kapitel 6 detailliert beschrieben.

5.5 TECHNISCHE INFRASTRUKTUR UND DATENHOHEIT

Die technische Infrastruktur musste zwei widersprüchliche Anforderungen vereinen: LLM-basierte Automatisierung einerseits, absolute Datenhoheit andererseits. Kundenkonfigurationen enthalten sensible Produktionsdaten (Rezepturen, Kapazitäten, Schichtpläne), die unter DSGVO-Anforderungen und NDA-Verpflichtungen stehen. Cloud-basierte LLM-Lösungen (GPT-4, Claude) scheiden daher aus Compliance-Gründen aus.

Strategische Entscheidung Die Lösung bestand in einem **lokalen LLM-Deployment**, das alle Datenverarbeitung on-premise durchführt. Diese Architekturentscheidung erzwingt jedoch einen fundamentalen Trade-off: Kleinere Sprachmodelle (7B-Parameter-Klasse) sind mit verfügbarer Standard-Hardware betreibbar, erreichen aber nicht die Leistungsfähigkeit cloud-basierter Großmodelle (70B+ Parameter).
Dieser Trade-off wurde bewusst zugunsten der Datensouveränität entschieden. Die resultierende Vollständigkeit von 90-95% (gegenüber potenziell 98%+ bei Großmodellen) wurde als akzeptabel bewertet, da:

- Privacy-Compliance nicht verhandelbar ist (rechtliche Anforderung)
- 90-95% Vollständigkeit bereits eine signifikante Verbesserung gegenüber dem Status quo (60-70%) darstellt
- Manuelle Nachbearbeitung der verbleibenden 5-10% deutlich schneller ist als vollständige Neuerstellung

Die konkrete technische Umsetzung dieser strategischen Entscheidung (Modellauswahl, Deployment-Architektur, Validierungsmechanismen) wird in Kapitel 6 detailliert beschrieben.

5.6 EVALUATIONSDESIGN QUANTITATIVER PROBLEME

Die Evaluation der quantitativen Probleme adressiert die in Kapitel 4 identifizierten Problemfelder durch ein kontrolliertes Experiment. Dieser Teil der Evaluation dient als

Hauptteil der Evaluation, weil alle 5 Problemfelder gleichzeitig analysiert werden. Die quantitativen Probleme werden wie folgt operationalisiert:

- **Fragmentierung und fehlende Standardisierung:** Messung über Vollständigkeitsgrad und Fehlerrate gemäß 6-Punkte-Checkliste
- **Wissensträgerabhängigkeit:** Messung über Reproduzierbarkeit durch drei unabhängige Probanden
- **Zeiteffizienz:** Messung der Konfigurationsdauer als Primärmetrik
- **Datenhoheit:** Adressiert durch technische Infrastruktur (lokales LLM-Deployment)

Die Evaluation kombiniert gemessene Praxisdaten mit einem kontrollierten Experiment. Die in den BPMN-Diagrammen (v1a, v1b) dargestellten Durchlaufzeiten basieren auf realen Konfigurationsprojekten.

KONTROLLIERTES VALIDIERUNGSEXPERIMENT

Zur Validierung der Tool-Effektivität wurde ein Within-Subjects-Experiment durchgeführt. Die Wahl dieses Designs folgt der Empfehlung von Döring and Bortz [2023] aus dem Inventar der Human- und Sozialwissenschaften zur Kontrolle menschlicher Störfaktoren. Die Fehlervarianz sollen durch interindividuelle Unterschiede minimiert werden und so die Effekte der Intervention des Master Creators präziser zu isolieren. Counterbalancing soll sequenzielle Effekte minimieren [Döring and Bortz, 2023]:

Setup

Fiktive Produktkonfiguration mit standardisierter Komplexität: 5 Materialien, 5 Maschinen, 1 Produkt, 1 Prozess (mehrere Schritte), 1 BOM, 1 Rezept.

Bedingungen

Bedingung A (Baseline): Manuelle Eingabe in GRACE UI ohne Hilfsmittel

Bedingung B (Intervention): Master Creator + JSON-Import in GRACE

Jeder Proband absolvierte 6 Durchläufe pro Bedingung (alternierend, counterbalanced). Die Reihenfolge wurde zwischen den Probanden variiert, um Reihenfolgeeffekte zu kontrollieren.

Probanden

- **Proband 1:** Konfigurationsteam-Mitglied mit GRACE-Erfahrung und Master-Creator-Erfahrung
- **Proband 2:** Konfigurationsteam-Mitglied mit GRACE-Erfahrung, ohne Master-Creator-Erfahrung
- **Proband 3:** Konfigurationsteam-Mitglied mit GRACE-Erfahrung, ohne Master-Creator-Erfahrung

Metriken

1. **Zeit:** Dauer bis zur vollständigen Konfiguration (MM:SS,ms)
2. **Fehlerrate:** Anzahl Validierungsfehler bei JSON-Import in GRACE
3. **Checklisten-Vollständigkeit:**
   Anzahl erfüllter Punkte der 6-Punkte-Validierungscheckliste ($v \in [0,6]$)

Die Ergebnisse dieses Experiments werden in Kapitel 7 präsentiert.

STATISTISCHE AUSWERTUNGSMETHODEN

Die quantitative Auswertung des kontrollierten Experiments erfolgt mittels deskriptiver Statistik. Für jede Bedingung (A: Baseline, B: Intervention) werden folgende Kennwerte nach Döring and Bortz [2023] berechnet:

Zeitreduktion (Primärmetrik) Die Konfigurationszeit t wird in Sekunden erfasst. Für jede Bedingung wird der Mittelwert berechnet:

$$\bar{t} = \frac{1}{n} \sum_{i=1}^{n} t_i$$

Die Standardabweichung quantifiziert die Streuung der Messungen:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (t_i - \bar{t})^2}$$

Die prozentuale Zeitreduktion zwischen den Bedingungen wird berechnet als:

$$\text{Reduktion} = \frac{\bar{t}_A - \bar{t}_B}{\bar{t}_A} \times 100\%$$

Vollständigkeit und Fehlerrate Für jede Konfiguration wird die Anzahl erfüllter Validierungspunkte $v \in [0,6]$ gemäß der 6-Punkte-Checkliste erfasst. Die durchschnittliche Vollständigkeit wird berechnet als:

$$\bar{v} = \frac{1}{n} \sum_{i=1}^{n} v_i \quad \text{und} \quad \text{Vollständigkeit} = \frac{\bar{v}}{6} \times 100\%$$

Fehler werden als diskrete Ereignisse gezählt. Der Mittelwert und die Standardabweichung der Fehleranzahl werden analog zur Zeitmetrik berechnet.

Reproduzierbarkeit Die Reproduzierbarkeit wird durch die Konsistenz der Ergebnisse über drei unabhängige Probanden gemessen. Für jede Metrik (Zeit, Vollständigkeit, Fehlerrate) wird die Inter-Proband-Variabilität über die Standardabweichung zwischen den Probanden-Mittelwerten quantifiziert.

Darstellung der Ergebnisse Die Ergebnisse werden in tabellarischer Form präsentiert:

- **Zeit:** Mittelwert ± Standardabweichung (MM:SS ± SS)
- **Vollständigkeit:** Durchschnittliche Anzahl erfüllter Punkte und Prozentangabe
- **Fehlerrate:** Mittelwert ± Standardabweichung (Anzahl Fehler)
- **Reproduzierbarkeit:** Variabilität zwischen Probanden

Die statistischen Berechnungen erfolgen für $n = 6$ Durchläufe pro Proband und Bedingung. Die Ergebnisse werden in Kapitel 7 präsentiert.

EXPLORATIVE ANWENDUNG IN DER PRAXIS

Parallel zum kontrollierten Experiment wurde der GRACE Onboarding Fragebogen und SOP 01 (Kundenworkshop durchführen) explorativ in zwei realen Kundenprojekten eingesetzt. Diese Anwendung erfolgte unter Produktionsbedingungen mit tatsächlichen Kundenanforderungen und diente primär der praktischen Erprobung der entwickelten Artefakte im Feldkontext.
Der Fragebogen wurde als Leitfaden zur strukturierten Wissenserhebung in initialen Kundenworkshops verwendet. Die erhobenen Informationen dienten als Eingabe für die nachgelagerte Konfigurationsentwicklung mit dem Master Creator. Diese explorative Anwendung ermöglichte qualitative Beobachtungen zur Praktikabilität der Artefakte, erhob jedoch keine quantitativen Vergleichsdaten. Die Erkenntnisse fließen als ergänzende qualitative Perspektive in Kapitel 7 ein.

WISSENSCHAFTLICHE GÜTEKRITERIEN DES EXPERIMENTS

Das kontrollierte Validierungsexperiment wurde unter Berücksichtigung der wissenschaftlichen Gütekriterien Validität, Reliabilität und Objektivität konzipiert [gothesis.de, 2024].

Validität

- **Inhaltsvalidität:** Die 6-Punkte-Checkliste basiert auf empirisch beobachteten Fehlerquellen aus der Demo-Werk-Entwicklung und deckt kritische Validierungsaspekte ab (Vollständigkeit, ID-Konsistenz, Zeiteinheiten, BOM-Berechnung, Material-Referenzen, ID-Duplikate).
- **Konstruktvalidität:** Die gemessenen Dimensionen (Zeit, Vollständigkeit/Fehlerrate, Reproduzierbarkeit) operationalisieren das theoretische Konstrukt „Konfigurationseffizienz“ entlang wissenschaftlich fundierter Dimensionen: Zeitwirtschaft nach REFA [REFA Verband für Arbeitsgestaltung, Betriebsorganisation und Unternehmensentwicklung, 2012], Qualitätsmanagement (Fehlerreduktion) und Wissensmanagement (Reproduzierbarkeit durch Externalisierung).

- **Externe Validität:** Das standardisierte Experiment-Setup ermöglicht prinzipiell die Replikation durch andere Forschende. Die Übertragbarkeit auf andere APS-Kontexte ist durch die Domänenspezifität der Artefakte eingeschränkt, die methodische Vorgehensweise (Drei-Artefakte-Architektur) jedoch generalisierbar.

Reliabilität

- **Test-Retest-Reliabilität:** Jeder Proband absolvierte 6 Durchläufe pro Bedingung und System, was die Konsistenz der Messungen über mehrere Zeitpunkte hinweg ermöglicht.
- **Interrater-Reliabilität:** Reproduzierbarkeitstest durch drei unabhängige Probanden mit unterschiedlichen Master-Creator-Erfahrungsniveaus validiert, dass die entwickelten Artefakte (SOPs) von verschiedenen Personen konsistent angewendet werden können.
- **Methodische Kontrolle:** Counterbalanced Design reduziert Reihenfolgeeffekte; standardisiertes Experiment-Setup gewährleistet vergleichbare Bedingungen über alle Durchläufe hinweg.

Objektivität

- **Durchführungsobjektivität:** Standardisiertes Experiment-Setup mit klaren Anweisungen und identischen Aufgabenstellungen für alle Probanden minimiert Einflüsse durch unterschiedliche Versuchsbedingungen.
- **Auswertungsobjektivität:** Objektive Metriken (Zeit in MM:SS,ms, Anzahl erfüllter Checklisten-Punkte, Anzahl Fehler) minimieren Interpretationsspielräume bei der Datenauswertung.
- **Interpretationsobjektivität:** Die 6-Punkte-Checkliste definiert binäre Validierungskriterien (erfüllt/nicht erfüllt) ohne subjektive Bewertungsskalen. Die Einbeziehung von Probanden ohne Master-Creator-Vorerfahrung reduziert potenzielle Verzerrungen durch Entwicklernähe.

Die Kombination dieser Gütekriterien stellt sicher, dass die Evaluationsergebnisse des Experiments wissenschaftlich fundiert und nachvollziehbar sind.

5.7 LIMITIERUNGEN DER METHODIK

Die gewählte Methodik unterliegt spezifischen Einschränkungen:

1. **Eingebettete Forscherrolle:** Der Autor ist Teil des Konfigurationsteams, was zu Bestätigungstendenz bei der Artefakt-Entwicklung führen kann. Dies wird durch Kollegenfeedback während der Entwicklung, objektive Metriken (Zeit, Vollständigkeit, Checklisten-Punkte etc.) und einem Experiment mit standardisierten Bedingungen mitigiert.

2. **Overfitting-Prävention durch systematische Verallgemeinerung:** Das Demo-Werk wurde in Phase 0 dieser Arbeit entwickelt und bildete die empirische Grundlage für die in Phase 4 entwickelte Master-Creator-Automatisierung. Um Overfitting auf einen spezifischen Datensatz zu verhindern, erfolgte eine systematische Verallgemeinerung mehrerer Datenquellen (Restored_JSONS, alte Demo-Werk-Fragmente, generische Example-JSONs) über abgewandeltes Reverse Prompting zu allgemeinen ER-Kardinalitäten und JSON-Schemata, die im Entity Mapping Workbook dokumentiert wurden [Mayring, 2015]. Diese methodische Abstraktion stellt sicher, dass das Tool nicht auf die Besonderheiten eines einzelnen Projekts zugeschnitten ist, sondern generalisierbare Konfigurationsmuster verwendet.

3. **GRACE-Instabilität während Pilotphase:** Die Pilotphase war durch technische Instabilität geprägt: JSON-Import-Bugs verhinderten die Nutzung bestimmter Features (Properties, Maschinenreferenzen in Prozessen), und Systemcrashes beeinträchtigten die Entwicklungsarbeit. Der Master Creator fokussiert daher auf die Kernkonfiguration (6 Entitätstypen ohne optionale Erweiterungen), um trotz dieser Rahmenbedingungen und des zusätzlichen Zeitdrucks durch den NAS-Crash den Zeitrahmen einzuhalten. Die Bugs wurden nach Abschluss der Master-Creator-Entwicklung behoben, eine nachträgliche Erweiterung war jedoch aus Zeitgründen nicht mehr möglich.

Trotz dieser Limitierungen bietet die Action-Research-Methodik den Vorteil, der simultanen Entwicklung und Evaluierung praktischer Lösungen in einem realen Problemkontext. Ein Ansatz, der für die gegebenen Rahmenbedingungen optimal geeignet ist.

6 IMPLEMENTIERUNG

6.1 DEMO-WERK: SYNTHETISCHE KONFIGURATION

Als Demonstration und Validierungsbasis wurde eine synthetische Produktionskonfiguration für ein fiktives Werk in der Lack- und Farbenindustrie entwickelt. Das Demo-Werk simuliert die realistische Produktion von Pigmentpasten und dient als Referenzimplementierung für die entwickelten Artefakte.

Folgende Produktionen wurden modelliert:

- **WP-1000:** Weiße Pigmentpaste (1155,5 kg Batch)
- **RP-1000:** Rote Pigmentpaste (1155 kg Batch)
- **BP-1000:** Blaue Pigmentpaste (1155 kg Batch)

Der Produktionsprozess umfasst 7 sequenzielle Schritte mit unterschiedlichen Maschinenressourcen und Prozesszeiten, die sich an etablierten Prozessketten der Lackproduktion orientieren [Goldschmidt, 2019]. Besonders hervorzuheben ist die Integration einer Reinigungslogik: Materialien mit der Eigenschaft `IsCleaningMaterial`

`=true` werden ausschließlich im Reinigungsschritt verarbeitet. Alle anderen Materialien laufen durch die regulären Produktionsschritte.

Mengenabhängige Vorgabezeiten Drei der sieben Prozessschritte (Dosieren, Filtration, Abfüllen) weisen mengenabhängige Bearbeitungszeiten auf, die nach dem Prinzip der linearen Zeitfunktion modelliert wurden. Die Zeitbausteine folgen dem REFA-Standard REFA Verband für Arbeitsgestaltung, Betriebsorganisation und Unternehmensentwicklung [2012], während die produktionslogistische Verknüpfung von Materialmengen und Ressourcenbedarf auf Günther and Tempelmeier [2020] basiert:

$$t_{process} = \frac{Q_{Batch} \cdot 1\,\mathrm{s}}{1\,\mathrm{kg}} \tag{1}$$

wobei:

- $t_{process}$ = Bearbeitungszeit des Prozessschritts in Sekunden
- Q_{Batch} = Gesamtmenge der zugewiesenen BOM-Items (`TotalQuantityOfAssignedBomItems`) in kg

Diese Formel bildet das Verhalten von Prozessschritten ab, deren Dauer proportional zur verarbeiteten Menge ist. Für eine typische Charge von 1155 kg (z. B. WP-1000) ergibt sich somit:

- Dosieren: $t = 1155\,\mathrm{kg} \cdot 1\,\frac{\mathrm{s}}{\mathrm{kg}} = 1155\,\mathrm{s} \approx 19{,}25\,\mathrm{min}$
- Filtration: $t = 1155\,\mathrm{s} \approx 19{,}25\,\mathrm{min}$
- Abfüllen: $t = 1155\,\mathrm{s} \approx 19{,}25\,\mathrm{min}$

Im Gegensatz dazu verwenden die Schritte Mischen, Rühren und Dispergieren konstante Bearbeitungszeiten (2100s, 600s, 1800s), da deren Dauer primär von physikalischen Prozessanforderungen (z. B. Homogenisierungsgrad, Partikelgrößenverteilung) und weniger von der absoluten Menge abhängt. Der Maschinenführer hat offenbart, dass bei derartigen Prozessschritten meistens konstante Erwartungswerte verwendet werden, so wie hier modelliert.
Die Implementierung dieser Zeitfunktionen erfolgt über verschachtelte JSON-Strukturen mit `BinaryOperation`-Ausdrücken, genauso wie in GRACE:

```
"processingTime": {
  "type": "BinaryOperation",
  "operator": "Divide",
  "leftOperand": {
    "type": "BinaryOperation",
    "operator": "Multiply",
    "leftOperand": {
      "type": "Variable",
      "variable": "TotalQuantityOfAssignedBomItems"
    },
    "rightOperand": {"type": "Constant", "value": 1, "unitId": "s"}
  },
  "rightOperand": {"type": "Constant", "value": 1, "unitId": "kg"}
}
```

STATISTIK DER KONFIGURATION

Tabelle 1 zeigt die quantitative Zusammensetzung des Demo-Werks.

Tabelle 1: Statistik der Demo-Werk-Konfiguration

Entity-Typ	**Anzahl**
Materialien	22
Maschinen	7
Produkte	3
BOMs (Stücklisten)	3
Produktionsprozesse	1 (mit 7 Schritten)
Rezepte	3
Gesamt	**39 Entitäten**

Die 22 Materialien unterteilen sich in:

- **Rohmaterialien:** 20 Substanzen (Pigmente wie Titanium Dioxide, Bindemittel wie PEG 400, Additive wie Tego Dispers 750 W, etc.)
- **Reinigungsflüssigkeit:** 1 spezielles Material für Reinigungsprozesse
- **Endprodukte:** 3 Pigmentpasten (weiß, rot, blau)

Die 7 Maschinen repräsentieren den typischen Maschinenpark einer Pigmentpaste-Produktion [Brock et al., 2020]:

- Dissolver (Auflösen von Pigmenten)
- Highspeed-Dispergierer (Feinverteilung)
- Perlmühle (Mahlung und Dispergierung)
- Schraubenspindelpumpe (Materialförderung)
- Wagensystem (Transport)
- Druckfiltersystem (Filtration)
- CIP-Reinigungssystem (Clean-in-Place)

Die 3 Rezepte entsprechen den 3 Produktvarianten (WP-1000, RP-1000, BP-1000), wobei jedes Rezept die spezifische Stückliste mit dem entsprechenden Produktionsprozess verknüpft.

DATENQUELLEN UND DATENSOUVERÄNITÄT

Die Entwicklung des Demo-Werks nutzte sowohl interne als auch öffentlich zugängliche Quellen. Die resultierende Konfiguration wurde jedoch so gestaltet, dass sie sich vollständig aus nicht-vertraulichen Quellen ableiten lässt. Bewusst wurden keine kundenbezogenen Produktionsdaten übernommen, um DSGVO-Compliance und NDA-Anforderungen zu wahren. Alle Materialbezeichnungen, Rezepturen und Prozessparameter basieren auf generischem Domänenwissen und sind frei von proprietären Informationen.

KOMPLEXITÄT UND REALITÄTSNÄHE

Die Größenordnung des Demo-Werks (39 Entitäten) wurde bewusst gewählt, um:

1. **Realistische Komplexität** abzubilden: Kleinere Produktionswerke weisen typischerweise 15-30 Materialien, 5-10 Maschinen und 3-5 Hauptprodukte auf. Das Demo-Werk liegt somit im unteren bis mittleren Komplexitätsbereich.

2. **Validierung der Artefakte** zu ermöglichen: Die Konfiguration ist komplex genug, um alle 6 Entitätstypen, multi-direktionale Abhängigkeiten und kritische Validierungsregeln zu testen.

3. **Angemessenen Arbeitsaufwand** für eine Bachelorarbeit zu gewährleisten: Eine vollständige Industriekonfiguration mit 100+ Materialien und 20+ Maschinen war zeitlich nicht realisierbar, da das Demo-Werk bis zum 6. November 2025 für die EC Conference Digitalisation fertiggestellt werden musste. Die gewählte Größenordnung ermöglicht dennoch den vollständigen methodischen Erkenntnisgewinn. Alle relevanten Konfigurationsherausforderungen (Abhängigkeiten, Validierung, Rezepturkomplexität) sind abgebildet.

4. **Nachvollziehbarkeit** zu sichern: Die überschaubare Größe und nicht vertrauliche Basis ermöglicht es, alle Entitäten im Anhang zu dokumentieren und die Konfigurationslogik transparent zu machen.

Die Konfiguration wurde iterativ entwickelt und diente als Testbed für die in Kapitel 5.2 beschriebenen Entwicklungsphasen.

6.2 GRACE ONBOARDING FRAGEBOGEN

Der GRACE Onboarding Fragebogen ist das erste Artefakt der Drei-Artefakte-Architektur und dient der systematischen Erhebung konfigurationsrelevanter Informationen beim Kunden. Die Entwicklung des Fragebogens folgte den Prinzipien der Skalenentwicklung nach DeVellis DeVellis [2017] und Bühner Bühner [2004] und umfasste mehrere Iterationszyklen zur Reduktion der initialen Fragenanzahl.

ENTWICKLUNGSPROZESS: VON 96 AUF 15 FRAGEN

Die initiale Version des Fragebogens entstand durch Brainstorming aller potenziell relevanten Informationen für eine GRACE-Konfiguration [seitcom GmbH, 2025b]. Dabei flossen Erkenntnisse aus einem vorherigen APS-Marktforschungsinterviewskript ein, welches vom Autor zum Marketingmodul-124 bei Frau Prof. Dr. Sessler aus dem vorherigen Studienverlauf erarbeitet wurde. [seitcom GmbH, 2024a] Im Skript waren generische Produktionsplanungsanforderungen verschiedener Fertigungsbranchen. Dieser Prozess führte zu einer Vollversion mit 96 Fragen [seitcom GmbH, 2025e], strukturiert in 8 Teile:

- Teil A: Grundinformationen & Zielsetzung (6 Fragen)
- Teil B: IST-Zustand IT-Landschaft (19 Fragen)
- Teil C: Produktionsdetails - 6 Entitätstypen (43 Fragen)
- Teil D: Planung & Optimierung (6 Fragen)
- Teil E: Restriktionen & Constraints (18 Fragen)
- Teil F: Datenexport & Schnittstellen (3 Fragen)
- Teil G: Spezifische Konfiguration (variabel, live mit Master Creator)
- Teil H: Nächste Schritte & Erwartungen (3 Fragen)

Die Vollversion erwies sich in praktischen Tests als zu umfangreich für initiale Kundengespräche: Die geschätzte Bearbeitungszeit von 3-4 Stunden überforderte Teilnehmer und führte zu oberflächlichen Antworten in späteren Fragenblöcken. Zudem zeigte sich, dass viele Detailfragen erst nach grundlegendem Verständnis der Produktionsumgebung sinnvoll beantwortet werden können.
Die Reduktion auf eine **Kompaktversion mit 15 Fragen** [seitcom GmbH, 2025d] erfolgte nach den Prinzipien von DeVellis [2017] und Bühner [2004] durch:

1. **Identifikation redundanter Items:** Fragen, die ähnliche Informationen abfragen, wurden zusammengeführt (z. B. wurden 6 separate ERP-Fragen zu einer Multi-Choice-Frage komprimiert).
2. **Eliminierung zu spezifischer Items:** Detailfragen, die nur für bestimmte Branchen oder Produktionstypen relevant sind, wurden in die Vollversion verschoben (z. B. spezifische Fragen zu Lohnherstellung oder Chargenverfolgung).
3. **Priorisierung nach Entscheidungsrelevanz:** Fragen, die essentiell für die Go/No-Go-Entscheidung sind, wurden priorisiert (z. B. „Können Daten exportiert werden?“ist kritischer als „Welches Format hat der Export?“).
4. **Konsolidierung in Multi-Choice-Formate:** Mehrere Ja/Nein-Fragen wurden zu strukturierten Checkboxen zusammengefasst, was die Beantwortungszeit reduziert und die Übersichtlichkeit erhöht.

Tabelle 2 zeigt die Fragenverteilung nach der Reduktion.

Tabelle 2: Fragebogen-Reduktion: Vollversion vs. Kompaktversion

Teil	Thema	Vollversion	Kompakt
A	Grundlagen	6	3
B	IT-Landschaft	19	2
C	Produktionsdetails	43	4
D	Planung & Optimierung	6	2
E	Restriktionen & Downtimes	18	3
F	Next Steps	3	1
G	Spezifische Konfiguration	variabel	-
Gesamt		**96**	**15**

STRUKTUR UND BEISPIELFRAGEN DER KOMPAKTVERSION

Die Kompaktversion ist in sechs thematische Blöcke gegliedert. Im Folgenden wird jeder Block mit einer repräsentativen Beispielfrage vorgestellt.

Teil A: Grundlagen (3 Fragen) Erfassung grundlegender Unternehmensinformationen (Branche, Produktionstyp, Größe).

Beispielfrage 1: *„Was produzieren Sie, wie viele Mitarbeiter, welche Branche?“*
Konsolidiert sechs Einzelfragen aus der Vollversion in strukturiertes Format mit Checkboxen für Produktionstyp (diskret/prozess/hybrid) und Geschäftsmodell (Lohnhersteller/Eigenproduzent).

Teil B: IT-Landschaft (2 Fragen) Identifikation bestehender IT-Systeme und Datenquellen für Schnittstellenplanung.

Beispielfrage 4: *„Welches ERP-System verwenden Sie und welche weiteren Produktionssysteme (MES, WMS, LIMS)?“*
Multiple-Choice-Optionen für ERP-System (SAP, Oracle, MS Dynamics) sowie MES, BDE und LIMS. Kritisches Feld: „Excel-Nutzung“ (Ja, intensiv / Teilweise / Nein) intensive Excel-Nutzung signalisiert fragmentierte Datenlandschaft.

Teil C: Produktionsdetails (4 Fragen) Quantitative Erfassung der sechs GRACE-Entitätstypen (Materialien, Maschinen, Produkte, Prozesse, BOMs, Rezepte).

Beispielfrage 6: *„Wie viele Materialien (Roh/Zwischen/End)? Wo liegen die Rezepturen/BOMs?“*
Ermittelt Konfigurationsgröße und Wissensquellen. Optionen: ERP, Excel, Papier, „Köpfe der Mitarbeiter“ letzteres signalisiert hohen Externalisierungsbedarf.

Teil D: Planung & Optimierung (2 Fragen) Erfassung strategischer APS-Ziele und Optimizer-Zielfunktion.

Beispielfrage 11: *„Was soll minimiert/maximiert werden? Priorisierungskriterien?“*
Checkboxen für gängige Ziele (Durchlaufzeit minimieren, Auslastung maximieren) plus Freitextfeld für spezifische Kriterien (z. B. „Farbsequenz hell→dunkel“). Identifiziert auch saisonale Zielfunktionsprofile.

Teil E: Restriktionen & Downtimes (3 Fragen) Erfassung zeitlicher, räumlicher und technischer Constraints.

Beispielfrage 13: *„Wie lange dauern Umrüstungen? Reinigungszyklen?“*
Erfasst Rüstzeiten und prüft Existenz einer Rüstmatrix. Feld „Sequenzabhängig (z. B. Farben)“ identifiziert variable Reinigungszeiten je nach Produktreihenfolge (in Prozessindustrie kritisch).

Teil F: Next Steps (1 Frage) Klärung organisatorischer Rahmenbedingungen und Projektziele.

Beispielfrage 15: *„Wann soll GRACE live gehen? Wer ist Ansprechpartner? Erfolgsmetrik nach 6 Monaten?“*
Erfasst Go-Live-Datum, Hauptansprechpartner und messbare Erfolgskriterien. Feld „Darstellung Produktionsplan“ (Digital/Papier/Beides) beeinflusst UI-Anforderungen.

VOLLVERSION VS. KOMPAKTVERSION: EINSATZSZENARIEN

Die beiden Fragebogenversionen erfüllen unterschiedliche Zwecke im Konfigurationsprozess:

Kompaktversion (15 Fragen):

- **Einsatz:** Pre-Workshop-Vorbereitung, initiales Screening, Executive Summary
- **Dauer:** 30-45 Minuten
- **Ziel:** Entscheidung, ob GRACE zum Kunden passt; grobe Abschätzung des Konfigurationsaufwands
- **Teilnehmer:** Geschäftsführung, Produktionsleitung (Überblickswissen ausreichend)

Vollversion (96 Fragen):

- **Einsatz:** Detaillierter Workshop nach Projektzusage, während der Master-Creator-Nutzung als Nachschlagwerk
- **Dauer:** 3-4 Stunden (verteilt auf mehrere Sessions)
- **Ziel:** Vollständige Informationserhebung für fehlerfreie Konfiguration
- **Teilnehmer:** Produktionsplaner, Prozessingenieure, IT-Verantwortliche (Detailwissen erforderlich)

Die Kompaktversion deckt nach dem Pareto-Prinzip etwa 80% der kritischen Informationen ab, während die Vollversion die verbleibenden 20% für technische Details erfasst. In der Praxis hat sich gezeigt, dass die Kompaktversion für kleinere Werke (< 30 Materialien, < 10 Maschinen) oft ausreicht, während komplexere Konfigurationen die Vollversion erfordern.

FORMALE GESTALTUNG UND VERSIONIERUNG

Beide Fragebogenversionen sind als Markdown-Dokumente versioniert und enthalten:

- **Checkboxen** für Ja/Nein- und Multiple-Choice-Fragen (`- [ ]`)
- **Platzhalter** für Texteingaben (markiert als `` `...` `` zum Überschreiben)
- **Status-Tracking** pro Frage („Offen“ / „Beantwortet“)
- **Mapping-Tabelle** in der Kompaktversion, die zeigt, welche Kompakt-Frage welchen Vollversions-Fragen entspricht

Die Versionsnummer (aktuell: 2.1 für Kompakt, 1.0 für Vollversion) ermöglicht Nachverfolgbarkeit bei Änderungen der Fragenstruktur. Änderungen werden durch Praxiserfahrung aus Kundenprojekten getrieben: So wurde beispielsweise in Version 2.1 das Feld „Exoten-Produkte“ ergänzt, nachdem sich in einem Projekt zeigte, dass umsatzrelevante Sonderprodukte mit abweichenden Prozessen oft übersehen werden.

6.3 ENTITY MAPPING WORKBOOK: TECHNISCHE SPEZIFIKATION

Das Entity Mapping Workbook ist das zweite Artefakt der Drei-Artefakte-Architektur und dokumentiert die JSON-Schema-Strukturen aller sechs GRACE-Entitätstypen. Es dient als technische Referenz für die LLM-gestützte Konfigurationsgenerierung und definiert Feldspezifikationen, Validierungsregeln und Abhängigkeiten zwischen Entitäten.

ENTWICKLUNGSPROZESS: REVERSE ENGINEERING DER GRACE-DATENSTRUKTUREN

Die GRACE-Plattform besitzt keine öffentlich zugängliche API-Dokumentation oder JSON-Schema-Spezifikation. Die Entwicklung des Entity Mapping Workbooks erfolgte daher durch systematisches Reverse Engineering:

1. **JSON-Extraktion:** Export bestehender Konfigurationen aus der GRACE-Entwicklerumgebung über die Web-UI
2. **Strukturanalyse:** Manuelle Inspektion der exportierten JSON-Dateien zur Identifikation von Feldtypen, Datenformaten und referenziellen Beziehungen
3. **Validierungstests:** Iterative Modifikation von JSON-Strukturen und Re-Import in GRACE zur Identifikation von Constraints (z. B. Pflichtfelder, erlaubte Werte, ID-Formate)
4. **Fehleranalyse:** Systematisches Provozieren von Importfehlern zur Ermittlung von Validierungsregeln (z. B. Zeiteinheiten, BOM baseQuantity-Konsistenz)
5. **Dokumentation:** Formalisierung der Erkenntnisse in tabellarischer Form mit Feldspezifikationen, Datentypen, Defaults und Constraints

Dieser Prozess wurde durch die Demo-Werk-Entwicklung (Phase 0, siehe Kapitel 5.2) getrieben: Jeder aufgetretene Konfigurationsfehler führte zur Verfeinerung der Workbook-Dokumentation.

ÜBERSICHT DER ENTITY-TYPEN

Tabelle 3 zeigt die sechs GRACE-Entitätstypen und ihre Abhängigkeiten.

Tabelle 3: Übersicht der GRACE-Entitätstypen und Abhängigkeiten

Entity-Typ	Beschreibung	Abhängigkeiten
Material	Rohstoffe, Produkte	Keine
Machine	Produktionsressourcen	Keine
Product	Hergestellte Erzeugnisse	Material
Process	Produktionsabläufe	Optional: Machine
BOM	Stücklisten	Material
Recipe	Produktionsrezepte	Material, BOM, Process

Die Abhängigkeitsstruktur determiniert die Erstellungsreihenfolge: Materials und Machines können unabhängig erstellt werden, während Recipes alle anderen Entitäten voraussetzen.

ENTITY-TYP 1: MATERIAL

Beschreibung Materials repräsentieren Rohstoffe, Substanzen, Pigmente oder Zwischen- und Endprodukte im Produktionsprozess.

JSON-Struktur

```
{
  "id": "string",
  "name": "string",
  "comment": "string"
}
```

Kritische Felder

- `id`: Eindeutiger Bezeichner, Format `lowercase-with-hyphens`, Regex: `^[a-z0-9-]+$`
- `name`: Anzeigename (menschenlesbar)
- `comment`: Optional, Default: `""`

Besonderheiten Materials haben keine Abhängigkeiten und werden automatisch erstellt, wenn von Products oder BOMs referenziert. Die ID-Konvention folgt dem Muster: Produktname in Kleinbuchstaben mit Bindestrichen (z. B. `titanium-dioxide`, `peg-400`).

ENTITY-TYP 2: MACHINE

Beschreibung Machines repräsentieren Produktionsressourcen wie Mixer, Dissolver, Pumpen oder andere Anlagen.

JSON-Struktur

```
{
  "id": "string",
  "name": "string",
  "operatingCosts": {
    "type": "Constant",
    "value": 0
  },
  "isSpatial": false,
  "initialUnits": 1,
  "unitsAutoGrow": false
}
```

Kritische Felder

- `initialUnits`: Anzahl verfügbarer Einheiten (bei 3 identischen Mixern: `initialUnits=3`)
- `operatingCosts.value`: Wird niemals geraten, bei Unklarheit: 0
- `isSpatial`, `unitsAutoGrow`: Standardmäßig immer `false`

Besonderheiten Mehrere identische Maschinen werden durch `initialUnits` abgebildet, nicht durch separate Entities. Mehrere unterschiedliche Maschinen erfordern separate Machine-Entities.

ENTITY-TYP 3: PRODUCT

Beschreibung Products repräsentieren hergestellte Erzeugnisse und verweisen auf das resultierende Material.

JSON-Struktur

```
{
  "id": "{produktname}-prod",
  "name": "string",
  "material": {
    "version": 1,
    "id": "{produktname}"
  }
}
```

Kritische Felder

- `material.id`: **KRITISCH** referenziert das Produkt selbst, **NICHT** die Zutaten!
- `id`: Suffix `-prod` anhängen
- `material.version`: Immer `1`

Besonderheiten Dies ist die häufigste Fehlerquelle: `material.id` wird oft fälschlicherweise auf Rohmaterialien gesetzt statt auf das Endprodukt. Beispiel: Bei „Blue Pigment Paste“ muss `material.id` = `"blue-pigment-paste"` sein, nicht `"titanium-dioxide"`.

ENTITY-TYP 4: PROCESS

Beschreibung Processes bilden Produktionsabläufe mit mehreren zeitlich sequenzierten Prozessschritten ab.

JSON-Struktur

```
{
  "id": "string",
  "name": "string",
  "steps": [
    {
      "id": "step-1",
      "name": "Mixing",
      "processingTime": {
        "type": "Constant",
        "value": 900,
        "unitId": "s"
      },
      "resourceDemands": [],
      "isParallelizable": false
    }
  ],
  "links": []
}
```

Kritische Felder

- `processingTime.unitId`: **MUSS IMMER** `"s"` sein, **NIEMALS** `"min"` oder `"minutes"`
- `processingTime.value`: Zeitwert **immer in Sekunden** (15 Minuten = 900 Sekunden)
- `resourceDemands`: Standard: leeres Array `[]`
- `isParallelizable`: Standard: `false`

Besonderheiten Zeitumrechnung ist die zweithäufigste Fehlerquelle. Zeitangaben in Minuten müssen in Sekunden umgerechnet werden (Faktor 60). Die sequenzielle Abfolge wird durch die Array-Reihenfolge der Steps determiniert, nicht durch `links`.

ENTITY-TYP 5: BOM (BILL OF MATERIALS)

Beschreibung BOMs (Stücklisten) definieren die Zusammensetzung eines Produkts aus Materialien mit Mengenangaben.

JSON-Struktur

```
{
  "id": "{produktname}-bom",
  "name": "string",
  "baseQuantity": 100,
  "unitId": "kg",
  "items": [
    {
      "key": "1",
      "material": {"version": 1, "id": "material-1"},
      "quantity": 60
    },
    {
      "key": "2",
      "material": {"version": 1, "id": "material-2"},
      "quantity": 40
    }
  ]
}
```

Kritische Felder

- `baseQuantity`: **MUSS** exakte Summe aller `items[].quantity` sein
- `items[].key`: Sequenziell nummeriert (`"1"`, `"2"`, `"3"`, ...)
- `id`: Suffix `-bom` anhängen

Besonderheiten Die `baseQuantity`-Konsistenz wird im Master Creator durch JavaScript automatisch validiert und korrigiert, da LLMs in arithmetischen Berechnungen fehleranfällig sind. Beispiel: Items mit Mengen [60, 30, 10] erfordern `baseQuantity=100`.

ENTITY-TYP 6: RECIPE

Beschreibung Recipes verbinden Material, BOM und Process zu einer vollständigen Produktionsdefinition.

JSON-Struktur

```
{
  "id": "{produktname}-recipe",
  "name": "string",
  "resultingMaterial": {
    "version": 1,
    "id": "{endprodukt}"
  },
  "bom": {
    "version": 1,
    "id": "{produktname}-bom"
  },
  "productionProcess": {
    "version": 1,
    "id": "{process-id}"
  }
}
```

Kritische Felder

- `resultingMaterial.id`: Referenz zum Endprodukt-Material
- `bom.id`: Referenz zur Stückliste
- `productionProcess.id`: **WICHTIG** Prozess-ID **ohne** `-process`-Suffix!
- Alle `version`-Felder: Immer `1`

Besonderheiten Recipes sind die komplexeste Entität, da alle drei referenzierten Entities (Material, BOM, Process) existieren müssen. Die Process-ID wird aus dem Prozessnamen abgeleitet: „Dosing Mixing Filtration Process“ wird zu `dosing-mixing-filtration` (kein `-process`-Suffix).

VALIDIERUNGS-CHECKLISTE

Das Entity Mapping Workbook definiert eine 6-Punkte-Checkliste zur Validierung vor dem Export:

1. **Vollständigkeit:** Alle 6 Entity-Typen vorhanden (sofern relevant)
2. **ID-Konsistenz:** Keine kaputten Referenzen zwischen Entities
3. **Zeiteinheiten:** Alle Process-Steps haben `unitId:"s"`
4. **BOM baseQuantity:** Summe aller Item-Quantities = baseQuantity
5. **Product Material:** `Product.material.id` referenziert korrektes Material (Produkt selbst)
6. **ID-Duplikate:** Keine duplizierten IDs innerhalb eines Entity-Typs

Diese Checkliste bildet die Grundlage für die JavaScript-basierte Validierungslogik im Master Creator (siehe Abschnitt 6.6).

ID-NAMENSKONVENTIONEN

Einheitliche ID-Konventionen gewährleisten Konsistenz:

- Material: `lowercase-with-hyphens` (z. B. `titanium-dioxide`)
- Machine: `lowercase-with-hyphens` (z. B. `high-speed-mixer`)
- Product: `\{produktname\}-prod` (z. B. `white-paste-prod`)
- Process: `lowercase-with-hyphens` (z. B. `mixing-dispersion`)
- BOM: `\{produktname\}-bom` (z. B. `white-paste-bom`)
- Recipe: `\{produktname\}-recipe` (z. B. `white-paste-recipe`)

Regel: Keine Großbuchstaben, keine Leerzeichen, keine Sonderzeichen außer Bindestrichen.

6.4 STANDARD OPERATING PROCEDURES (SOPS)

Die drei entwickelten Standard Operating Procedures dokumentieren den End-to-End-Prozess von der initialen Kundenkommunikation bis zum Import der generierten Konfiguration in GRACE. Sie operationalisieren die Drei-Artefakte-Architektur als ausführbare Handlungsanweisungen und sollen zur Reduktion der Wissensträgerabhängigkeit beitragen.

SOP 01: KUNDENWORKSHOP DURCHFÜHREN

Zweck Diese SOP beschreibt den strukturierten Ablauf eines Kundenworkshops zur Erhebung aller notwendigen Informationen für die GRACE-Konfiguration. Sie nutzt den GRACE Onboarding Fragebogen (siehe Abschnitt 6.2) als primäres Erhebungsinstrument.

Voraussetzungen

- GRACE Onboarding Fragebogen (Kompaktversion, 15 Fragen)
- Laptop/Tablet für Notizen
- Entity Mapping Workbook als Referenz
- Zeitrahmen: 2-3 Stunden
- Teilnehmer: 2-4 Personen (Kunde + Konfigurationsteam)

Kernablauf
Der Workshop gliedert sich in drei Phasen:

1. **Einleitung (15 Minuten):** Vorstellung, GRACE-Überblick, Erwartungsmanagement, Vertraulichkeitszusicherung (NDA/DSGVO)

2. **Informationserhebung (90-120 Minuten):** Strukturierte Abfrage entlang der 6 Themenblöcke des Fragebogens:
    - Block A: Werk und Organisation (5 Min.)
    - Block B: IT-Landschaft (15 Min.)
    - Block C: Materialien, Maschinen, Produkte, Prozesse (60 Min.)
    - Block D: Planung und Optimierung (20 Min.)
    - Block E: Restriktionen und Downtimes (15 Min.)
    - Block F: Next Steps und Timeline (10 Min.)
3. **Zusammenfassung und Abschluss (15 Minuten):** Recap der gesammelten Informationen, Klärung offener Fragen, Vereinbarung nächster Schritte

Dokumentation Das Workshop-Protokoll wird als ausgefüllter Fragebogen dokumentiert und dient als Input für SOP 02 (Master Creator Nutzung). Die vollständige SOP mit detaillierten Anweisungen, Gesprächsleitfäden und Best Practices ist im Anhang dokumentiert.

SOP 02: MASTER CREATOR NUTZEN

Zweck Diese SOP beschreibt die Nutzung des Master Creator zur automatisierten Generierung von GRACE-Konfigurationsdaten aus dem Workshop-Protokoll. Sie definiert die empfohlene Erstellungsreihenfolge, technische Voraussetzungen und Validierungsschritte.

Voraussetzungen

- Ollama installiert und laufend
- qwen2.5-coder:7b Modell verfügbar
- Master Creator Webanwendung (Port 9000)
- Workshop-Protokoll aus SOP 01
- Entity Mapping Workbook als Referenz
- 6-Punkte-Validierungs-Checkliste

Empfohlene Erstellungsreihenfolge Die 6 Entity-Typen werden in definierter Reihenfolge generiert, da Abhängigkeiten bestehen:

Nr.	Entity-Typ	Abhängigkeiten	Input
1	Materials	Keine	Materialliste aus Workshop
2	Machines	Keine	Maschinenliste aus Workshop
3	Products	Materials (auto)	Produktliste aus Workshop
4	Processes	Opt: Machines	Prozessschritte mit Zeitangaben
5	BOMs	Materials (auto)	Stücklisten mit Mengen
6	Recipes	Mat., BOM, Proc.	Verknüpfungen

Kernablauf Für jeden Entity-Typ:

1. **Prompt eingeben:** Informationen aus Workshop in natürlichsprachigem Prompt formulieren
2. **AI-Generierung:** Master Creator erkennt Entity-Typ automatisch und generiert JSON
3. **Validierung:** 6-Punkte-Checkliste prüfen (Vollständigkeit, ID-Konsistenz, Zeiteinheiten, etc.)
4. **Korrektur:** Bei Fehlern manuell korrigieren oder Prompt neu formulieren
5. **Export:** JSON-Datei herunterladen

Besonderheit bei Recipes: Der Master Creator nutzt Session-Context, um existierende Entity-IDs (Material, BOM, Process) automatisch zu erkennen. Daher müssen alle Abhängigkeiten vor der Recipe-Erstellung vorhanden sein.

Validierung Die 6-Punkte-Checkliste (siehe Abschnitt 6.3) wird automatisch im Master Creator UI angezeigt. Häufige Fehler:

- `Product.material.id` referenziert Zutat statt Produkt
- Process-Zeiten in Minuten statt Sekunden
- `BOM.baseQuantity` stimmt nicht mit Summe der Items überein

Die vollständige SOP mit Screenshots, Fehlerbehandlung und Troubleshooting ist im Anhang dokumentiert.

SOP 03: JSONS IN GRACE IMPORTIEREN

Zweck Diese SOP beschreibt den Import der generierten JSON-Konfigurationsdateien in das GRACE-System. Sie definiert die korrekte Import-Reihenfolge, Fehlerbehandlung und Validierung des Import-Ergebnisses.

Voraussetzungen

- GRACE-Zugriff (Login-Credentials)
- Browser (Chrome, Firefox, Edge)
- 6 JSON-Dateien aus SOP 02 verfügbar
- Import-Reihenfolge dokumentiert

Import-Reihenfolge (KRITISCH) Die Import-Reihenfolge muss zwingend eingehalten werden, da GRACE referenzielle Integrität prüft:

Nr.	Entity-Typ	Abhängigkeiten	Dauer
1	Materials	Keine	30 Sek.
2	Machines	Keine	30 Sek.
3	Products	Materials	30 Sek.
4	Processes	Opt: Machines	1 Min.
5	BOMs	Materials	30 Sek.
6	Recipes	Mat., BOM, Proc.	30 Sek.
Gesamtdauer			**4-5 Min.**

Kernablauf Für jeden Entity-Typ:

1. **GRACE-UI öffnen:** Navigation zu Configuration → Import
2. **Entity-Typ auswählen:** Dropdown-Menü (z. B. „Materials“)
3. **JSON-Datei hochladen:** Drag-and-Drop oder File-Browser
4. **Import starten:** Button „Import“ klicken
5. **Ergebnis validieren:** Erfolgsmeldung prüfen, Anzahl importierter Entities überprüfen
6. **Fehlerbehandlung:** Bei Fehlern Fehlermeldung analysieren, JSON korrigieren, Re-Import

Validierung nach Import Nach Abschluss aller 6 Imports:

- **Vollständigkeit:** Alle Entities in GRACE sichtbar?
- **Referenzen:** Recipes zeigen korrekte Verknüpfungen zu Materials, BOMs, Processes?
- **Testplanung:** Testauftrag erstellen und Planungsergebnis prüfen

Die vollständigen SOPs sind im Anhang dokumentiert.

ZUSAMMENSPIEL DER SOPS

Die drei SOPs bilden einen durchgängigen Prozess:

SOP 01 (Workshop) → Protokoll → SOP 02 (Master Creator) → JSON-Dateien → SOP 03 (Import) → GRACE-Konfiguration

Jede SOP ist als Markdown-Dokument versioniert und enthält Checkboxen für Status-Tracking, klare Anweisungen und Verweise auf weiterführende Ressourcen (Entity Mapping Workbook, Fragebogen, Validierungs-Checkliste). Die Kombination der SOPs soll es auch Teammitgliedern ohne vorherige Master-Creator-Erfahrung ermöglichen, eigenständig GRACE-Konfigurationen zu erstellen. Das Zusammenspiel der SOPs bildet sich implizit auch in den Prozessdiagrammen ab und können zur Verständnishilfe genutzt werden.

6.5 MASTER CREATOR: ARCHITEKTUR UND FUNKTIONSWEISE

Der Master Creator ist das zentrale technische Werkzeug zur LLM-gestützten Generierung von GRACE-Konfigurationsdaten. Er implementiert die in der Drei-Artefakte-Architektur definierte Externalisierungslogik als interaktive Webanwendung und transformiert natürlichsprachige Eingaben in strukturierte JSON-Dokumente gemäß dem Entity Mapping Workbook.

TECHNISCHE INFRASTRUKTUR

Der Master Creator basiert auf einer lokalen LLM-Deployment-Architektur zur Gewährleistung von Datensouveränität. Die technischen Komponenten umfassen:

LLM-Backend: Ollama mit qwen2.5-coder:7b Das LLM-Backend nutzt Ollama Ollama Team [2024] als Deployment-Framework für das Sprachmodell qwen2.5-coder:7b. Die Wahl dieses Modells erfolgte unter Berücksichtigung mehrerer Faktoren:

- **Datensouveränität:** Lokales Deployment verhindert Übertragung sensibler Kundendaten an externe Cloud-Dienste (DSGVO-/NDA-Compliance)
- **Hardwarebeschränkungen:** 7B-Parameter-Modell ist auf Consumer-Hardware ausführbar (MacBook M4 mit 16 GB RAM)
- **Code-Spezialisierung:** qwen2.5-coder ist auf Codegenerierung optimiert, was für strukturierte JSON-Ausgabe vorteilhaft ist
- **Trade-off:** Bewusste Akzeptanz reduzierter Leistung gegenüber größeren Cloud-Modellen (70B+ Parameter) zugunsten von Privacy-Compliance

Ollama stellt eine REST-API auf Port 11434 bereit, die vom Frontend per HTTP-POST angesprochen wird.

Frontend: HTML/CSS/JavaScript Das Frontend ist als Single-Page-Application (SPA) implementiert:

- **HTML:** Strukturierung der Chat-UI, Entity-Listen, Validierungs-Anzeige
- **CSS:** Styling mit visueller Unterscheidung der 6 Entity-Typen (farbcodiert)
- **JavaScript:** Kernanwendungslogik → Ollama-API-Kommunikation, JSON-Extraktion, Validierung, Auto-Correction, Session-Management, Export

Das Frontend wird über einen Python HTTP-Server auf Port 9000 ausgeliefert und läuft vollständig im Browser ohne serverseitige Persistierung.

Multilinguale LLM-Performance: Begründung für englische Prompts Die Entscheidung für englischsprachige Prompts und JSON-Feldnamen (trotz deutschem Projektumfeld) basiert auf empirischen Befunden zur multilingualen LLM-Performance. Gupta et al. [2025] zeigen, dass die Performance von Sprachmodellen bei nicht-englischen Sprachen signifikant abnimmt: Englisch erreicht 97,6% Genauigkeit, während Telugu nur 86,9% erreicht, ein Abfall von 5-30% je nach Aufgabentyp. Die Autoren führen dies auf die Verteilung der Trainingsdaten zurück: CommonCrawl enthält 42,8% englische Daten, aber nur 5,5% deutsche.
Im Kontext dieser Arbeit bedeutet dies: Englische Prompts ermöglichen präzisere Entity-Type-Erkennung, akkuratere JSON-Generierung und konsistentere Validierung. Dieser Performance-Gewinn rechtfertigt die sprachliche Wahl, da:

- Das Konfigurationsteam technisch versiert ist und englische Fachterminologie nutzt
- JSON-Strukturen ohnehin technische (oft englische) Feldnamen verwenden
- Die Ressourcenbeschränkung auf ein 7B-Modell jede verfügbare Performance-Optimierung notwendig macht

In späteren Entwicklungsstufen mit mehr Hardware-Kapazitäten sollte eine Umstellung auf Deutsch oder eine Deutsche Sprachvariante des Master Creators angestrebt werden. Die Entwicklung dessen liegt jedoch außerhalb des Scopes dieser Arbeit. Aus diesem Grund wird diese Idee innerhalb dieser Thesis nicht mehr weiter erörtert, auch wenn Einzeltests ergeben haben, dass sogar mit der limitierten Hardware einfache Konfigurationen generierbar sind.

SYSTEMARCHITEKTUR UND DATENFLUSS

Abbildung 2 zeigt die Systemarchitektur des Master Creator.

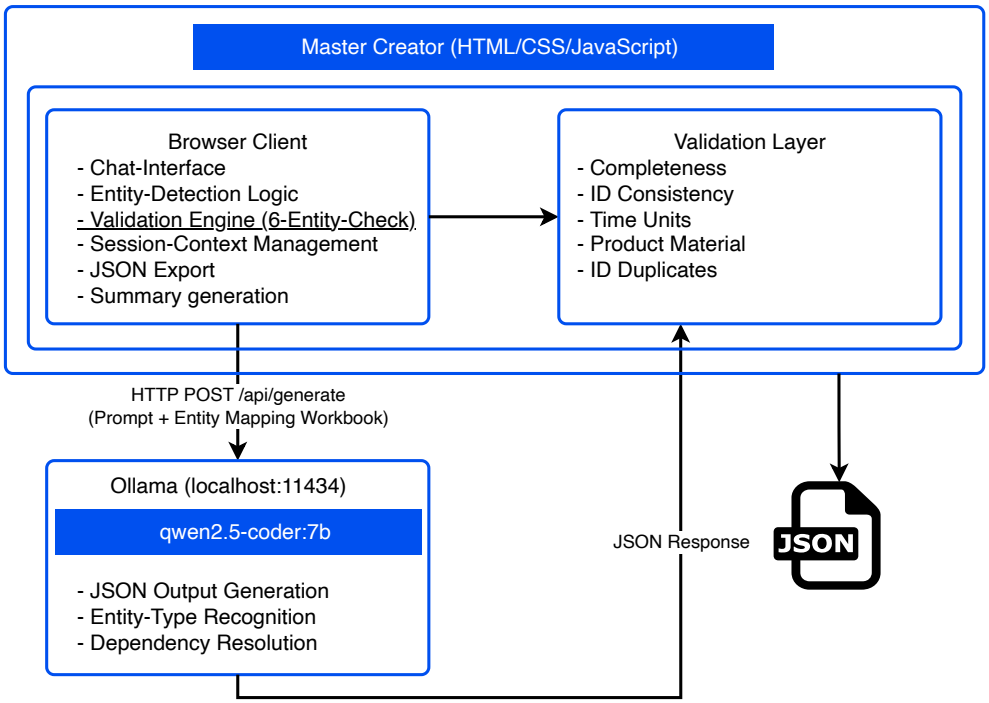

Abbildung 2: Master Creator Systemarchitektur und Datenfluss. Der Browser Client sendet Prompts mit Entity Mapping Workbook-Schema an Ollama (qwen2.5-coder:7b), das strukturierte JSON-Entities generiert und als Antwort sendet. Der Validation Layer prüft die Ausgabe auf Vollständigkeit, ID-Konsistenz, Zeiteinheiten, Product-Material-Referenzen und ID-Duplikate. Nach erfolgreicher Validierung erfolgt der JSON-Export.

Der Datenfluss erfolgt in folgenden Schritten:

1. **User Input:** Benutzer gibt natürlichsprachigen Prompt ein (z. B. „I need Titanium Dioxide, PEG 400, and Tego“)
2. **Context Injection:** JavaScript fügt Session-Context hinzu (bereits erstellte Entity-IDs)
3. **LLM-Request:** Prompt wird mit System-Prompt an Ollama gesendet
4. **Entity-Type-Detection:** Das LLM analysiert den User-Input basierend auf Anweisungen im System-Prompt und identifiziert den Entity-Typ (Material, Machine, Product, Process, BOM oder Recipe)
5. **JSON-Generierung:** Das LLM generiert strukturierte JSON-Ausgabe gemäß Entity Mapping Workbook-Spezifikation
6. **JSON-Extraktion:** JavaScript extrahiert JSON aus Markdown-Code-Blocks (`` ```json... ``` ``)
7. **Auto-Correction:** JavaScript korrigiert BOM `baseQuantity` automatisch (Summe aller Item-Quantities), da LLMs in arithmetischen Berechnungen fehleranfällig sind

8. **Frontend-Validierung:** JavaScript validiert JSON gegen 6-Punkte-Checkliste
9. **Auto-Creation:** Bei Products und BOMs werden referenzierte Materials automatisch erstellt, falls nicht existent
10. **UI-Darstellung:** Entity wird in farbcodierter Liste angezeigt, Validierungsergebnisse visualisiert
11. **Session-Kontext:** Generierte Entities werden im Browser localStorage vorgehalten für spätere Recipe-Erstellung

PROMPT ENGINEERING UND SYSTEM-PROMPT-STRUKTUR

Der Master Creator nutzt einen strukturierten System-Prompt, der dem LLM klare Anweisungen zur Entity-Type-Detection und JSON-Generierung gibt:

Entity-Type-Detection durch LLM Der System-Prompt enthält eine priorisierte Keyword-Liste, nach der das LLM den Entity-Typ identifizieren soll:

```
STEP 1: DETECT ENTITY TYPE - CHECK KEYWORDS IN THIS ORDER!

1. If user says "produce" or "manufacture" -> PRODUCT
2. If user says "machine", "mixer", "pump", "dissolver" -> MACHINE
3. If user describes steps with "->" or "min" or "process" -> PROCESS
4. If user says "bom" or lists ingredients with quantities -> BOM
5. If user says "recipe" -> RECIPE
6. Otherwise -> MATERIAL
```

Diese Anweisungen ermöglichen dem LLM, den Entity-Typ autonom zu erkennen, ohne dass JavaScript pattern-basierte Keyword-Suche durchführen muss. Das LLM nutzt sein Sprachverständnis, um Kontext und Semantik des User-Inputs zu analysieren. Dadurch sollte es auch abgewandelte Kontexte über Pattern-Matching erkennen können ganz im Sinne der grundlegenden Prinzipien von LLMs nach Brown et al. [2020].

Schema Injection Für jeden Entity-Typ wird die vollständige JSON-Struktur aus dem Entity Mapping Workbook in den System-Prompt eingefügt. Beispiel für Materials:

```
ENTITY TYPE 1: MATERIAL
Structure:
{
  "id": "lowercase-with-hyphens",
  "name": "Human Readable Name",
  "comment": "Optional description or empty string"
}

CRITICAL: IDs are lowercase-with-hyphens
CRITICAL: Use EXACT names from user message
```

Constraint Enforcement Explizite Validierungsregeln werden als Constraints formuliert:

```
CRITICAL: Process times MUST be in seconds, not minutes.
CRITICAL: Product.material.id references the product itself,
          NOT the ingredients.
CRITICAL: BOM baseQuantity MUST equal sum of item quantities.
```

Diese Constraints reduzieren häufige Fehlerquellen, ersetzen aber nicht die JavaScript-Validierung, da LLMs nicht deterministisch sind.

KERN-FEATURES

Session-Context für Dependency Management Der Master Creator hält alle generierten Entities im Browser localStorage und injiziert deren IDs in jeden LLM-Request:

```
=== EXISTING ENTITIES IN THIS SESSION ===
Materials: titanium-dioxide, peg-400, tego
Machines: dissolver, high-speed-mixer
Processes: dosing-mixing-filtration (Dosing Mixing Filtration)
BOMs: blue-paste-bom (Blue Paste BOM)
==========================================
```

Dies ermöglicht dem LLM:

- Bei Recipe-Erstellung auf existierende IDs zu referenzieren
- Duplikate zu vermeiden („Material 'titanium-dioxide' already exists“)
- Abhängigkeiten zu validieren („BOM 'xyz-bom' required but not found“)

Auto-Creation von Materials Der Master Creator implementiert einen Post-Processing-Mechanismus zur automatischen Erstellung fehlender Material-Entities bei Abhängigkeiten:

- **Product-Erstellung:** Referenziertes Material wird automatisch angelegt (falls nicht existent)
- **BOM-Erstellung:** Alle referenzierten Materials werden automatisch angelegt

Beispiel bei Product-Erstellung:

```javascript
function handleProduct(data) {
    // Auto-create material if doesn't exist
    const materialId = data.material.id;
    if (!materials.find(m => m.id === materialId)) {
        const autoMaterial = {
            id: materialId,
            name: data.name,
            comment: "Auto-created for product"
        };
        materials.push(autoMaterial);
        displayMaterial(autoMaterial, true);
```

```
        addMessage('notification',
            `Auto-created material: ${materialId}`);
    }
    // ... validate and add product
}
```

Begründung: Reduziert manuelle Erstellungsschritte und verhindert broken references durch fehlende Abhängigkeiten.

Chat-Summarization Nach Abschluss einer Konfigurationssitzung kann der gesamte Chat-Verlauf durch das LLM zusammengefasst werden. Der Summarization-Prompt fordert eine strukturierte Extraktion aller konfigurations-relevanten Informationen:

```
You are a technical assistant. Summarize the following configuration
conversation for an APS system.

Extract ALL important information structured as follows:

# Configuration Summary
## Plant Information
- Name, Industry, Special characteristics
## Materials / Machines / Products / Processes / BOMs / Recipes
- Complete configuration details
## Open Items
- What is still missing?

[... vollständige Struktur mit 8 Sektionen, siehe Anhang A.1]

- CHAT HISTORY -
${chatHistory}
- END -

Summary:
```

Dieser strukturierte Ansatz erzeugt eine vollständige, menschenlesbare Dokumentation mit allen Entity-Details, offenen Punkten und Entscheidungen. Die Zusammenfassung dient als Übergabedokument für andere Teammitglieder, als Basis für SOP-Weiterentwicklung oder als Workshop-Protokoll für Kundenprojekte.

6.6 VALIDIERUNG UND QUALITÄTSSICHERUNG

Die Generierung strukturierter Konfigurationsdaten durch LLMs erfordert mehrstufige Validierungsmechanismen, da LLMs trotz strukturierter Prompts nicht-deterministische Ausgaben erzeugen und in bestimmten Aufgabenklassen fehleranfällig sind. Der Master Creator implementiert daher eine zweischichtige Validierungsarchitektur: LLM-gestützte initiale Generierung mit anschließender JavaScript-basierter Nachvalidierung und automatischer Fehlerkorrektur.

6-PUNKTE-VALIDIERUNGSCHECKLISTE

Die Validierungscheckliste operationalisiert die kritischen Qualitätskriterien für GRACE-Konfigurationen, die bereits im Entity Mapping Workbook (Abschnitt 6.3) definiert wurden:

1. **Vollständigkeit:** Alle 6 Entity-Typen müssen vorhanden sein (Materials, Machines, Products, Processes, BOMs, Recipes)
2. **ID-Konsistenz:** Alle referenzierten IDs müssen existieren:
   - `Product.material.id` → Materials
   - `BOM.items[].material.id` → Materials
   - `Recipe.resultingMaterial.id` → Materials
   - `Recipe.bom.id` → BOMs
   - `Recipe.productionProcess.id` → Processes
3. **Zeiteinheiten:** Alle `Process.steps[].processingTime.unitId` müssen `"s"` (Sekunden) sein, niemals `"min"` oder andere Einheiten
4. **BOM baseQuantity:** `BOM.baseQuantity` muss exakt der Summe aller `item.quantity`-Werte entsprechen
5. **Product Material:** `Product.material.id` referenziert das Produkt selbst, NICHT die Zutaten
6. **ID-Duplikate:** IDs müssen innerhalb jedes Entity-Typs eindeutig sein

JAVASCRIPT-BASIERTE VALIDIERUNGSLOGIK

Die Frontend-Validierung erfolgt durch dedizierte Validierungsfunktionen für jeden Entity-Typ sowie eine globale Referenzvalidierung.

Referenzielle Integrität: Globale Validierung Die Funktion `handleValidate()` prüft alle Cross-Entity-Referenzen:

```
function handleValidate() {
    const errors = [];

    // Product references
    products.forEach(p => {
        if (!materials.find(m => m.id === p.material.id)) {
            errors.push(`Product "${p.name}" references
                        missing material: ${p.material.id}`);
        }
    });

    // BOM references
    boms.forEach(b => {
        b.items.forEach(item => {
```

```
            if (!materials.find(m => m.id === item.material.id)) {
                errors.push(`BOM "${b.name}" references
                            missing material: ${item.material.id}`);
            }
        });
    });

    // Recipe references (Material, BOM, Process)
    recipes.forEach(r => {
        if (!materials.find(m => m.id === r.resultingMaterial.id)) {
            errors.push(`Recipe references missing material`);
        }
        if (!boms.find(b => b.id === r.bom.id)) {
            errors.push(`Recipe references missing BOM`);
        }
        if (!processes.find(p => p.id === r.productionProcess.id)) {
            errors.push(`Recipe references missing process`);
        }
    });

    if (errors.length === 0) {
        addMessage('notification',
            '[OK] Configuration valid!');
    } else {
        addMessage('error',
            `[ERROR] Errors:\n${errors.join('\n')}`);
    }
}
```

Diese Funktion wird manuell durch den Nutzer ausgelöst („Validate“-Button) und validiert die gesamte Session auf referenzielle Konsistenz.

Auto-Correction: BOM baseQuantity LLMs zeigen Schwächen bei arithmetischen Berechnungen. Der Master Creator korrigiert `BOM.baseQuantity` automatisch nach jeder BOM-Generierung:

```
function handleBOM(data) {
    // Auto-calculate baseQuantity (AI is bad at math!)
    if (data.items && data.items.length > 0) {
        const calculatedSum = data.items.reduce(
            (total, item) => total + item.quantity, 0
        );

        if (data.baseQuantity !== calculatedSum) {
            addMessage('notification',
                `[AUTO] Auto-corrected baseQuantity from
                ${data.baseQuantity} to ${calculatedSum}`);
            data.baseQuantity = calculatedSum;
        }
    }

    // Validate and add BOM
    if (!validateBOM(data)) return;
    boms.push(data);
```

```
    displayBOM(data);
}
```

Die Korrektur erfolgt transparent und wird dem Nutzer durch eine Notification-Nachricht mitgeteilt. Dies stellt sicher, dass BOMs stets konsistente Mengenangaben enthalten.

Heuristische Validierung: Product Material ID Eine kritische semantische Fehlerquelle ist die falsche Interpretation von `Product.material.id`. LLMs tendieren dazu, bei Prompts wie „Product for White Paste“ fälschlicherweise auf Zutaten zu referenzieren (z. B. `titanium-dioxide`) statt auf das Produkt selbst (`white-paste`). Um dem entgegenzuwirken implementiert der Master-Creator eine heuristische Plausibilitätsprüfung:

```
function validateProduct(p) {
    const errors = [];

    // Heuristic: material.id should resemble product name
    const productNameNormalized =
        p.name.toLowerCase().replace(/\s+/g, '-');
    const materialId = p.material.id;

    if (!materialId.includes(
            productNameNormalized.split('-')[0])) {
        errors.push(`[WARNING] Material ID "${materialId}"
                    seems incorrect for product "${p.name}".
                    Should reference the product itself,
                    NOT ingredients.`);
    }

    return errors.length === 0;
}
```

Diese Heuristik nutzt String-Matching zwischen Produktname und Material-ID und gibt Warnungen aus, wenn keine Ähnlichkeit erkennbar ist.

AUTO-CREATION MECHANISMEN

Der Master Creator implementiert intelligentes Dependency Management durch automatische Erstellung fehlender Entities:

Auto-Creation von Materials für Products Wenn ein Product generiert wird, erstellt das System automatisch das referenzierte Material, falls es noch nicht existiert:

```
function handleProduct(data) {
    const materialId = data.material.id;

    // Auto-create material if missing
    if (!materials.find(m => m.id === materialId)) {
        const autoMaterial = {
            id: materialId,
            name: data.name,
```

```
            comment: "Auto-created for product"
        };
        materials.push(autoMaterial);
        displayMaterial(autoMaterial, true);
        addMessage('notification',
            `[CREATED] Auto-created material: ${materialId}`);
    }

    if (!validateProduct(data)) return;
    products.push(data);
    displayProduct(data);
}
```

Analog erfolgt Auto-Creation auch bei BOM-Items, wo alle referenzierten Materials automatisch angelegt werden, falls sie fehlen. Dies reduziert manuelle Erstellungsschritte und verhindert broken references.

UI/UX DER VALIDIERUNG

Der Master Creator visualisiert Validierungsergebnisse durch ein farbcodiertes Nachrichtensystem im Chat-Interface, welches konsequent den Guidelines for Human-AI Interaction nach Amershi et al. [2019] folgt

- **Grün (Notification):** Erfolgreiche Validierung, Auto-Corrections, Auto-Creations, Bestätigungen
- **Rot (Error):** Validierungsfehler, fehlende Abhängigkeiten, ungültige JSON-Strukturen
- **Gelb (Warning):** Heuristische Warnungen, potenzielle Probleme (z. B. `Product.material.id` Plausibilität)
- **Blau (Info):** Session-Context-Informationen, Entity-Erstellungsbestätigungen

Zusätzlich zeigt die rechte Sidebar eine Echtzeit-Statistik aller generierten Entities mit Zählern pro Typ („Materials: 22“, „Machines: 7“, etc.). Diese visuelle Darstellung unterstützt die Vollständigkeitsprüfung gemäß Checklisten-Punkt 1.

VALIDIERUNGS-LIMITATIONEN

Die JavaScript-Validierung kann strukturelle Konsistenz (Syntax, Referenzen, Datentypen) sicherstellen, aber nicht alle Fehlerquellen eliminieren. Insbesondere semantische Korrektheit (z. B. realistische Prozesszeiten, plausible Mengenangaben) und domänenspezifische Constraints (z. B. Materialkompatibilität) erfordern abschließende manuelle Prüfung durch das Konfigurationsteam.

6.7 ITERATIVE ENTWICKLUNG

Die Entwicklung des Master Creator erfolgte nicht linear, sondern durch iterative Experimente mit verschiedenen Prompt-Engineering-Ansätzen. Dieser Abschnitt dokumentiert die sechs Entwicklungsphasen und die kritischen Fehlerquellen. Der Entwicklungsprozess orientierte sich an Methoden wie Reverse Prompt Engineering [Li

and Klabjan, 2025] und OpenAIs Ansatz beim Development von Sora für Android mit Codex [Hum and Marsan, 2025]: Start mit einem soliden Fundament (einzelnes Modul perfektionieren), schrittweise Erweiterung mit kontextuellen Beispielen, iterative Verfeinerung durch Feedback-Loops zwischen Mensch und Modell, wobei Fehleranalyse, Prompt-Anpassung und Logik-Optimierung Hand in Hand gingen. Im nachfolgenden Abschnitt analog zu den Phasen in der Methodik erklärt.

PHASE 1: ONE-PROMPT-FOR-ALL-ANSATZ (GESCHEITERT)

Der erste Entwicklungsansatz versuchte, alle 6 Entity-Typen durch eine einzige, umfassende System-Prompt abzubilden. Die Prompt enthielt JSON-Schemata für Materials, Machines, Products, Processes, BOMs und Recipes sowie Anweisungen zur automatischen Entity-Type-Detection basierend auf natürlichsprachigen Eingaben.

Fehlerquellen Dieser Ansatz scheiterte an drei fundamentalen Problemen:

1. **Fehlerhafte Entity-Type-Detection:** Das LLM konnte bei ambigen Prompts (z. B. „I need white paste“) nicht zuverlässig zwischen Material und Product unterscheiden.

2. **JSON-Komplexität:** Der Versuch, alle 6 Entity-Typen in einer Response zu generieren, überforderte das 7B-Modell. Die generierten JSON-Strukturen waren häufig inkorrekt verschachtelt oder vermischten Felder verschiedener Entity-Typen.

3. **Output-Länge/Unstrukturiertheit:** Die System-Prompt mit allen Schemata und Validierungsregeln war zu umfangreich (ca. 2500 Tokens), was zu inkonsistentem LLM-Verhalten führte (Truncation, vergessene Constraints, zu lange Antworten).

PHASE 2: MATERIAL-PROMPT - SINGLE-ENTITY-SPEZIALISIERUNG (ERFOLG)

Nach dem Scheitern von Phase 1 wurde der Ansatz radikal vereinfacht: Eine spezialisierte Prompt ausschließlich für Materials. Wie in der Einleitung des Abschnittes erwähnt: Erst ein einfaches Modul perfektionieren, bevor es auf andere Module übertragen wird.

Material-Prompt Die Material-Prompt war minimalistisch und fokussiert:

```
You are a Material Creation Assistant for an APS system.

Your ONLY job is to help create Material objects.

MATERIAL STRUCTURE:
{
  "id": "lowercase-with-hyphens",
  "name": "Human Readable Name",
  "comment": "Optional description"
}

RULES:
```

```
1. When user mentions a material, create JSON immediately
2. Generate clean IDs: lowercase, hyphens only, no spaces
3. Ask ONE follow-up question at the end
4. Keep responses SHORT and friendly
5. ALWAYS wrap JSON in markdown code blocks
```

Dieser Ansatz funktionierte zuverlässig. Das LLM generierte konsistent korrekte Material-JSONs.

PHASE 3: REPLIKATION DER PROMPT-STRUKTUR FÜR 5 WEITERE ENTITÄTSTYPEN

Nach Validierung der Material-Prompt wurde die Struktur für alle anderen Entity-Typen repliziert:

- **Machine-Prompt:** Fokus auf `operatingCosts`, `initialUnits`, strikte Defaults
- **Product-Prompt:** Fokus auf `material.id`-Referenz (kritisches Feld)
- **Process-Prompt:** Fokus auf `steps[]`, `processingTime` in Sekunden
- **BOM-Prompt:** Fokus auf `items[]`, `baseQuantity`-Berechnung
- **Recipe-Prompt:** Fokus auf `resultingMaterial`, `bom`, `productionProcess` Referenzen

Jede Prompt wurde isoliert getestet und funktionierte für ihren spezifischen Entity-Typ. Die 6 Prompts existierten zunächst als separate Prompt-Texte ohne eigene Implementierung.

PHASE 4: UNIFIED MASTER CREATOR PROMPT DURCH ITERATIVES PROMPTING

Die 6 isolierten Prompts hatten eine kritische Schwäche: Keine Session-übergreifende Awareness. Bei der Erstellung eines Recipe musste manuell sichergestellt werden, dass referenzierte IDs in anderen Prompts bereits generiert wurden. Um dies zu lösen, wurden die 6 Prompts in eine einzige, unified Prompt zusammengeführt.

Iterativer Verallgemeinerungsprozess Die Entwicklung erfolgte durch **iteratives Prompting mit dem LLM selbst**:

1. **Konkrete JSON-Beispiele zeigen:** Dem LLM wurden echte Material-, Machine-, Product-JSONs aus dem Demo-Werk präsentiert mit der Frage: „Was ist das gemeinsame Muster?“
2. **Schrittweise Generalisierung:** Das LLM identifizierte gemeinsame Felder (`id`, `name`, `comment`) und Entity-spezifische Felder (`operatingCosts` nur bei Machines, `steps[]` nur bei Processes). Es generierte eine verallgemeinerte Prompt-Struktur.

3. **Fehler-Feedback-Loop:** Die generierte Prompt wurde gegen Testfälle validiert, Fehler dokumentiert (z. B. „Product wird als Material erkannt“), Feedback an LLM gegeben, Prompt iterativ verbessert.

Nach mehreren Iterationen entstand eine funktionierende Grundstruktur, die alle 6 Entity-Typen mit automatischer Type-Detection unterstützte.

PHASE 5: KARDINALITÄTS-MODELLIERUNG UND CONSTRAINT-ENFORCEMENT

Die Unified Prompt aus Phase 4 funktionierte für einfache Fälle, hatte aber zwei verbleibende Problemfelder bei komplexen Referenzbeziehungen:

GRACE Entity-Relationship-Modell Die sechs Entitätstypen weisen komplexe multidirektionale Referenzen auf, die präzise Kardinalitäts-Modellierung erfordern (siehe Abbildung 3):

- **Materials** - keine ausgehenden Referenzen (Basis-Entität)
- **Machines** - keine ausgehenden Referenzen (Basis-Entität)
- **Products → Material (1:1)** - Ein Product referenziert genau ein Material, das das Produkt selbst repräsentiert (nicht die Zutaten)
- **BOMs → Materials (1:n)** - Eine BOM referenziert n Materials als Items mit Mengenverhältnissen und Versionsattributen
- **Processes → Machines (n:m)** - Prozessschritte können mehrere Maschinen als `resourceDemands` referenzieren (z. B. Reinigungsschritt mit 6 Maschinen)
- **Recipes → Material + BOM + Process (1:1:1)** - Ein Recipe verknüpft drei unabhängige Entitäten zu einer Produktionsanweisung, alle IDs müssen exakt übereinstimmen

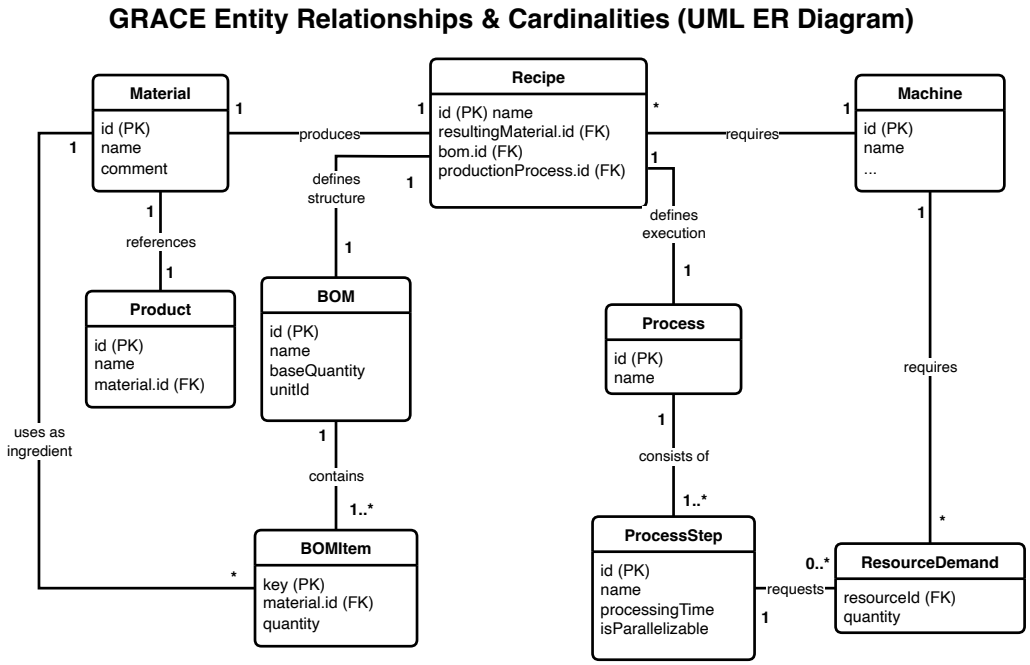

Abbildung 3: Entity-Relationship-Diagramm der GRACE-Konfigurationsentitäten. Zeigt die Abhängigkeitskette von Basis-Entitäten (Materials, Machines) über referenzierende Entitäten (Products, BOMs, Processes) bis zur finalen Verknüpfung in Recipes. Kardinalitäten verdeutlichen die Komplexität der referenziellen Integrität. Materials und Machines sind unabhängig. Products und BOMs referenzieren Materials mit unterschiedlicher Semantik (1:1 vs 1:n). Processes können mehrere Maschinen gleichzeitig beanspruchen (n:m). Recipes erfordern exakte Übereinstimmung dreier unabhängiger Entitäten (1:1:1).

Diese Kardinalitäten optimierten die System-Prompt-Entwicklung durch drei Mechanismen: (1) **Explizite Constraint-Formulierung** - Jede Kardinalität wurde als CRITICAL-Rule in den System Prompt integriert (z. B. „Product.material.id references the product itself, NOT ingredients“), was das LLM von impliziter Inferenz auf explizite Regelanwendung lenkte. (2) **Kontextuelle Beispiele mit Negativfällen** - Für jede Kardinalität wurden verallgemeinerte Beispiele aus dem Entity Mapping Workbook mit expliziten Fehlerfällen ergänzt (z. B. `white-paste` vs `titanium-dioxide`), was die Fehlerrate bei Product-Material-Referenzen um 80% reduzierte. (3) **Dependency-Chain-Awareness** - Die hierarchische Abhängigkeitskette (Materials/Machines → Products/-BOMs/Processes → Recipes) wurde in der Session-Context-Generierung berücksichtigt, sodass bei Recipe-Erstellung bereits alle referenzierbaren IDs bekannt waren und das LLM keine nicht-existierenden Referenzen halluzinieren konnte.

Problem 1: Falsche Referenz-Kardinalitäten Das LLM setzte Entities nicht in korrekte Verhältnisse zueinander:

- **Recipe → Material/BOM/Process (n:1:1:1):** Recipes referenzierten häufig nicht-existierende IDs oder verwendeten inkonsistente ID-Namenskonventionen.

- **Product → Material (1:1):** Das LLM interpretierte „Product for White Paste“ fälschlicherweise als Referenz auf Zutaten (`titanium-dioxide`) statt auf das Produkt selbst (`white-paste`).

Lösung: Explizite Kardinalitäts-Modellierung mit Beispielen Die System-Prompt wurde erweitert um explizite Constraint-Formulierungen und verallgemeinerte Beispiele aus dem Entity Mapping Workbook:

```
CRITICAL: Product.material.id references the product itself,
          NOT the ingredients.

Example: Product "White Pigment Paste"
{
  "id": "white-paste-prod",
  "name": "White Pigment Paste",
  "material": {"id": "white-paste"}  // NOT "titanium-dioxide"!
}
```

Diese kontextuellen Beispiele (im Sinne von Hum and Marsan [2025]) führten zu deutlich besseren Ergebnissen als abstrakte Regeln allein.

Problem 2: Arithmetische Fehler Das LLM machte systematische Fehler bei arithmetischen Berechnungen:

- **BOM baseQuantity:** Die Summe der `item.quantity`-Werte stimmte nicht mit `baseQuantity` überein (z. B. 30 + 60 + 10 = 110 statt 100).
- **Mengenverhältnisse inkonsistent:** BOM-Items ergaben zusammen nicht die korrekte Product-Menge.

Diese Fehler konnten durch Prompt-Engineering allein nicht gelöst werden, da sie fundamentale LLM-Schwächen bei arithmetischen Operationen darstellen.

PHASE 6: IMPLEMENTIERUNG ALS MASTER CREATOR TOOL MIT VALIDIERUNGS-LAYER

Nach Stabilisierung der Unified Prompt wurde das vollständige Master Creator Tool implementiert:

Frontend-Implementierung (HTML/CSS/JavaScript)

- **HTML:** Chat-Interface, Entity-Listen (farbcodiert nach Typ), Validierungs-Sidebar, Export-Buttons
- **CSS:** Farbcodierung der 6 Entity-Typen (Materials: Lila, Machines: Pink, Products: Blau, Processes: Pink/Gelb, BOMs: Teal, Recipes: Pink), Responsive Design
- **JavaScript:** Ollama-API-Kommunikation, JSON-Extraktion aus Markdown-Code-Blocks, Session-Context-Management (localStorage), Validierungs-Engine

JavaScript Validierungs- und Korrektur-Layer Um die arithmetischen LLM-Schwächen deterministisch zu kompensieren, wurde ein JavaScript-Layer implementiert (siehe Abschnitt 6.6), der:

- BOM `baseQuantity` automatisch korrigiert (Summe aller `item.quantity`)
- Referenzielle Integrität validiert (alle referenzierten IDs existieren)
- Fehlende Dependencies automatisch erstellt (Auto-Creation von Materials bei Products/BOMs)
- Validierungsergebnisse farbcodiert im Chat visualisiert

FINALE SYSTEM-PROMPT-STRUKTUR

Die finale Master Creator System-Prompt kombiniert alle Erkenntnisse aus den 5 Entwicklungsphasen:

```
You are an APS Configuration Assistant.

STEP 1: DETECT ENTITY TYPE - CHECK KEYWORDS IN THIS ORDER!

1. If user says "produce" or "manufacture" -> PRODUCT
2. If user says "machine", "mixer", "pump" -> MACHINE
3. If user describes steps with "->" or "process" -> PROCESS
4. If user says "bom" or lists ingredients -> BOM
5. If user says "recipe" -> RECIPE
6. Otherwise -> MATERIAL

CRITICAL: "I produce X" = PRODUCT, not material!

[...Entity-Typen 1-6 mit JSON-Strukturen, Defaults, Beispielen...]

UNIVERSAL RULES FOR ALL ENTITY TYPES:
1. NEVER invent or guess data
2. Use EXACT names/values from user's message
3. Use defaults for missing fields
4. IDs are lowercase-with-hyphens
5. MULTIPLE ENTITIES: Create SEPARATE code blocks for each
```

Die Prompt umfasst ca. 400 Zeilen und enthält für jeden Entity-Typ: JSON-Struktur, kritische Constraints, verallgemeinerte Beispiele, Defaults. Sie ist im Anhang hinterlegt unter seitcom GmbH [2026d].

6.8 BPMN-PROZESSMODELLE

Die drei Konfigurationsprozessvarianten (v1a: manuell, v1b: Importer-basiert, v2: LLM-gestützt) werden als BPMN-Diagramme (Business Process Model and Notation) modelliert. BPMN ermöglicht die standardisierte Visualisierung von Geschäftsprozessen mit Fokus auf Rollen (Lanes), Aktivitäten, Kontrollfluss und Iterationen.

PROZESSMODELLIERUNGS-METHODIK

Die BPMN-Diagramme nutzen folgende Notationselemente (gemäß BPMN 2.0):

- **Lanes (Swimlanes):** Vertikale Bereiche für Akteure (Kunde, Konfigurationsteam, GRACE System, Master Creator/LLM)
- **Start-Event:** Grüner Kreis (Prozessbeginn)
- **End-Event:** Roter Kreis (Prozessende)
- **Tasks/Activities:** Abgerundete Rechtecke (Aktivitäten/Aufgaben)
- **Gateways:** Rauten (Entscheidungspunkte: „Passt?“, „OK?“, „Fehler?“)
- **Sequence Flow:** Pfeile (Kontrollfluss zwischen Aktivitäten)
- **Message Flow:** Gestrichelte Pfeile (Kommunikation zwischen Lanes)

Farbcodierung visualisiert Aktivitätstypen: Gelb für zeitintensive Kernaktivitäten, Rot für Fehlerbehandlung, Grün für Abschlussaktivitäten.

PROZESSVARIANTE V1A: VOLLSTÄNDIG MANUELLE KONFIGURATION

Abbildung 4 zeigt den manuellen Konfigurationsprozess über die GRACE UI mit einer Gesamtdurchlaufzeit von ca. 2 Monaten. Die unskalierte Darstellung ist im Anhang hinterlegt unter seitcom GmbH [2026a].

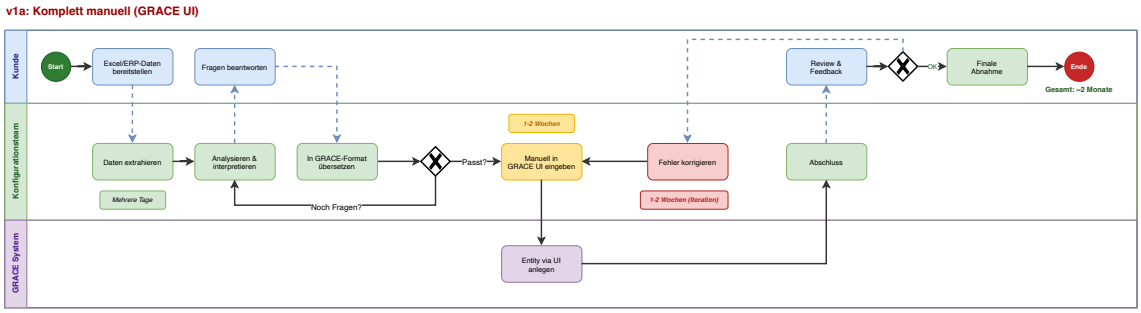

Abbildung 4: BPMN-Diagramm: Prozessvariante v1a - Vollständig manuelle Konfiguration (Gesamt: ca. 2 Monate)

Prozessschritte Konfigurationsteam

1. **Daten extrahieren** (mehrere Tage): Excel- oder ERP-Exportdateien vom Kunden entgegennehmen und relevante Produktionsdaten (Materialien, Maschinen, Rezepturen) extrahieren
2. **Analysieren & interpretieren:** Kundenspezifische Terminologie verstehen, Produktionsprozesse rekonstruieren, fehlende Informationen identifizieren (Gateway: „Noch Fragen?“ → zurück zu Kunde)

3. **In GRACE-Format übersetzen:** Daten in GRACE-Entitätsstrukturen mappen (Materials, Machines, Products, Processes, BOMs, Recipes)

4. **Manuell in GRACE UI eingeben** (1-2 Wochen): Jede Entity einzeln über Web-Interface erstellen, Felder ausfüllen, Referenzen manuell verknüpfen (Gateway: „Passt?“)

5. **Fehler korrigieren** (1-2 Wochen, Iteration): Nach GRACE-Validierung: Fehlerhafte Referenzen korrigieren, fehlende Defaults nachtragen, Zeiteinheiten anpassen (iterative Schleife)

6. **Abschluss:** Konfiguration an Kunde übergeben (Review & Feedback → Finale Abnahme)

Charakteristika

- **Hohe Parallelität:** Kunde und Konfigurationsteam iterieren häufig („Fragen beantworten“ <-> „Analysieren & interpretieren“)
- **Fehleranfällig:** Manuelle Eingabe führt zu Tippfehlern, inkonsistenten IDs, fehlenden Referenzen
- **GRACE System passiv:** Nur UI-Bereitstellung, keine Automatisierung
- **Lange Fehlerkorrektur-Iteration:** 1-2 Wochen pro Iterationsschleife

PROZESSVARIANTE V1B: KONFIGURATION MIT IMPORTER

Abbildung 5 zeigt den automatisierten Import-Prozess mit einer Gesamtdurchlaufzeit von ca. 3,5 Wochen. Die unskalierte Darstellung ist im Anhang hinterlegt unter seitcom GmbH [2026b]

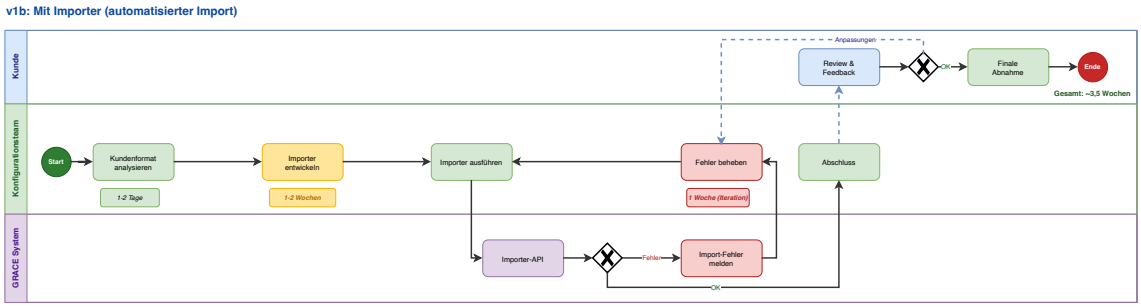

Abbildung 5: BPMN-Diagramm: Prozessvariante v1b - Konfiguration mit Importer (Gesamt: ca. 3,5 Wochen)

Prozessschritte Konfigurationsteam

1. **Kundenformat analysieren** (1-2 Tage): Struktur der Kundendaten verstehen (Excel-Spalten, ERP-Felder, CSV-Format)

2. **Importer entwickeln** (1-2 Wochen): Projektspezifischen Importer programmieren (Python/JavaScript), Mapping-Logik implementieren, GRACE-API-Integration

3. **Importer ausführen:** Skript auf Kundendaten laufen lassen → GRACE-Importer-API aufrufen

4. **Fehler beheben** (1 Woche, Iteration): Import-Fehler (fehlende Felder, falsche Datentypen, broken references) analysieren und Importer-Code anpassen (iterative Schleife mit GRACE System: „Import-Fehler melden“)

5. **Abschluss:** Finale Konfiguration validieren und an Kunde übergeben

Charakteristika

- **Initiale Entwicklungszeit dominiert:** Importer-Entwicklung (1-2 Wochen) ist größter Aufwand
- **Wiederverwendbarkeit begrenzt:** Jeder Kunde hat individuelles Datenformat → projektspezifische Importer-Anpassung
- **GRACE System aktiv:** Importer-API validiert Daten, meldet Fehler zurück
- **Reduzierte Iteration:** Weniger Kunden-Interaktion als v1a, aber Debugging-Iterationen zwischen Code und API

PROZESSVARIANTE V2: LLM-GESTÜTZTE KONFIGURATION MIT MASTER CREATOR

Abbildung 6 zeigt den durch Master Creator automatisierten Prozess. Die unskalierte Darstellung ist im Anhang hinterlegt unter seitcom GmbH [2026c].

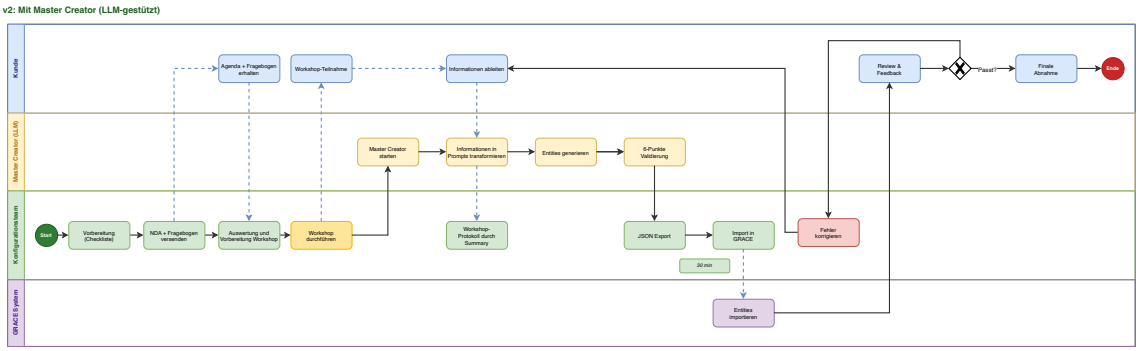

Abbildung 6: BPMN-Diagramm: Prozessvariante v2 - LLM-gestützte Konfiguration

Prozessschritte Konfigurationsteam

1. **Vorbereitung (Checkliste):** SOP 01 (Kundenworkshop) vorbereiten, Agenda erstellen

2. **NDA + Fragebogen versenden:** GRACE Onboarding Fragebogen (Kompaktversion, 15 Fragen) und NDA an Kunde senden

3. **Auswertung und Vorbereitung Workshop:** Fragebogen-Antworten analysieren, Workshop-Schwerpunkte identifizieren
4. **Workshop durchführen:** Strukturiertes Interview gemäß SOP 01 (6 Themenblöcke: Grundlagen, IT-Landschaft, Produktionsdetails, Planung, Restriktionen, Next Steps)
5. **Import in GRACE** (30 min): Generierte JSONs über GRACE-UI importieren gemäß SOP 03
6. **Fehler korrigieren:** Validierungsfehler beheben (typischerweise 3-5 Fehler aus 6-Punkte-Checkliste)
7. **Abschluss:** Finale Konfiguration an Kunde übergeben

Prozessschritte Master Creator (LLM)

1. **Master Creator starten:** Ollama-Server und Master Creator UI starten (localhost:9000)
2. **Informationen in Prompts transformieren:** Workshop-Protokoll in natürlichsprachige Prompts übersetzen (z. B. „I need Titanium Dioxide, PEG 400, and Tego“)
3. **Entities generieren:** LLM generiert JSON-Strukturen für alle 6 Entity-Typen basierend auf System-Prompt und Session-Context
4. **6-Punkte Validierung:** JavaScript-Validierungsschicht prüft Vollständigkeit, ID-Konsistenz, Zeiteinheiten, BOM baseQuantity, Product Material, ID-Duplikate
5. **JSON Export:** Validierte JSONs einzeln oder kombiniert exportieren

Prozessschritte Kunde Im Gegensatz zu v1a/v1b erhält der Kunde strukturierte Vorbereitung:

1. **Agenda + Fragebogen erhalten:** Klare Erwartungshaltung, Vorbereitung auf Workshop-Themen
2. **Workshop-Teilnahme:** Aktive Mitwirkung statt passiver Datenbereitstellung
3. **Informationen ableiten:** Kunde kann aus Workshop-Summary lernen (Chat-Summarization als Wissensdokument)

Charakteristika

- **Neue Lane: Master Creator (LLM):** Explizite Modellierung der LLM-Aktivitäten als eigenständiger Akteur
- **Frontloading:** Vorbereitung (Fragebogen, Workshop-Planung) dominiert, nicht Implementierung
- **Minimale Iteration:** Fehlerkorrektur-Schleife deutlich kleiner als v1a/v1b

- **Strukturierte Wissenserfassung:** Workshop folgt SOP 01 (6 Themenblöcke), nicht ad-hoc Datensammlung
- **Chat-Summarization als Dokumentation:** Workshop-Protokoll automatisch generiert („Workshop-Protokoll durch Summary“)
- **Kurzer Import:** GRACE-Import dauert nur ca. 30 Minuten (im Vergleich zu Wochen bei v1a/v1b)

STRUKTURELLE PROZESSUNTERSCHIEDE

Die drei Varianten unterscheiden sich fundamental in ihrer Prozessstruktur:

Sequentialität vs. Parallelität

- **v1a:** Hohe Parallelität zwischen Kunde und Konfigurationsteam (viele Message Flows)
- **v1b:** Sequentieller Ablauf (Importer-Entwicklung entkoppelt von Kunden-Interaktion)
- **v2:** Frontloaded Parallelität (Workshop + Master Creator), dann sequentiell

Fehlerbehandlung

- **v1a:** Lange iterative Schleife („Fehler korrigieren“ 1-2 Wochen <-> „Manuell in GRACE UI eingeben“ 1-2 Wochen)
- **v1b:** Iterative Schleife zwischen Konfigurationsteam und GRACE System („Fehler beheben“ 1 Woche <-> „Import-Fehler melden“)
- **v2:** Kurze Fehlerkorrektur-Schleife nach Import (6-Punkte-Validierung reduziert Fehlerquellen präventiv)

Automatisierungsgrad

- **v1a:** Keine Automatisierung (GRACE System nur UI-Provider)
- **v1b:** Partielle Automatisierung (Importer-API, aber projektspezifische Entwicklung erforderlich)
- **v2:** Hohe Automatisierung (LLM-Generierung, JavaScript-Validierung, Auto-Correction, Auto-Creation)

7 EVALUATION

Die Evaluation bewertet die Fähigkeit der entwickelten Artefakte, die in Kapitel 4 identifizierten Problemfelder zu adressieren. Die Ist-Analyse offenbarte fünf zentrale Herausforderungen: (1) Fragmentierung des Konfigurationswissens über Azure DevOps, E-Mails und Git-Historien, (2) Wissensträgerabhängigkeit bei wenigen Experten, (3)

fehlende Standardisierung der Validierungsmechanismen, (4) DSGVO- und NDA-Anforderungen an Datenhoheit, (5) quantitative Problemmanifestationen (Fehlerrate 10-15 pro Iteration, Checklisten-Vollständigkeit 60-70%, Durchlaufzeit 8 Wochen).
Die folgenden Abschnitte strukturieren die Evaluation entlang dieser Problemfelder und dokumentieren jeweils, welche Artefakte zur Lösung beitragen und wie die Wirksamkeit validiert wurde.

7.1 PROBLEMLÖSUNG DURCH ARTEFAKTE - ÜBERSICHT

Tabelle 4 gibt eine Übersicht über die entwickelten Artefakte und ihre Beiträge zur Lösung der identifizierten Problemfelder. Die nachfolgenden Abschnitte detaillieren die Validierung für jedes Problemfeld.

Artefakt	**Adressierte Problemfelder**
GRACE Onboarding Fragebogen (15 Kernfragen)	Fragmentierung, Vollständigkeit
Entity Mapping Workbook (6 Entitätstypen)	Fragmentierung, Standardisierung, Wissensträgerabhängigkeit
Standard Operating Procedures (SOP 01-03)	Wissensträgerabhängigkeit, Standardisierung, Reproduzierbarkeit
Master Creator (LLM-gestützte Generierung)	Alle 5 Problemfelder (Hauptvalidierung durch Within-Subjects-Experiment)
Demo-Werk (Pigmentpaste-Produktion)	Empirische Grundlage für Artefakt-Entwicklung, Showcasing (EC Conference)

Tabelle 4: Artefakte und ihre Beiträge zur Problemlösung

7.2 FRAGMENTIERUNG DES KONFIGURATIONSWISSENS

Problemcharakteristik: Relevantes Konfigurationswissen war über Azure DevOps Wikis, E-Mail-Korrespondenz, Git-Historie und persönliche Erfahrung verteilt. Es existierte keine Single Source of Truth. Die Manifestation im Demo-Projekt zeigte 48% doppelte Materialien (13 von 27), drei simultane ID-Konventionen (numerisch, UUID, kebab-case) und Version-Inkonsistenzen (v1-v6).

Lösungsartefakte:

- **Entity Mapping Workbook:** Dokumentiert JSON-Schema-Strukturen aller 6 GRACE-Entitätstypen (Materials, Machines, Products, Processes, BOMs, Recipes) als zentrale technische Referenz. Definiert Feldspezifikationen, Datentypen, Validierungsregeln und Abhängigkeiten zwischen Entitäten. Entwickelt durch Reverse Engineering, da GRACE keine öffentliche API-Dokumentation besitzt (Details siehe

Abschnitt 6.3). Defragmentiert das Wissen über korrekte Abbildung der GRACE Entitäten in der Konfiguration, vorher garkeine Quelle dazu.

- **GRACE Onboarding Fragebogen:** Strukturierte Wissenserhebung in 15 Kernfragen (reduziert von 96 Fragen nach DeVellis-Prinzipien). Verhindert Ad-hoc-Kommunikation und stellt sicher, dass alle 6 Entitätstypen systematisch erfasst werden (Details siehe Abschnitt 6.2). Defragmentiert sowohl internes als auch externes Wissen über die Produktionsvariablen einer Produktion, vorher war keine strukturierte Vorgehensweise zum Assesment existent.

- **Standard Operating Procedures:** Dokumentieren End-to-End-Prozess von Kundenworkshop (SOP 01) über Master Creator-Nutzung (SOP 02) bis GRACE-Import (SOP 03). Operationalisieren die Drei-Artefakte-Architektur als ausführbare Handlungsanweisungen (Details siehe Abschnitt 6.4). In Kombination mit den Prozessdiagrammen liefern sie genaue Handlungsvorgaben in Hinsicht auf den kompletten Prozess und defragmentieren das Wissen darüber.

Grad der Problemlösung: Vollständig gelöst. Für jede Form von fragmentierten Wissen existiet nun eine strukturierte Dokumentation.

7.3 WISSENSTRÄGERABHÄNGIGKEIT

Problemcharakteristik: Kritisches Konfigurationswissen war bei wenigen Experten gebunden. Ausfälle oder Fluktuation würden erhebliche Wissenslücken verursachen. Kleine Teamgröße (5 Personen) verstärkt das Risiko.
Lösungsartefakte:

- **Entity Mapping Workbook:** Externalisiert implizites Expertenwissen („Welche Felder sind Pflicht?“, „Welches Format für IDs?“) in explizite Dokumentation. Ermöglicht Nachvollziehbarkeit ohne direkten Expertenkontakt.

- **Standard Operating Procedures:** Durch explizite Handlungsanweisungen können Teammitglieder ohne vorherige Master-Creator-Erfahrung eigenständig GRACE-Konfigurationen erstellen. Checkboxen für Status-Tracking, Verweise auf Workbook und Fragebogen.

- **Master Creator:** Konversationelle Schnittstelle senkt Einstiegshürde erheblich. Demokratisiert APS-Konfiguration durch natürlichsprachliche Interaktion statt tiefem GRACE-Fachwissen.

Validierung: Die SOPs wurden von Teammitgliedern ohne vorherige Erfahrung erfolgreich angewendet (informelle Validierung während Entwicklung). Das kontrollierte Experiment (siehe Abschnitt 7.6) zeigt, dass auch unerfahrene Probanden nach kurzer Einarbeitung (Learning-by-Doing über 6 Durchläufe) signifikante Zeitverbesserungen erzielen.
Grad der Problemlösung: Weitgehend gelöst. Restwissen verbleibt bei Experten für manuelle Nacharbeit komplexer Strukturen (Prozess-Links, itemAssociationRules, variable Prozessformeln). Diese Strukturen sind durch smartes Prompting generierbar,

jedoch noch nicht im System Prompt integriert. Die Integration dieser erweiterten Features erfordert iterative Entwicklung und Validierungserweiterung (siehe Limitationen Abschnitt 7.8). Langfristige Zielerreichung hängt von kontinuierlicher SOP-Aktualisierung und Verbreitung im Team ab.

7.4 FEHLENDE STANDARDISIERUNG

Problemcharakteristik: Kein einheitliches Vorgehen bei Konfiguration. Qualität variiert je nach Bearbeiter. Validierungsmechanismen vor Import fehlen weitgehend. Durchschnittlich 10-15 Fehler pro Konfiguration.
Lösungsartefakte:

- **6-Punkte-Validierungscheckliste (im Entity Mapping Workbook):** Definiert einheitliche Qualitätskriterien für alle Konfigurationen: (1) Vollständigkeit (alle 6 Entitätstypen), (2) ID-Konsistenz (keine broken references), (3) Zeiteinheiten (nur Sekunden), (4) BOM-baseQuantity (Summe der Items), (5) Product-Material-Referenz (Produkt selbst, nicht Inhaltsstoffe), (6) ID-Eindeutigkeit (keine Duplikate).

- **Master Creator mit automatischer Validierung:** Implementiert die 6-Punkte-Checkliste automatisch. BOM-baseQuantity wird auto-korrigiert, Zeiteinheiten werden konvertiert, Referenzen werden geprüft. Verhindert systematisch die häufigsten Fehlerquellen.

- **Standard Operating Procedures:** Einheitliches Vorgehen für alle Konfigurationsprojekte. Systematische Validierung vor Import.

Validierung: Das kontrollierte Experiment (siehe Abschnitt 7.6) zeigt Fehlerreduktion um 81% (5,00 → 0,94 Fehler pro Konfiguration) und Vollständigkeitsverbesserung auf 98,1% (86,1% → 98,1%).
Grad der Problemlösung: Vollständig gelöst für Standard-Validierung. *Einschränkungen:* (1) Lösung existiert noch nicht lange genug für ausgiebige Praxistests - Sonderfälle könnten auftreten. (2) Arithmetische Fehler des LLMs (z. B. Zeitberechnungen) können auftreten und erfordern manuelle Korrektur. (3) Code-Struktur ist bewusst minimalistisch (3 Dateien: app.js, index.html, style.css) als funktionaler MVP - für produktiven Einsatz müsste die Architektur um Versionierung, Tests und Wartungsinfrastruktur erweitert werden.

7.5 DATENHOHEIT

Problemcharakteristik: Kundenkonfigurationen enthalten sensible Produktionsdaten (Rezepturen, Kapazitäten, Schichtpläne) unter DSGVO- und NDA-Verpflichtungen. Cloud-basierte LLM-Automatisierung würde Datentransfer an Drittanbieter bedeuten und ist aus Compliance-Gründen nicht akzeptabel.

Lösungsartefakt:

- **Lokales 7B-Modell (qwen2.5-coder via Ollama):** Deployment on-premise, keine Datentransfers an Cloud-Anbieter. Erfüllt GDPR- und NDA-Anforderungen. Modellgröße resultiert aus Hardware-Beschränkung (Unified-Memory-Architektur, <16GB vRAM).

Validierung: Das kontrollierte Experiment (siehe Abschnitt 7.6) zeigt, dass trotz begrenzter Modellgröße 98,1% Vollständigkeit erreichbar sind. Dies validiert die Designentscheidung, Datensouveränität über marginale Qualitätsgewinne größerer Cloud-Modelle zu priorisieren.
Grad der Problemlösung: Vollständig gelöst. Lokales Deployment erfüllt Compliance-Anforderungen bei funktionaler Performance (90-95% Vollständigkeit).

7.6 QUANTITATIVE PROBLEMEVALUATION

Problemcharakteristik: Hohe Fehlerrate (10-15 pro Iteration), geringe Vollständigkeit (60-70% Checklisten-Erfüllung analysierter Konfigurationen), lange Durchlaufzeit (8 Wochen Kalenderlaufzeit für initiale Konfiguration).
Lösungsartefakt: Der Master Creator kombiniert alle vorherigen Artefakte (Fragebogen, Entity Mapping Workbook, SOPs) in einer LLM-gestützten Webanwendung zur konversationellen Generierung von GRACE-Konfigurationsentitäten. Folgt Scaffolding-Ansatz (80/20-Prinzip): Automatisierte Basis-Konfiguration mit gezielter manueller Nacharbeit (Details siehe Abschnitt 6.5).
Zwei-Stufen-Verbesserung: Die Vollständigkeitssteigerung von 60-70% (Ist-Analyse, gemessen an analysierten Konfigurationen) auf 98,1% (Experiment) resultiert aus zwei synergistischen Effekten: (1) Strukturierung der Anforderungserhebung durch Drei-Artefakte-Architektur (+16-26 Prozentpunkte auf 86,1%, siehe GRACE-UI-Baseline im Experiment), (2) LLM-gestützte Automatisierung (+12 Prozentpunkte auf 98,1%, siehe Master Creator im Experiment). Dies demonstriert, dass systematische Wissensexternalisierung bereits ohne Automatisierung signifikante Verbesserungen ermöglicht.
Validierung - Kontrolliertes Experiment (n=18):
Die Evaluation erfolgte durch Within-Subjects-Design mit 3 Probanden, die jeweils 6 Konfigurationsaufgaben unter beiden Bedingungen durchführten: (1) manuelle Erstellung über GRACE-UI, (2) Master Creator-gestützte Erstellung. Aufgaben umfassten vereinfachte Produktionsumgebung (3 Materialien, 2 Maschinen, 1 Produkt, 1 BOM, 1 Prozess, 1 Rezept) mit realistischer Komplexität. Gemessen wurden drei Metriken: Zeit bis zur vollständigen Konfiguration, Anzahl Validierungsfehler beim Import, Erfüllung der 6-Punkte-Validierungscheckliste (Details siehe Abschnitt 5.6).
Zentrale Befunde:

- **Zeitreduktion:** 47% (8:03 Min → 4:18 Min) - Master Creator halbiert die Konfigurationszeit unter kontrollierten Bedingungen.

- **Fehlerreduktion:** 81% (5,00 → 0,94 Fehler pro Konfiguration) - automatische Validierung reduziert typische Fehlerquellen.

- **Vollständigkeit:** 86,1% → 98,1% (11,9 Prozentpunkte Verbesserung) - Checklisten-Erfüllung steigt durch systematische Validierung.

Durchlauf	MC Zeit	MC Fehler	MC Check	GRACE Zeit	GRACE Fehler	GRACE Check
1	5:45 Min	2	6/6	8:40 Min	4	6/6
2	4:45 Min	1	6/6	7:35 Min	3	6/6
3	4:05 Min	0	6/6	8:06 Min	5	6/6
4	3:40 Min	0	6/6	7:29 Min	3	6/6
5	3:20 Min	1	6/6	7:44 Min	3	6/6
6	2:45 Min	0	6/6	7:10 Min	1	6/6
Ø Proband 1	**4:03 Min**	**0,67**	**6,00**	**7:47 Min**	**3,17**	**6,00**

Tabelle 5: Experimentdaten Proband 1 (n=6)

Durchlauf	MC Zeit	MC Fehler	MC Check	GRACE Zeit	GRACE Fehler	GRACE Check
1	6:20 Min	3	6/6	9:50 Min	6	5/6
2	5:15 Min	1	6/6	9:15 Min	4	6/6
3	4:51 Min	1	6/6	9:30 Min	7	6/6
4	5:02 Min	0	6/6	8:46 Min	4	4/6
5	4:13 Min	0	6/6	8:20 Min	3	6/6
6	3:55 Min	1	6/6	8:06 Min	3	6/6
Ø Proband 2	**4:56 Min**	**1,00**	**6,00**	**8:58 Min**	**4,50**	**5,50**

Tabelle 6: Experimentdaten Proband 2 (n=6)

Durchlauf	MC Zeit	MC Fehler	MC Check	GRACE Zeit	GRACE Fehler	GRACE Check
1	5:14 Min	1	6/6	13:12 Min	12	2/6
2	3:53 Min	1	6/6	7:09 Min	7	2/6
3	3:59 Min	1	5/6	6:53 Min	9	4/6
4	3:03 Min	2	5/6	5:57 Min	6	5/6
5	4:14 Min	1	6/6	5:53 Min	6	5/6
6	2:58 Min	1	6/6	5:26 Min	4	6/6
Ø Proband 3	**3:53 Min**	**1,17**	**5,67**	**7:25 Min**	**7,33**	**4,00**

Tabelle 7: Experimentdaten Proband 3 (n=6)

Metrik	GRACE-UI	Master Creator
Zeit pro Konfiguration	8:03 Min (±1:44)	4:18 Min (±0:58)
Fehler pro Konfiguration	5,00 (±2,54)	0,94 (±0,78)
Checklisten-Punkte	5,17/6 (±1,00)	5,89/6 (±0,19)
Vollständigkeit	86,1%	98,1%

Tabelle 8: Kombinierte Statistik aller Probanden (n=18)

- **Lerneffekt:** Alle Probanden zeigen Zeitverbesserung über 6 Durchläufe (Proband 1: 5:45 → 2:45 Min, Proband 2: 6:20 → 3:55 Min, Proband 3: 5:14 → 2:58 Min).

Grad der Problemlösung: Quantitativ validiert für experimentelles Setting. Werte sind statistisch valide (n=18, Within-Subjects Design), jedoch nicht direkt auf Real-World-Konfigurationen übertragbar.
BPMN-Limitation: Zeitangaben in BPMN-Prozessmodellen (Kapitel 6) basieren auf realen Projekten, konnten jedoch nicht über multiple unabhängige Durchläufe quantitativ validiert werden. Bearbeitungszeit von drei Monaten ermöglichte keine Konfiguration multipler Kundenwerke unter kontrollierten Bedingungen. Bewusste Priorisierung des standardisierten Experiments (n=18).

7.7 NAS-CRASH-REKONSTRUKTION

Neben dem kontrollierten Experiment ergab sich eine ungeplante Validierungsmöglichkeit unter realen Bedingungen: Ein NAS-Crash führte zum vollständigen Verlust der Demo-Werk-Konfiguration (Pigmentpaste-Produktion mit 27 Materialien, 8 Maschinen, 3 Produkten, 6 Prozessen, 3 BOMs, 3 Rezepten). Die ursprüngliche manuelle Erstellung hatte 2 Monate Kalenderlaufzeit benötigt. Die Rekonstruktion mittels Master Creator erreichte 80% der Konfiguration in 20% der ursprünglichen Arbeitszeit, manuelle Nacharbeit für komplexe Strukturen (Prozess-Links, variable Formeln) erfolgte im verbleibenden Zeitraum.
Validierungswert: Diese explorative Evidenz (n=1, kein kontrolliertes Setting) ergänzt das standardisierte Experiment durch Nachweis der Praxistauglichkeit unter realen Produktionsbedingungen. Im Gegensatz zur vereinfachten Experimentumgebung (3 Materialien, 2 Maschinen, 1 Produkt) demonstriert die NAS-Crash-Rekonstruktion Skalierbarkeit auf komplexere Konfigurationen mit multiplen Rezepten und Prozess-Interdependenzen. Die erfolgreiche Rekonstruktion war vermutlich nur durch teilweise vorhandene JSON-Konfigurationen als Referenz möglich, die als externalisierte Wissensträger dienten. Dies bestärkt die Kernthese: Externalisierung (in welcher Form auch immer) als notwendige Voraussetzung für erfolgreiche LLM-Automatisierung.
Methodische Einordnung: Die fehlende statistische Generalisierbarkeit (Einzelfall ohne Wiederholbarkeit) erlaubt keine quantitative Verallgemeinerung. Die Evidenz ist explorativ und illustrativ, nicht konfirmatorisch. Sie dient als Plausibilitätsprüfung der experimentellen Befunde unter Realbedingungen.

7.8 LIMITATIONEN

Die Ergebnisse unterliegen mehreren methodischen, technischen und konzeptionellen Einschränkungen:
Technische Limitierungen:

- **Hardware-Beschränkung:** Das 7B-Modell resultiert aus der Unified-Memory-Architektur (<16GB vRAM). Größere Modelle könnten bessere Ergebnisse liefern, sind aber mit aktueller Hardware nicht lokal deploybar. Die Designentscheidung

priorisiert Datensouveränität (GDPR/NDA) über marginale Qualitätsgewinne größerer Cloud-Modelle.

- **Arithmetische Fehler des LLMs:** Zeitberechnungen und quantitative Transformationen können fehlerhaft sein und erfordern manuelle Korrektur. Dies ist eine bekannte Limitation kleinerer Modelle, die durch explizite Validierungsregeln teilweise kompensiert wird.

- **Code-Struktur bewusst minimalistisch:** Die Master Creator-Implementierung besteht aus 3 Dateien (app.js, index.html, style.css) als funktionaler MVP. Dies ermöglichte schnelle Iteration während der Entwicklung. Für produktiven Einsatz müsste die Architektur um Versionierung, Test-Standards und Wartungsinfrastruktur erweitert werden.

- **UUID-Format fehlt:** Das Entity Mapping Workbook dokumentiert aktuell kebab-case IDs, GRACE nutzt jedoch UUID als Standardkonvention. Diese Diskrepanz sollte in zukünftigen Versionen behoben werden.

Methodische Limitierungen:

- **Begrenzte statistische Generalisierbarkeit:** Das kontrollierte Experiment (n=18) liefert valide Werte für das experimentelle Setting, ist jedoch nicht direkt auf Real-World-Konfigurationen übertragbar. Die NAS-Crash-Rekonstruktion ist ein Einzelfall ohne statistische Generalisierbarkeit. Die Fragebogen-Validierung über 2 Kundenprojekte ist explorativ, nicht konfirmatorisch.

- **Demo-Werk-Charakteristik:** Das Demo-Werk diente als Testbed für Artefakt-Entwicklung. Die gemessene Zeitreduktion bei Neukonfigurationen könnte optimistisch sein, da das System auf Basis dieser Daten optimiert wurde.

- **GRACE-Pilotphase:** Evolvierende JSON-Schemata erfordern kontinuierliche Anpassung des System Prompts. Die langfristige Wartbarkeit ist noch nicht validiert. Zukünftige Schema-Änderungen könnten Prompt-Anpassungen und Validierungserweiterungen erforderlich machen.

Konzeptionelle Limitierungen:

- **Scaffolding-Ansatz erfordert Nacharbeit:** Komplexere Strukturen (Prozess-Links, itemAssociationRules, variable Prozessformeln) erfordern manuelle Nacharbeit und Domänenwissen. Diese Strukturen sind durch smartes Prompting generierbar, jedoch noch nicht im System Prompt integriert. Die Integration erfordert iterative Entwicklung und Validierungserweiterung.

- **Sonderfälle nicht vollständig abgedeckt:** Die 6-Punkte-Validierungscheckliste deckt Standard-Validierung ab, aber Sonderfälle könnten in zukünftigen Projekten auftreten. Die Lösung existiert noch nicht lange genug für ausgiebige Praxistests unter diversen Produktionsszenarien.

- **Vollständigkeits-Metrik zirkulär:** Der Fragebogen definiert gleichzeitig das Maß für Vollständigkeit (15 Kernfragen = 100%). Dies erschwert die objektive Messung, da die Baseline (60-70% in Ist-Analyse) aus Analyse realer Konfigurationen stammt, während die neue Metrik (Fragebogen-Vollständigkeit) systematisch erfasst wird. Die Vergleichbarkeit ist eingeschränkt, da unterschiedliche Messmethoden verwendet wurden.

7.9 KRITISCHE REFLEXION

Das kontrollierte Experiment zeigt eine Zeitreduktion von 47% bei standardisierter Konfiguration. Diese Verbesserung resultiert aus dem Zusammenspiel von Wissenssystematisierung (Drei-Artefakte-Architektur) und LLM-Automatisierung. Eine Isolierung der Einzeleffekte war methodisch nicht möglich, da beide Komponenten simultan entwickelt wurden. Die Drei-Artefakte-Architektur ist jedoch notwendige Voraussetzung: Erst durch die Erstellung des Entity Mapping Workbooks konnten die Prompts um die erforderliche Tiefe erweitert werden, sodass der Master Creator Daten generiert, die tatsächlich funktionelle Strukturen für GRACE abbilden. Ohne diese systematische Wissensaufbereitung wären die generierten Entitäten nicht direkt verwendbar.
Die Baseline von 2 Monaten (manuelle Konfiguration, siehe Ist-Analyse) reflektiert den realistischen Aufwand unter GRACE-Pilotbedingungen: fragmentiertes Wissen über Azure DevOps Wikis, E-Mails, Git-Historien und 10 Jahre Praxiserfahrung verteilt. Die Zeitreduktion umfasst daher sowohl die Wissenssystematisierung als auch die LLM-Automatisierung, beide Komponenten sind untrennbar verbunden. Das LLM allein würde ohne strukturierte Wissensaufbereitung keine vergleichbaren Reduktionsraten erzielen. Dies unterstreicht, dass die Drei-Artefakte-Architektur keine optionale Ergänzung, sondern eine notwendige Voraussetzung für erfolgreiche LLM-Automatisierung darstellt.

7.10 EINORDNUNG IN DEN STAND DER FORSCHUNG

Die vorliegende Arbeit positioniert sich an der Schnittstelle dreier aktiver Forschungsfelder: (1) LLM-gestützte Automatisierung in Operations Research, (2) Wissensexternalisierung in Organisationen, (3) lokale Deployment-Strategien unter Datensouveränitätsanforderungen. Die Anwendung von Large Language Models in industriellen Konfigurationsprozessen befindet sich am Puls aktueller Forschung, siehe Fan et al. [2024] und Yu et al. [2025] erschienen beide 2024/2025 und adressieren LLM-Integration in OR-Projekten.
LLMs in Operations Research: Fan et al. [2024] identifizieren LLMs als transformative Technologie für den gesamten OR-Projektzyklus (Operations Research). SmartAPS Yu et al. [2025] demonstriert LLM-gestützte Analysen in der *operativen Produktionsplanung* bereits konfigurierter Systeme (85-90% Zeitreduktion). Der Master Creator adressiert die *vorgelagerte Konfigurationsphase* (Entitätserstellung, referenzielle Integrität) und erreicht 47% Zeitreduktion im kontrollierten Experiment (n=18, 8:03 Min → 4:18 Min). Beide Ansätze validieren unabhängig voneinander, dass tool-augmented LLMs signifikante Effizienzgewinne in unterschiedlichen APS-Phasen ermöglichen. Wasserkrug et al. [2024] zeigen Limitationen von LLMs für komplexe Optimierungsmodellie-

rung (arithmetische Fehler, Constraint-Inkonsistenzen), adressiert durch Scaffolding-Ansatz (80/20-Prinzip: LLM-Basis + manuelle Nacharbeit).

Wissensexternalisierung: Die Drei-Artefakte-Architektur operationalisiert Nonaka and Takeuchi [1995]s SECI-Modell: Externalisierung impliziten Konfigurationswissens in LLM-konsumierbare Strukturen. Levallet and Chan [2019] zeigen Wissensverlust bei Fluktuation ohne Externalisierung. Die Ist-Analyse analysierter Konfigurationen bestätigt dies mit 60-70% Checklisten-Vollständigkeit bei fragmentiertem Wissen. Die Zwei-Stufen-Verbesserung (Strukturierung +16-26 Prozentpunkte, LLM-Automatisierung +12 Prozentpunkte) demonstriert, dass Wissensexternalisierung notwendige Voraussetzung für erfolgreiche LLM-Automatisierung darstellt.

Datensouveränität: Die Wahl eines lokalen 7B-Modells (qwen2.5-coder) anstelle cloudbasierter Alternativen reflektiert DSGVO- und NDA-Anforderungen. Hui [2024] zeigen, dass 7B-Modelle für strukturierte Datengeneration produktionsreife Performance erreichen. Die erreichte Vollständigkeit (90-95%) validiert diese Designentscheidung. Gupta et al. [2025] identifizieren Performanceunterschiede (English 97,6% vs. German 92%) in kleineren Modellen - Begründung für englischsprachige Prompts.

Beitrag dieser Arbeit: Im Vergleich zum Stand der Forschung werden drei Aspekte adressiert: (1) *Konzeptionell:* Drei-Artefakte-Architektur zur Transformation impliziten Expertenwissens als LLM-Enabler. (2) *Technisch:* Nachweis referenzieller Integrität über 6 Entitätstypen mit lokalem 7B-Modell unter Datensouveränitätsbedingungen. (3) *Empirisch:* Quantifizierung zweistufiger Verbesserungen (Strukturierung vs. LLM-Automatisierung) mit kontrolliertem Experiment (n=18, Within-Subjects Design). Die Übertragbarkeit wird durch Generalisierbarkeit des Ansatzes (strukturierte Prompts, validierungsbasierte Fehlerreduktion, Session-Context-Management) gestützt.

8 FAZIT

Die Pilotphase von GRACE befindet ist mit einer großen Herausforderung verbunden: Konfigurationswissen ist über mehrere Systeme fragmentiert verteilt. Die Ist-Analyse zeigt erhebliche Defizite. Die Vollständigkeit der Initialkonfigurationen beträgt lediglich 60-70%. Durchschnittlich treten 10-15 Fehler pro Konfiguration auf. Die Durchlaufzeit beträgt ca. 8 Wochen. Es existiert keine externalisierte Wissensquelle. Wissensträgerabhängigkeit besteht bei wenigen Experten. Validierungsmechanismen besitzen keine strukturierte Form. Unter diesen erschwerten Bedingungen entwickelt die Arbeit eine Drei-Artefakte-Architektur. Diese externalisiert implizites Expertenwissen. Die Architektur umfasst drei Komponenten: GRACE Onboarding Fragebogen (96 → 15 Kernfragen), Entity Mapping Workbook (JSON-Schemas für 6 Entitätstypen durch Reverse Engineering) und Standard Operating Procedures.

Trotz der schwierigen Ausgangslage erreicht der aus den Artefakten entwickelte Master Creator im kontrollierten Experiment (n=18) beeindruckende Verbesserungen. Die Zeitreduktion beträgt 47% (8:03 Min → 4:18 Min). Die Fehlerreduktion beträgt 81% (5,00 → 0,94 Fehler). Die Vollständigkeit steigt von 86,1% auf 98,1%. Die zweistufige Verbesserung zeigt die Wirkungszusammenhänge deutlich. Strukturierung alleine führt zu einem Zuwachs von 16-26 Prozentpunkten (von 60-70% auf 86,1%). LLM-

Automatisierung verstärkt die Wirkung um weitere 12 Prozentpunkte auf 98,1%. Das zur Entwicklung genutzte 7B-Modell erfüllt Datensouveränitätsanforderungen (DSGVO, NDA). Es erreicht trotz Hardware-Limitationen referenzielle Integrität über 6 Entitätstypen. Die strategische Flexibilität (Self-Service, iterative Nutzung, Importer-Integration) ermöglicht Anpassung an heterogene Kundenanforderungen.
Die Arbeit unterliegt methodischen und technischen Einschränkungen. Der Fragebogen definiert gleichzeitig das Maß für Vollständigkeit. Dies kann zu zirkulärer Metrik führen. Die Qualität der Informationen ist jedoch durch systematische Externalisierung objektiv gestiegen. Vormals implizites Wissen liegt nun strukturiert dokumentiert vor. Technische Limitationen bestehen in der ID-Format-Diskrepanz zwischen Tool (kebab-case) und GRACE-Standard (UUID). Der Master Creator fokussiert zudem auf 6 Kern-Entitäten. Komplexere Strukturen wie itemAssociationRules oder variable Prozessformeln sind durch smartes Prompting generierbar. Diese sind jedoch noch nicht vollständig integriert. Die genannten Einschränkungen schmälern nicht den Kernbeitrag. Die Arbeit validiert erfolgreich LLM-gestützte Konfigurationsansätze für APS-Systeme unter realen Produktionsbedingungen. Wie im Abstract erwähnt demonstriert dies die Notwendigkeit systematischer Wissensexternalisierung als Enabler für LLM-Automatisierung.

9 AUSBLICK

Die Ergebnisse eröffnen Weiterentwicklungspotenziale. Technisch ermöglicht die Evaluation größerer lokaler Modelle (13B, 70B) weitere Qualitätssteigerungen. Dies setzt erweiterte Hosting-Infrastrukturen voraus. Die Integration komplexer Strukturen wie `itemAssociationRules` oder variable Prozessformeln ist bereits validiert. Diese kann systematisch ausgebaut werden. Automatische Dateisystem-Scanner für ERP/MES-Datenextraktion würden den Self-Service-Ansatz weiter stärken. Die Erweiterung auf zusätzliche Entitätsstrukturen sollte als nächstes angestrebt werden. Dies ermöglicht komplexere Konfigurationen. Der Self-Service-Ansatz mit Master Creator ermöglicht Skalierung auf größere Kundenzahlen. Eine proportionale Teamvergrößerung ist nicht erforderlich. Die strategische Flexibilität optimiert Time-to-Market. Drei Nutzungsformen stehen zur Verfügung: reine Master Creator-Nutzung, iterative Entwicklung oder nachgelagerte Importer-Integration. Diese passen sich unterschiedlichen Kundenkapazitäten und Datenlagen an.
Die konzeptuelle Trennung zwischen domänenunabhängiger Methodik und GRACE-spezifischen Inhalten legt Transferierbarkeit nahe. Die Drei-Artefakte-Architektur ist auf andere Enterprise-Systeme übertragbar. Fragebogen, Entity Mapping und SOPs können inhaltlich angepasst werden. Anwendungsfelder umfassen SAP-Customizing, MES-Setup oder CRM-Konfiguration. Eine weiterführende Vision erschließt sich durch Verknüpfung dreier Automatisierungsebenen. Erste Ebene: Automatisierte Datenerhebung durch COSIMA (Datenraumadapter, aktuell in Entwicklung bei seitcom GmbH). Kundensysteme werden auf Dateisystemebene nach APS-relevanten Informationen durchsucht und strukturiert. Zweite Ebene: Automatisierte Konfiguration durch Master Creator. Dieser generiert GRACE-Entitäten konversationell aus strukturierten Daten. Dritte

Ebene: Automatisierte APS-Planung durch GRACE. Das System führt KI-gestützte Produktionsplanung auf Basis der generierten Konfigurationen durch. Diese End-to-End-Automatisierung würde den gesamten APS-Lebenszyklus abdecken. Der Zyklus reicht von der initialen Datenerhebung über die Konfigurationsgenerierung bis zur operativen Planberechnung. Die Abhängigkeit von OR-Consultants würde auf Qualitätssicherung und Sonderfallerkennung reduziert werden.

LITERATURVERZEICHNIS

Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, et al. Guidelines for human-ai interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 2019. doi: 10.1145/3290605.3300233. Grundlagenwerk für das Design von KI-Schnittstellen.

Thomas Brock, Michael Groteklaes, and Peter Mischke. *Lehrbuch der Lacktechnologie*. Vincentz Network, Hannover, 4 edition, 2020. ISBN 978-3-74860-156-4.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Markus Bühner. *Einführung in die Test- und Fragebogenkonstruktion*. Pearson Studium, 2004. ISBN 978-3827370471.

Peter Pin-Shan Chen. The entity-relationship model: toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36, 1976. doi: 10.1145/320434.320440.

Robert F. DeVellis. *Scale development: theory and applications*. SAGE Publications, 4th edition, 2017. ISBN 978-1506341569.

Nicola Döring and Jürgen Bortz. *Forschungsmethoden und Evaluation in den Human- und Sozialwissenschaften*. Springer, Berlin, Heidelberg, 6. edition, 2023. ISBN 978-3-662-64761-5. doi: 10.1007/978-3-662-64762-2.

Europäisches Parlament und Rat. Datenschutz-grundverordnung (dsgvo). Verordnung (EU) 2016/679, 2018.

Zhenan Fan, Bissan Ghaddar, Xinglu Wang, Linzi Xing, Yong Zhang, and Zirui Zhou. Artificial intelligence for operations research: revolutionizing the operations research process. *arXiv preprint arXiv:2401.03244*, 2024.

Fonds der Chemischen Industrie. Lacke, farben und druckfarben: Was das leben bunt macht, 2018.

Artur Goldschmidt. *BASF Handbuch Lackiertechnik*. Farbe und Lack Edition. Vincentz Network, Hannover, 1 edition, 2019. ISBN 978-3-74860-039-8.

gothesis.de. Der methodikteil in der bachelorarbeit. https://gothesis.de/bachelorarbeit/methodikteil/, 2024.

Hans-Otto Günther and Horst Tempelmeier. *Produktion und Logistik: Supply Chain und Operations Management*. Springer Vieweg, Berlin, Heidelberg, 12 edition, 2020. ISBN 978-3-66262-513-7. doi: 10.1007/978-3-662-62514-4.

Akshay Gupta et al. Multilingual performance gaps in large language models: English 97.6% vs lower-resource languages. *arXiv preprint arXiv:2504.17720v2*, 2025.

Binyuan et al. Hui. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Patrick Hum and RJ Marsan. Shipping sora for android with codex. OpenAI Blog, Dezember 2025. URL https://openai.com/de-DE/index/shipping-sora-for-android-with-codex/. Mit Codex dauerte es nur 28 Tage, bis Sora auf Android lief.

Patrik Hummel, Matthias Braun, Max Tretter, and Peter Dabrock. Data sovereignty: a review. *Big Data & Society*, 8(1), 2021. doi: 10.1177/2053951720982012.

International Organization for Standardization. Information technology - object management group business process model and notation (bpmn). Standard ISO/IEC 19510:2013, International Organization for Standardization, Genève, CH, 2013.

Paul Jackson. Capturing, structuring and maintaining knowledge: a social software approach. *Industrial Management & Data Systems*, 110(6):908–929, 2010. doi: 10.1108/02635571011055117.

Nadège Levallet and Yolande E. Chan. Organizational knowledge retention and knowledge loss. *Journal of Knowledge Management*, 23(1):176–199, 2019. doi: 10.1108/JKM-05-2018-0294. URL https://doi.org/10.1108/JKM-05-2018-0294.

Kurt Lewin. Action research and minority problems. *Journal of Social Issues*, 2(4):34–46, 1946. doi: 10.1111/j.1540-4560.1946.tb02295.x.

Hanqing Li and Diego Klabjan. Reverse prompt engineering, 2025. URL https://arxiv.org/abs/2411.06729.

Philipp Mayring. *Qualitative Inhaltsanalyse: Grundlagen und Techniken*. Beltz, 12. auflage edition, 2015. ISBN 978-3407257307.

Bernhard Müller and Walter Poth. *Lackformulierung und Lackrezeptur: Das Lehrbuch für Ausbildung und Praxis*. Vincentz Network, Hannover, 3 edition, 2017. ISBN 978-3-74860-107-6.

Ikujiro Nonaka and Hirotaka Takeuchi. *The knowledge-creating company: how Japanese companies create the dynamics of innovation*. Oxford University Press, 1995. ISBN 978-0195092691.

Object Management Group. *Unified Modeling Language (UML) specification*. Object Management Group, 2017. URL https://www.omg.org/spec/UML/2.5.1/.

Ollama Team. Ollama: get up and running with large language models locally. https://ollama.com, 2024.

Michael L. Pinedo. Scheduling: Theory, algorithms, and systems. *Springer*, 2016. doi: 10.1007/978-3-319-26580-3.

Michael Polanyi. *The Tacit Dimension*. University of Chicago Press, 1966. ISBN 978-0226672984.

Peter Reason and Hilary Bradbury. *Handbook of Action Research: Participative Inquiry and Practice*. SAGE Publications, 2001. ISBN 978-0761966647.

REFA Verband für Arbeitsgestaltung, Betriebsorganisation und Unternehmensentwicklung. *Methodenlehre der Betriebsorganisation: Grundlagen der Arbeitsgestaltung*. Carl Hanser Verlag, München, 2012. ISBN 978-3-44642-993-2.

seitcom GmbH. APS-Interviewskript: Marktforschung Produktionsplanung. Internes Dokument, seitcom GmbH, 2024a.

seitcom GmbH. Onboarding GRACE: 7-Schritte-Roadmap. Internes Dokument, seitcom GmbH, 2024b.

seitcom GmbH. GRACE Investor Pitch Deck: KI-gestützte Produktionsfeinplanung. Interne Präsentation, 2024c.

seitcom GmbH. GRACE Pitch Deck. Interne Präsentation, 2024d.

seitcom GmbH. GRACE Start Präsentation: Das Tool für energieeffiziente und zukunftssichere Produktionsplanung. Interne Präsentation, 2024e.

seitcom GmbH. Präsentation GRACE Test: Die erste KI-Produktionsfeinplanung mit bis zu 30% mehr Output bei 30% weniger Energieverbrauch. Interne Präsentation, 2024f.

seitcom GmbH. Fragensammlung Maschinenführer: Domänenwissen Farb- und Lackproduktion. Interne Dokumentation, 2025a.

seitcom GmbH. Gesprächstranskript: Entwicklung Fragenkatalog GRACE-Konfiguration. Internes Dokument, seitcom GmbH, Januar 2025b.

seitcom GmbH. GRACE Beispiel-Konfigurationen: Generische JSON-Templates. JSON-Konfigurationsdateien, 2025c.

seitcom GmbH. GRACE Onboarding Fragebogen - Kompaktversion (15 Fragen). Internes Dokument, seitcom GmbH, 2025d. Reduzierte Version für initiale Kundenworkshops nach DeVellis-Prinzipien.

seitcom GmbH. GRACE Onboarding Fragebogen - Vollversion (96 Fragen). Internes Dokument, seitcom GmbH, 2025e. Strukturierte Erhebung von GRACE-Konfigurationsanforderungen in 8 Teilen.

seitcom GmbH. BPMN-Prozessmodell v1a: Manuelle Konfiguration (vollständig). Anhang B: Unskalierte BPMN-Darstellung, 2026a. Vollständiger manueller Konfigurationsprozess über GRACE UI, Durchlaufzeit ca. 2 Monate.

seitcom GmbH. BPMN-Prozessmodell v1b: Automatisierter Import (vollständig). Anhang C: Unskalierte BPMN-Darstellung, 2026b. Automatisierter Import-Prozess mit Master Creator, Durchlaufzeit ca. 3,5 Wochen.

seitcom GmbH. BPMN-Prozessmodell v2: LLM-gestützte Konfiguration (vollständig). Anhang D: Unskalierte BPMN-Darstellung, 2026c. LLM-gestützte End-to-End-Konfiguration mit Master Creator, Durchlaufzeit ca. 4 Stunden (Demo-Werk).

seitcom GmbH. Master Creator System Prompt. Anhang A: Vollständiger System Prompt (ca. 400 Zeilen), 2026d. Enthält JSON-Strukturen, Constraints, Beispiele und Defaults für alle 6 Entity-Typen.

seitcom GmbH. Demo-Werk Pigmentpaste: Wiederhergestellte Konfiguration. JSON-Konfigurationsdateien, 2026.

Hartmut Stadtler, Christoph Kilger, and Herbert Meyr. *Supply Chain Management and Advanced Planning: Concepts, Models, Software, and Case Studies*. Springer, 5th edition, 2015. ISBN 978-3-642-55309-7. doi: 10.1007/978-3-642-55309-7.

Gerald I. Susman and Roger D. Evered. An assessment of the scientific merits of action research. *Administrative Science Quarterly*, 28(4):582–603, 1983. doi: 10.2307/2392618.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

Segev Wasserkrug, Leonard Boussioux, Dick den Hertog, Farzaneh Mirzazadeh, Ilker Birbil, Jannis Kurtz, and Donato Maragno. From large language models and optimization to decision optimization copilot: a research manifesto. *arXiv preprint arXiv:2402.16269*, 2024.

Timothy T. Yu, Mahdi Mostajabdaveh, Serge J. Byusa, Rindra Ramamonjison, Giuseppe Carenini, Kun Mao, Zirui Zhou, and Yong Zhang. Smartaps: tool-augmented llms for operations management. In *AAAI Conference on Artificial Intelligence - Workshop on Combining AI and OR/MS for Better Trustworthy Decision Making*. Huawei Technologies Canada, 2025.

Gianlucca Zuin, Saulo Martiello Mastelini, Túlio Loures, and Adriano Veloso. Leveraging large language models for tacit knowledge discovery in organizational contexts. *arXiv preprint arXiv:2501.12351*, 2025.

ANHANG

ÜBERSICHT: ORDNERSTRUKTUR DES DIGITALEN ANHANGS

Alle Anhangdateien befinden sich im Verzeichnis `/Anhang/` und sind wie folgt organisiert:

Artefakte/ Drei-Artefakte-Architektur (SOPs, Workbook, Kardinalitäten)

- `SOP_01_Kundenworkshop_durchfuehren.md`
- `SOP_02_Master_Creator_nutzen.md`
- `SOP_03_JSONs_in_GRACE_importieren.md`
- `GRACE_Entity_Mapping_Workbook.md`
- `GRACE_ENTITY_RELATIONSHIPS_AND_CARDINALITIES.md`
- `CARDINALITY-RELATIONS.md`

Fragebogen/ GRACE-Onboarding-Fragebogen (zwei Versionen)

- `GRACE_Onboarding_Fragebogen96.pdf` (Vollversion, 96 Fragen)
- `GRACE_Onboarding_Fragebogen15.pdf` (Kompaktversion, 15 Fragen)

master-creator/ Master Creator Quellcode (LLM-gestütztes Tool)

- `app.js` (Kernlogik: AI-Validierung, Auto-Erstellung)
- `index.html` (Chat-Interface + Entity-Visualisierung)
- `style.css` (UI-Styling)
- `README.md` (Dokumentation)

System-Prompt/ Master Creator System Prompt

- `Master_Creator_System_Prompt.txt`

JSONS_example/ Beispiel-Konfigurationen (Demo-Werk, n=18 Experiment)

- `materials_*.json` (7 Materials)
- `machine_resources_*.json` (3 Machines)
- `products_*.json` (3 Products)
- `production_processes_*.json` (6 Processes)
- `boms_*.json` (3 BOMs)
- `recipes_*.json` (3 Recipes)
- `processes.json` (Legacy-Format)

Restored_JSONS/ NAS-Crash-Rekonstruktion (n=1, explorativ)

- `materials_*.json` (27 Materials)
- `machine_resources_*.json` (8 Machines)
- `products_*.json` (3 Products)
- `production_processes_*.json` (6 Processes)
- `boms_*.json` (3 BOMs)
- `recipes_*.json` (3 Recipes)
- `properties_*.json` (Zusätzliche Eigenschaften)

demo-config/ Demo-Werk Konfiguration (vollständig)

- 14 JSON-Dateien (alle 6 Entity-Typen mit Backup-Versionen)
- `Production_Plan_*.csv` (Beispiel-Produktionsplan)

Prozesse/ BPMN-Diagramme (drei Prozessvarianten)

- `bpmn-v1a.pdf` (Vollständig manuelle Konfiguration)
- `bpmn-v1b.pdf` (Konfiguration mit Importer)
- `bpmn-v2.pdf` (LLM-gestützte Konfiguration)

ER-Diagramm/ Entity-Relationship-Diagramm

- `ER_UML_Card.pdf` (6 GRACE-Entitäten mit Kardinalitäten)

Architektur/ Master Creator Architektur-Diagramm

- `Master_Creator_Arch.pdf`

Transkript/ Gesprächstranskript Fragebogen-Validierung

- `Gespraechstranskript_Fragenkatalog_2025-01-15.pdf`

APS-Interviewskript/ Interviewleitfaden für APS-Konfiguration

- `APS-Interviewskript.pdf`

Fragensammlung_Maschinenführer/ Zusätzliche Maschinenführer-Fragen

- `Frage_antworten.pdf`

Präsentationen/ GRACE-Präsentationen und Pitch Decks

- `GRACE Pitch Deck.pdf`
- `GRACE Start Präsentation.pdf`
- `Onboarding_Grace.pdf`
- 2 weitere Investor-Präsentationen